Manual técnico

Sistema de Métodos Numéricos Iterativos

Arquitectura, despliegue, mantenimiento y extensión de la aplicación web

Sistema	Métodos Numéricos Iterativos (aplicación web full-stack)
Repositorio	https://github.com/Aryeisor/metodos-numericos
Autores	Miguel Ángel Madariaga Cantor Cristhian Orlando Robles Pico Dani Alejandro Carvajal Rabelo
Asignatura	Métodos numéricos
Docente	Dr. Geinner Giovanny Barbosa Casanova
Institución	Fundación De Estudios Superiores Comfanorte
Versión del documento	1.0
Fecha	13 de noviembre de 2026

Control de versiones del documento

Versión	Fecha	Autor	Descripción del cambio
1.0	2026-11-13	Miguel madariaga	Primera versión del manual técnico, elaborada a partir del código del commit `2408a2a`.

Tabla de contenido

1. Introducción 7

1.1 Propósito del documento 7

1.2 Alcance 7

1.3 Audiencia 8

1.4 Definiciones y acrónimos 8

2. Visión general del sistema 9

2.1 Descripción 9

2.2 Objetivos y requisitos del curso 10

2.3 Catálogo de métodos 11

2.4 Funcionalidades transversales 13

3. Stack tecnológico 14

3.1 Backend 14

3.2 Frontend 15

3.3 Infraestructura 16

3.4 Herramientas de desarrollo 17

3.5 Justificación de las decisiones 17

4. Arquitectura del sistema 18

4.1 Vista de contexto 18

4.2 Arquitectura en capas 19

4.3 Flujo de una petición de principio a fin 20

4.4 Patrones de diseño aplicados 21

4.5 Decisiones de arquitectura y sus consecuencias 25

4.6 Restricciones conocidas 26

5. Estructura de carpetas 27

6. Backend 30

6.1 Configuración de Django 30

6.2 Registro de métodos 32

6.3 Capa de vistas y rutas 33
6.4 Serializers 35
6.5 Solvers 37
6.6 Manejo simbólico de expresiones 43
6.7 Álgebra lineal 44
6.8 Datos de ejemplos 45
6.9 Manejo de errores 46
7. Frontend 47
7.1 Arranque de la aplicación 47
7.2 Enrutamiento 48
7.3 Cliente HTTP 48
7.4 Registro de interfaces y contrato de categoría 49
7.5 Vistas genéricas 52
7.6 Componentes compartidos 52
7.7 Composables 54
7.8 Utilidades 54
7.9 Sistema de temas 55
7.10 Menú de navegación 57
8. API REST 58
8.1 Convenciones 58
8.2 Catálogo de endpoints 58
8.3 Especificación de cada endpoint 59
8.4 Contrato de respuesta común 67
8.5 Ejemplos de uso 69
9. Modelo de datos 70
10. Seguridad 71
10.1 Superficie de ataque 71
10.2 Defensas del parser 71

10.3 Límites contra denegación de servicio 73
10.4 Configuración segura en producción 75
10.5 Pendientes de seguridad 75
11. Despliegue 76
11.1 Requisitos previos 76
11.2 Modo desarrollo 77
11.3 Modo Docker 78
11.4 Scripts de Windows 83
11.5 Variables de entorno 83
11.6 Compilación del frontend 85
11.7 Procedimiento de actualización 85
11.8 Acceso desde otro equipo de la red 86
12. Pruebas 86
12.1 Estrategia 86
12.2 Cómo ejecutarlas 87
12.3 Cobertura por archivo 87
12.4 Verificación manual 89
13. Rendimiento y límites 90
14. Guía de mantenimiento y extensión 92
14.1 Agregar un método a una categoría existente 92
14.2 Agregar una categoría nueva 93
14.3 Agregar una fórmula de derivación o una regla de integración 94
14.4 Agregar una página de teoría 94
14.5 Agregar ejemplos precargados 95
14.6 Convenciones de código 95
14.7 Lista de verificación antes de dar por terminado un cambio 96
15. Solución de problemas 97
16. Glosario 98

Anexo Comandos de referencia rápida 100

Anexo. Referencias 101

Documentación de las tecnologías 101

Bibliografía de métodos numéricos 102

1. Introducción

1.1 Propósito del documento

Este manual describe cómo está construido el sistema de Métodos Numéricos Iterativos, cómo se despliega, cómo se verifica y cómo se extiende. Su objetivo es que un desarrollador que no participó en el proyecto pueda, sin otra fuente, levantar el sistema en su equipo, entender el recorrido de una petición desde el navegador hasta el solver numérico y agregar un método, una fórmula o una categoría sin romper lo existente.

El documento describe el código del commit `2408a2a` de la rama `main`. Todas las rutas, nombres de clases y funciones, constantes y cifras que aparecen aquí se verificaron contra ese estado del repositorio. Cuando el `README.md` difiere del código, este manual sigue al código.

1.2 Alcance

Cubre:

- la arquitectura del backend (Django + Django REST Framework) y del frontend (Vue 3 + Vite);
- la API REST completa: rutas, cuerpos de petición y de respuesta, códigos de estado;
- las decisiones numéricas de cada familia de métodos (criterios de parada, tolerancias por defecto, mínimos y máximos);
- la seguridad del intérprete de expresiones matemáticas;
- el despliegue en modo desarrollo y en Docker, y los scripts de Windows;
- la suite de pruebas automatizadas y la verificación manual;
- los procedimientos de mantenimiento y extensión.

No cubre:

- la teoría matemática de cada método, que la aplicación expone en sus páginas «Teoría» (`/teoria/<slug>`);
- el uso de la aplicación por parte del estudiante (corresponde a un manual de usuario);
- la descripción línea por línea de cada componente visual; se documentan los contratos y los componentes compartidos.

1.3 Audiencia

Perfil	Necesidad	Secciones recomendadas
Desarrollador que hereda el proyecto	Entender la estructura y extender el sistema	4, 5, 6, 7, 14
Evaluador técnico o docente	Valorar la arquitectura, la calidad y las pruebas	2, 3, 4, 10, 12
Responsable de despliegue	Instalar, configurar y actualizar	3.3, 11, 15
Desarrollador de frontend	Consumir la API o crear interfaces	7, 8
Revisor de seguridad	Evaluar la exposición del intérprete de expresiones	10, 6.6

1.4 Definiciones y acrónimos

Término	Definición
API	*Application Programming Interface*. Conjunto de rutas HTTP que el backend expone al frontend.
REST	*Representational State Transfer*. Estilo de API basado en recursos, verbos HTTP y respuestas sin estado.
DRF	Django REST Framework, biblioteca para construir APIs sobre Django.
SPA	*Single Page Application*. Aplicación web que carga un solo documento HTML y cambia de vista en el cliente.
SFC	*Single File Component*. Archivo `.vue` que reúne plantilla, lógica y estilos de un componente.
ORM	*Object-Relational Mapper*. Capa de Django que traduce clases Python a tablas; el sistema no la usa para datos de dominio.
CORS	*Cross-Origin Resource Sharing*. Mecanismo del navegador que autoriza peticiones entre orígenes distintos.
CSRF	*Cross-Site Request Forgery*. Ataque que reutiliza la sesión del usuario; Django incluye un middleware contra él.
WSGI	*Web Server Gateway Interface*. Interfaz estándar entre un servidor HTTP de Python (gunicorn) y la aplicación (Django).
KaTeX	Biblioteca JavaScript que convierte LaTeX en HTML en el navegador.
LaTeX	Lenguaje de marcado de fórmulas; el backend lo genera como texto y el frontend lo dibuja con KaTeX.
slug	Identificador de un método en las URL (`jacobi`, `trapecio`, `cinco-puntos`).
solver	Función de Python que recibe datos validados y devuelve el resultado de un método.
serializer	Clase de DRF que valida y normaliza la entrada de un endpoint.
tolerancia	Umbral del error por debajo del cual un método iterativo se considera convergido.

Término	Definición
iteración	Una aplicación de la regla de actualización del método; en diferenciación e integración, una fila de la tabla de refinamiento.
criterio de parada	Condición que detiene el método: error ≤ tolerancia con un mínimo de filas, o el máximo de iteraciones.
norma infinito	$\| v \|_\infty = \max_i \|v_i\|$; el sistema la usa como error entre iteraciones de los métodos para sistemas.
error estimado	Diferencia entre dos aproximaciones sucesivas; no requiere conocer el valor exacto.
error verdadero (E_t)	Diferencia entre el valor exacto (o de referencia) y la aproximación.
cota	Máximo teórico del error de truncamiento de una fórmula, calculado con un máximo de la derivada correspondiente.
refinamiento	Reducción sistemática del paso (h a la mitad, n al doble) entre filas consecutivas.
`lambdify`	Función de sympy que convierte una expresión simbólica en una función numérica de Python.
composable	Función de Vue 3 que encapsula estado reactivo reutilizable (`useTheme`, `useExpressionPreviews`).
gunicorn	Servidor WSGI para Linux que ejecuta Django en Docker.
nginx	Servidor HTTP que, en Docker, entrega el frontend compilado y hace de proxy de `/api/`.

2. Visión general del sistema

2.1 Descripción

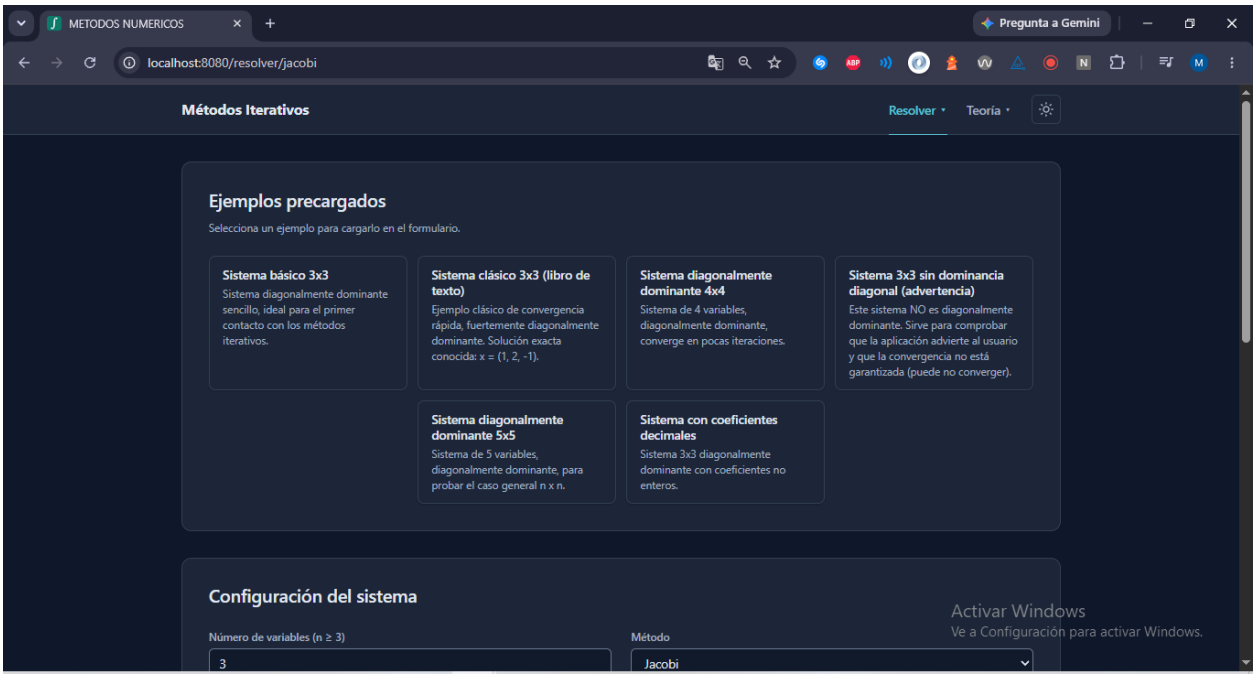
El sistema es una aplicación web full-stack que resuelve, paso a paso, problemas de cinco familias del análisis numérico: sistemas de ecuaciones lineales, sistemas de ecuaciones no lineales, raíces de polinomios, diferenciación numérica e integración numérica. Ofrece doce métodos. Para cada uno, el usuario ingresa los datos (una matriz, un sistema de ecuaciones escrito en texto, un polinomio, una función o una tabla de puntos), configura la tolerancia y el máximo de iteraciones, y obtiene la solución con la tabla completa de iteraciones, el detalle de cada paso con las sustituciones numéricas, gráficas de convergencia o de la aproximación, una comprobación del resultado y la exportación a PDF.

El sistema se divide en dos aplicaciones. El **backend**, escrito en Python con Django y Django REST Framework, valida la entrada, interpreta las expresiones matemáticas de forma segura con sympy, ejecuta los métodos y devuelve un JSON con todos los números ya calculados. El

frontend, una SPA en Vue 3 compilada con Vite, presenta los formularios, dibuja las fórmulas con KaTeX y las gráficas con Chart.js, y genera el PDF en el navegador con jsPDF. No hay base de datos de dominio ni usuarios: cada petición es independiente.

Además de la vista «Resolver», cada método tiene una página «Teoría» con la definición formal, el algoritmo, un ejemplo resuelto y referencias bibliográficas. Los ejemplos de las páginas de teoría de diferenciación e integración se guardan como respuestas reales del solver en archivos JSON, y las pruebas del backend verifican que coincidan con lo que el solver calcula hoy.

CAPTURA T-01 — Aplicación cargada en el navegador



Aplicación cargada en el navegador

2.2 Objetivos y requisitos del curso

El repositorio no contiene el enunciado formal de la asignatura. Los requisitos siguientes se infieren de las restricciones que el código impone de forma explícita, y se indica dónde se aplica cada uno.

Requisito	Implementación verificada	Ubicación
Definición formal de cada método	Docstring «Definición formal» en cada solver y página «Teoría» por método	backend/numeric_methods/solvers/linear/jacobi.py, backend/numeric_methods/solvers/nonlinear/newton.py, frontend/src/methods/*/theory/
Restricciones	Validación en	backend/numeric_methods/serializers/,

Requisito	Implementación verificada	Ubicación
de aplicabilidad	serializers y solvers (dominancia diagonal, diagonal sin ceros, grado del polinomio, espaciado uniforme, despeje único)	backend/numeric_methods/solvers/*/validation.py
Algoritmo paso a paso	Detalle de cada iteración (IterationStep.extra) y reconstrucción en el cliente para los lineales	backend/numeric_methods/solvers/base.py, frontend/src/utils/iterationSteps.js
Ejemplos precargados	Entre 3 y 9 ejemplos por método; 84 asignaciones método–ejemplo, 78 ejemplos distintos	backend/numeric_methods/examples_data.py
Precisión configurable	Campo tolerance en todos los métodos iterativos	Serializers de cada categoría
Mínimo de iteraciones	MIN_ITERATIONS = 6: ningún método declara convergencia antes de 6 iteraciones o filas	backend/numeric_methods/solvers/validation.py
Máximo de iteraciones configurable	Campo max_iterations (por defecto 100 en sistemas y Bairstow; rangos propios en diferenciación e integración)	Serializers de cada categoría
Soporte multivariable	Sistemas lineales de $n \geq 3$ variables; sistemas no lineales de 2 a 6 ecuaciones	backend/numeric_methods/solvers/linear/validation.py, backend/numeric_methods/solvers/nonlinear/common.py

2.3 Catálogo de métodos

El catálogo se lee de backend/numeric_methods/registry.py. Hay **12 métodos** en **5 categorías**. El orden es el del menú (CategorySpec.order y, dentro de cada categoría, el orden de declaración en METHODS).

Orden	Categoría (`category`)	Método	Slug	Qué resuelve
1	Sistemas lineales (`linear_system`)	Jacobi	`jacobi`	$A\mathbf{x} = \mathbf{b}$, $n \times n$ con $n \geq 3$; actualiza todas las variables con los valores de la iteración anterior.
1	Sistemas lineales	Gauss-Seidel	`gauss-seidel`	El mismo sistema; usa cada valor nuevo apenas se calcula.
2	Ecuaciones no lineales (`nonlinear_system`)	Punto Fijo (Iterativo Secuencial)	`punto-fijo`	$F(\mathbf{x}) = \mathbf{0}$ con 2 a 6 ecuaciones; despeja automáticamente cada ecuación con sympy.
2	Ecuaciones no lineales	Newton	`newton`	$F(\mathbf{x}) = \mathbf{0}$ con 2 a 6 ecuaciones; matriz jacobiana simbólica y regla de Cramer.
3	Polinomios (`polynomial`)	Bairstow	`bairstow`	Todas las raíces, reales y complejas, de un polinomio de grado 3 a 10, por factores $x^2 - rx - s$.
4	Diferenciación numérica (`differentiation`)	Diferencias progresiva y regresiva	`diferencias-finitas`	$f'(x_0)$ con error $O(h)$, desde $f(x)$ con refinamiento de h o desde una tabla de 2 a 20 puntos.
4	Diferenciación numérica	Fórmula de 3 puntos	`tres-puntos`	$f'(x_0)$ con punto medio y extremos, error $O(h^2)$; tabla de 3 a 20 puntos igualmente espaciados.
4	Diferenciación numérica	Fórmula de 5 puntos	`cinco-puntos`	$f'(x_0)$ con punto medio y extremos, error $O(h^4)$; tabla de 5 a 20 puntos igualmente espaciados.
5	Integración numérica (`integration`)	Método del trapecio	`trapecio`	$\int_a^b f\,(x)\,dx$, regla simple y compuesta con refinamiento de n; tabla de 2 a 30 puntos con espaciado libre.
5	Integración numérica	Método de Simpson	`simpson`	Reglas 1/3 y 3/8, simples y compuestas; tabla de 3 a 30 puntos igualmente espaciados.
5	Integración numérica	Integración de Romberg	`romberg`	Tabla triangular de trapecios y extrapolación de Richardson; tabla de 3, 5, 9, 17 o 33 puntos.
5	Integración numérica	Cuadratura	`gauss`	Gauss-Legendre de m

Orden	Categoría (`category`)	Método	Slug	Qué resuelve
		gaussiana		puntos, simple o compuesta en N subintervalos; solo modo función.

2.4 Funcionalidades transversales

Las funcionalidades siguientes son comunes a todos los métodos o a una categoría completa. Se implementan una sola vez y cada categoría las reutiliza a través del contrato de interfaz descrito en la sección 7.4.

Funcionalidad	Descripción	Implementación principal
Vista «Resolver»	Formulario, configuración (tolerancia, máximo de iteraciones), selector de método y de ejemplo, botones «Resolver» y «Limpiar», resultado. Una única vista para los 12 métodos.	`frontend/src/views/SolverView.vue`
Vista «Teoría»	Página de teoría por método, con definición formal, algoritmo, ejemplo resuelto y referencias. En Simpson, Romberg y cuadratura gaussiana, el botón «Resolver este ejemplo» abre `/resolver/<slug>?ejemplo=<id>`, que carga y resuelve el ejemplo.	`frontend/src/views/TheoryView.vue`, `frontend/src/methods/*/theory/`
Ejemplos precargados	Selector que carga un ejemplo del servidor en el formulario, con su tolerancia y su máximo de iteraciones.	`GET /api/examples/?method=<slug>`; `loadExample` de cada store
Comprobación	Sustitución de la solución en el sistema original, en el polinomio o en la antiderivada, calculada en el backend.	Clave `verification` de la respuesta; componentes `*Verification.vue`
Gráficas	Convergencia del error (escala logarítmica), aproximación de la derivada, reglas de	`frontend/src/components/ConvergenceChart.vue`, `frontend/src/components/TrapezoidChart.vue`, `frontend/src/methods/differentiation/DifferentiationChart.vue`

Funcionalidad	Descripción	Implementación principal
	integración.	
Exportación a PDF	Documento A4 generado en el navegador con los datos de entrada, el resultado, la gráfica y la tabla de iteraciones.	`frontend/src/utils/exportPdf.js` y `frontend/src/methods/*/pdf.js`
Botón «Limpiar»	Restablece el formulario al estado inicial del método; pide confirmación con un diálogo si hay datos o un resultado en pantalla, y descarta respuestas tardías.	`SolverView.vue` (`requestClear`, `clearExercise`, `solveRequestId`), `frontend/src/components/ConfirmDialog.vue`
Modo claro y oscuro	Tema según la preferencia del sistema operativo, con elección manual persistida en `localStorage`.	`frontend/src/composables/useTheme.js`, `frontend/src/assets/theme.css`, script en línea de `frontend/index.html`
Menú de navegación	Panel con las categorías en columnas, buscador sin tildes y modo móvil a pantalla completa; se genera desde el catálogo.	`frontend/src/components/NavMenu.vue`
Vista previa de ecuaciones	Mientras el usuario escribe, el backend interpreta la ecuación y devuelve su LaTeX o el error.	`POST /api/expressions/preview`; `frontend/src/composables/useExpressionPreviews.js`
Entrada con fracciones	Los campos numéricos aceptan decimales y fracciones (`6/7`).	`frontend/src/utils/numberInput.js`

3. Stack tecnológico

3.1 Backend

Dependencia	Versión	Origen de la versión	Uso en el sistema
Python	3.11.0 (desarrollo); 3.12.16 (Docker)	Entorno local; imagen `python:3.12-slim`	Lenguaje del backend.
Django	5.2.17	`backend/requirements.txt`	Framework web: configuración,

Dependencia	Versión	Origen de la versión	Uso en el sistema
			enrutamiento de URL, middleware, servidor de desarrollo.
djangorestframework	3.18.1	`backend/requirements.txt`	Vistas `APIView`, serializers de validación y respuestas JSON.
django-cors-headers	4.9.0	`backend/requirements.txt`	Cabeceras CORS para el modo desarrollo (Vite en el puerto 5173, Django en el 8000).
sympy	1.14.0	`backend/requirements.txt`	Parseo de expresiones, derivación simbólica, despejes (`solve`), antiderivadas (`integrate`), conversión a LaTeX y `lambdify`.
mpmath	1.3.0	Dependencia transitiva de sympy	Evaluación de funciones que el módulo `math` no tiene e integral de referencia de alta precisión (`mpmath.quad`).
gunicorn	23.0.0	`backend/Dockerfile` (`pip install ... gunicorn==23.0.0`)	Servidor WSGI en el contenedor. No figura en `requirements.txt` porque no funciona en Windows.
SQLite	Incluido en Python	`backend/core/settings.py`	Solo para las tablas internas de Django; no almacena datos del dominio.

3.2 Frontend

Las versiones de `frontend/package.json` usan el prefijo `^`; la columna «Instalada» indica la versión resuelta en `package-lock.json` y presente en `node_modules`.

Dependencia	Rango declarado	Instalada	Uso en el sistema
vue	^3.5.42	3.5.42	Framework de interfaz; componentes SFC con `<script setup>` y Composition API.
vue-router	^4.6.4	4.6.4	Rutas `/resolver/<slug>` y `/teoria/<slug>` con `createWebHistory`.
axios	^1.20.0	1.20.0	Cliente HTTP de la API (`frontend/src/api/client.js`).

Dependencia	Rango declarado	Instalada	Uso en el sistema
katex	^0.18.7	0.18.7	Dibujo de fórmulas LaTeX en `frontend/src/components/MathFormula.vue`.
chart.js	^4.5.1	4.5.1	Gráficas de convergencia, de la aproximación y de las reglas de integración.
jspdf	^4.2.1	4.2.1	Generación del PDF en el navegador.
jspdf-autotable	^5.0.8	5.0.8	Tablas dentro del PDF.
vite	^8.3.0	8.3.0	Servidor de desarrollo y compilación de producción.
@vitejs/plugin-vue	^6.0.8	6.0.8	Compilación de los archivos `.vue`.
Node.js	`^20.19.0 || >=22.12.0` (exigido por Vite 8)	22.20.0 (desarrollo); imagen `node:20-alpine` (Docker)	Entorno de ejecución de las herramientas de compilación.

3.3 Infraestructura

Componente	Versión	Configuración
Docker Engine	29.6.2 (equipo de desarrollo)	Construye y ejecuta las dos imágenes.
Docker Compose	v5.3.1 (equipo de desarrollo)	`docker-compose.yml`, servicios `backend` y `frontend`.
nginx	1.31.8 (imagen `nginx:alpine` en ejecución)	`frontend/nginx.conf`: archivos estáticos, proxy de `/api/`, gzip, caché.
gunicorn	23.0.0	3 workers, `--timeout 120`, escucha en `0.0.0.0:8000` dentro de la red de Compose.
Imagen base backend	`python:3.12-slim`	Usuario sin privilegios `app` (UID 1000).
Imagen base frontend	`node:20-alpine` (compilación) y `nginx:alpine` (servicio)	Compilación en dos etapas.

Las etiquetas `node:20-alpine` y `nginx:alpine` no fijan un parche concreto: cada reconstrucción con imágenes base nuevas puede traer una versión menor distinta. Las versiones de la tabla son las observadas el 13 de noviembre de 2026.

3.4 Herramientas de desarrollo

Herramienta	Versión observada	Uso
Git	2.47.0	Control de versiones; repositorio remoto en GitHub.
Visual Studio Code	No fijada	Editor; `frontend/.vscode/extensions.json` recomienda la extensión de Vue.
Vite	8.3.0	`npm run dev` (recarga en caliente en el puerto 5173), `npm run build`, `npm run preview`.
npm	10.9.3	Instalación de dependencias del frontend.
pip y venv	Incluidos en Python	Entorno virtual `backend/.venv` e instalación de `requirements.txt`.

3.5 Justificación de las decisiones

Django + Django REST Framework. El backend no necesita persistencia ni autenticación, pero sí validación de entrada estructurada, mensajes de error por campo y un enrutamiento que se genere a partir de datos. Los serializers de DRF cubren la validación declarativa y la traducción automática de errores a respuestas 400; las vistas `APIView` permiten una sola clase genérica para todos los métodos (`BaseSolveView`). Django aporta además un middleware de seguridad maduro y un servidor de desarrollo sin configuración.

Vue 3 con Composition API. Cada categoría encapsula su estado de formulario en una función fábrica (`createStore`) y su lógica reutilizable en composables. La Composition API permite que ese estado viva fuera de los componentes y que la vista genérica `SolverView.vue` lo reciba como dato, sin herencia de componentes. Los SFC mantienen juntas la plantilla, la lógica y los estilos de cada pieza.

sympy. Los usuarios escriben funciones y ecuaciones como texto. sympy ofrece en una sola biblioteca el parseo a un árbol de expresión, la derivación (jacobiano de Newton, derivadas de orden alto para las cotas), los despejes de Punto Fijo, las antiderivadas exactas y la conversión a LaTeX. La evaluación numérica repetida no usa sympy sino funciones compiladas con `lambdify`, lo que mantiene el costo por iteración en el orden de una llamada a `math`.

KaTeX en lugar de imágenes. El backend genera LaTeX como texto; el navegador lo convierte en HTML con KaTeX. Las fórmulas quedan seleccionables, escalan con el zoom, heredan los colores del tema claro u oscuro y no requieren un servicio de renderizado en el servidor. KaTeX renderiza de forma síncrona, sin el parpadeo de las alternativas asíncronas.

Chart.js. Las gráficas son de líneas y dispersión con ejes lineales o logarítmicos y hasta seis series. Chart.js cubre esos tipos con un solo paquete, dibuja en `<canvas>` (lo que permite exportar la imagen al PDF con una llamada) y lee los colores de las variables CSS del tema.

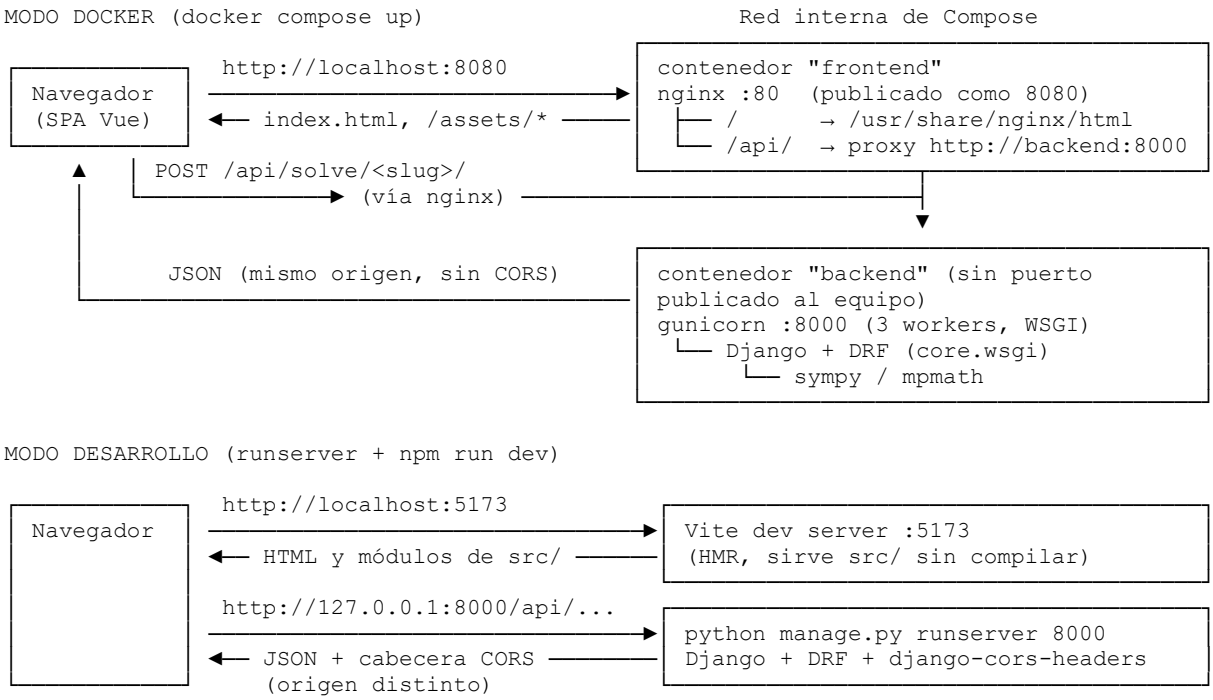
jsPDF en el cliente. El PDF se arma con datos que el navegador ya tiene: el resultado de la API y la imagen de la gráfica. Generarlo en el cliente evita un endpoint adicional, una dependencia de renderizado en el servidor y una segunda petición con la misma información. El backend sigue siendo una función pura de la entrada.

Docker. La aplicación se presenta en equipos distintos. Docker reduce la instalación a Docker Desktop y un comando, aísla las versiones de Python y Node, y reproduce una topología de producción (nginx + gunicorn, un solo origen, backend sin puerto publicado) que el modo desarrollo no ofrece. Una vez construidas las imágenes, la aplicación funciona sin conexión a internet.

4. Arquitectura del sistema

4.1 Vista de contexto

El sistema tiene dos modos de ejecución con la misma base de código. En **modo Docker** el navegador habla con un solo origen (nginx en el puerto 8080 del equipo) y nginx reenvía las rutas `/api/` al backend. En **modo desarrollo** el navegador carga la SPA desde el servidor de Vite y llama directamente a Django en otro puerto, por lo que interviene CORS.

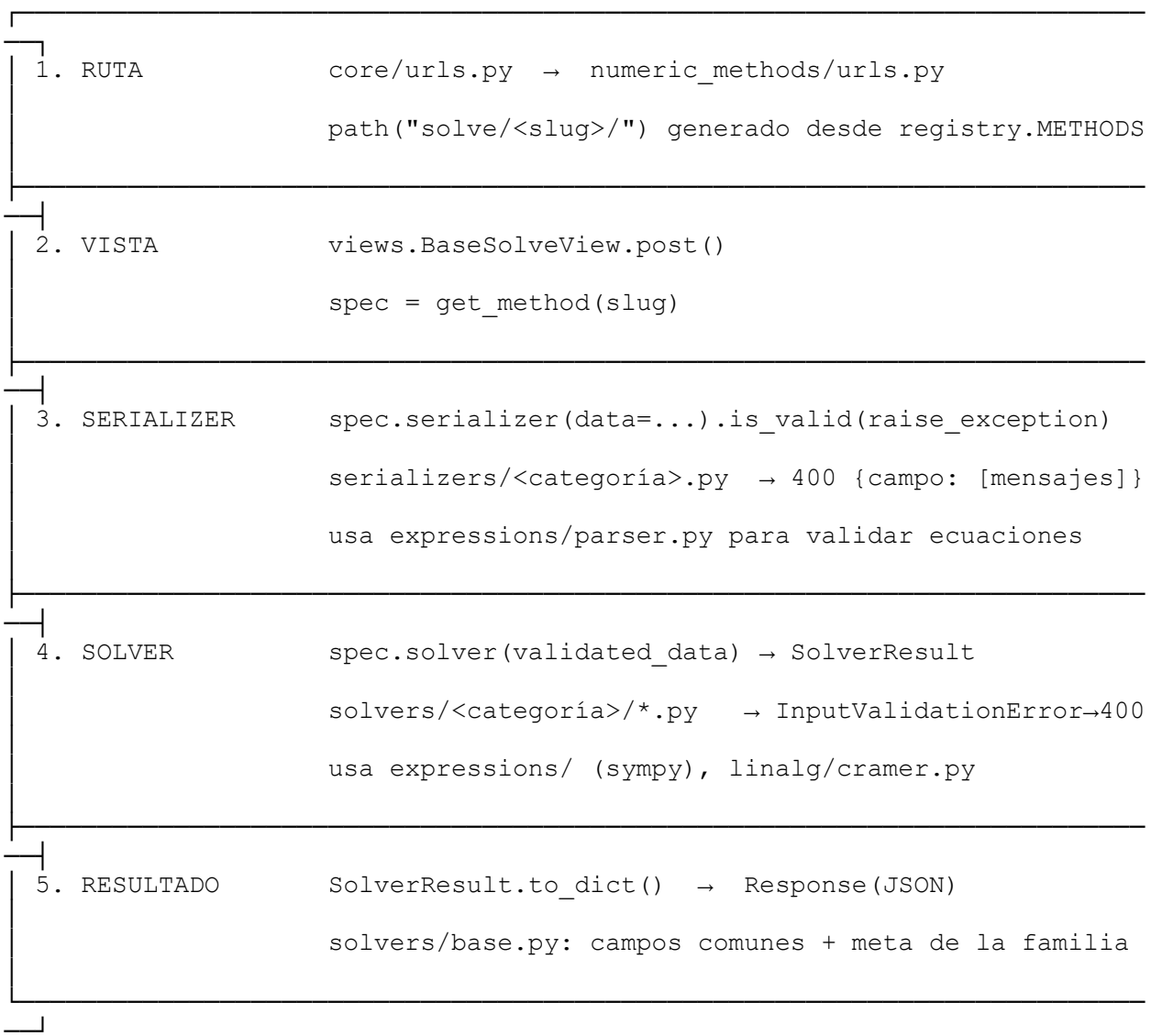


Cabecera CORS: Access-Control-Allow-Origin: http://localhost:5173

Puerto	Proceso	Modo	Accesible desde
8080	nginx (contenedor `frontend`, puerto interno 80)	Docker	El equipo y la red local
8000	gunicorn (contenedor `backend`)	Docker	Solo la red interna de Compose
8000	`python manage.py runserver 8000`	Desarrollo	`127.0.0.1` del equipo
5173	Vite (`npm run dev`)	Desarrollo	`localhost` del equipo

4.2 Arquitectura en capas

Backend. Cada petición de resolución atraviesa cuatro capas con responsabilidades separadas. Solo la primera y la segunda dependen de Django; los solvers, el parser de expresiones y el álgebra lineal son Python puro y se prueban de forma aislada.

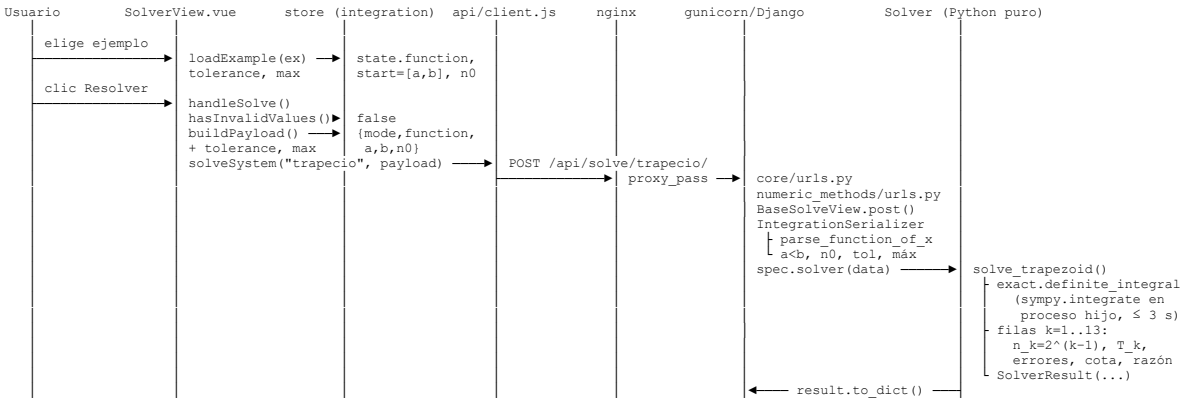


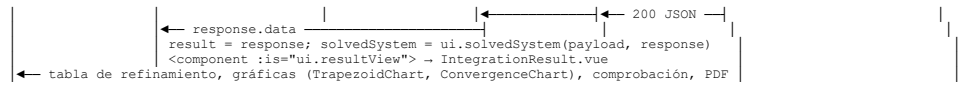
Frontend. La vista genérica no conoce ningún método en particular: recibe del registro la «interfaz de categoría» del método activo y monta sus componentes.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ VISTA              views/SolverView.vue  (una para los 12 métodos)           │
│                    views/TheoryView.vue                                      │
├──────────────────────────────────────────────────────────────────────────────┤
│ REGISTRO           methods/registry.js: catálogo del backend + CATEGORY_UI   │
│                    getMethod(slug).ui  → interfaz de la categoría            │
├──────────────────────────────────────────────────────────────────────────────┤
│ INTERFAZ DE        methods/<categoría>/index.js                              │
│ CATEGORÍA          createStore, form, configFields, resultView, pdf...       │
├──────────────────────────────────────────────────────────────────────────────┤
│ COMPONENTES        components/*.vue (compartidos)                            │
│                    methods/<categoría>/*.vue (propios)                       │
│                    composables/, utils/                                      │
├──────────────────────────────────────────────────────────────────────────────┤
│ CLIENTE HTTP       api/client.js (Axios, baseURL = VITE_API_BASE_URL)        │
└──────────────────────────────────────────────────────────────────────────────┘
```

4.3 Flujo de una petición de principio a fin

El caso concreto es el **ejemplo 1 del método del trapecio** (`trap-chapra`, «Polinomio de Chapra en [0, 0.8]»): $f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$, $a = 0$, $b = 0.8$, $n_0 = 1$, tolerancia 10^{-6}, máximo 20 refinamientos. En Docker, la respuesta es un JSON de 571 060 bytes, con 13 filas y $T = 1.6405331807454469$.





Archivos y funciones que intervienen, en orden:

Paso	Archivo	Función o elemento
1	`frontend/src/views/SolverView.vue`	`loadExample`, `handleSolve`
2	`frontend/src/methods/integration/store.js`	`createIntegrationStore` → `loadExample`, `hasInvalidValues`, `buildPayload`
3	`frontend/src/api/client.js`	`solveSystem(method, payload)`
4	`frontend/nginx.conf`	`location /api/ { proxy_pass http://backend:8000; }`
5	`backend/core/urls.py`	`path('api/', include('numeric_methods.urls'))`
6	`backend/numeric_methods/urls.py`	`solve_patterns`, una ruta por slug
7	`backend/numeric_methods/views.py`	`BaseSolveView.post`
8	`backend/numeric_methods/serializers/integration.py`	`IntegrationSerializer.validate`
9	`backend/numeric_methods/solvers/integration/trapezoid.py`	`solve_trapezoid(data)`
10	`backend/numeric_methods/solvers/integration/exact.py`	`definite_integral`, `_integrate_with_timeout`
11	`backend/numeric_methods/solvers/integration/rules.py`	`TRAPEZOID`, `apply`
12	`backend/numeric_methods/solvers/base.py`	`SolverResult.to_dict`
13	`frontend/src/methods/integration/IntegrationResult.vue`	Vista del resultado (`resultView` de la categoría)

4.4 Patrones de diseño aplicados

Registro central

`backend/numeric_methods/registry.py` es la única fuente de verdad sobre qué métodos existen. Cada método es un `MethodSpec` inmutable que asocia un slug con su serializer y su solver:

`backend/numeric_methods/registry.py`



```python
@dataclass(frozen=True)
class MethodSpec:
    slug: str
    name: str
    category: str
    serializer: type
    solver: Callable
    summary: str = ""
    keywords: tuple = ()

METHODS = {
    spec.slug: spec
    for spec in (
        MethodSpec(
            slug="jacobi",
            name="Jacobi",
            category="linear_system",
            serializer=LinearSystemSerializer,
            solver=solve_jacobi,
            # ...
        ),
        # ... 11 métodos más
    )
}
```

De ese diccionario se derivan, sin código adicional, las rutas de resolución (`urls.py` recorre `METHODS`) y el catálogo `GET /api/methods/` (`public_catalog()`). En el frontend, `frontend/src/methods/registry.js` descarga ese catálogo al arrancar y `frontend/src/router/index.js` crea `/resolver/<slug>` y `/teoria/<slug>` por cada método. El resultado es que **ninguna capa enumera los métodos a mano**: un método nuevo de una categoría existente aparece en la API, en el menú y en el enrutador con solo agregar su entrada a `METHODS` y sus ejemplos. Las vistas, las URL y los componentes no dependen de los métodos concretos, sino del registro (inversión de dependencias).

Contrato de categoría en el frontend

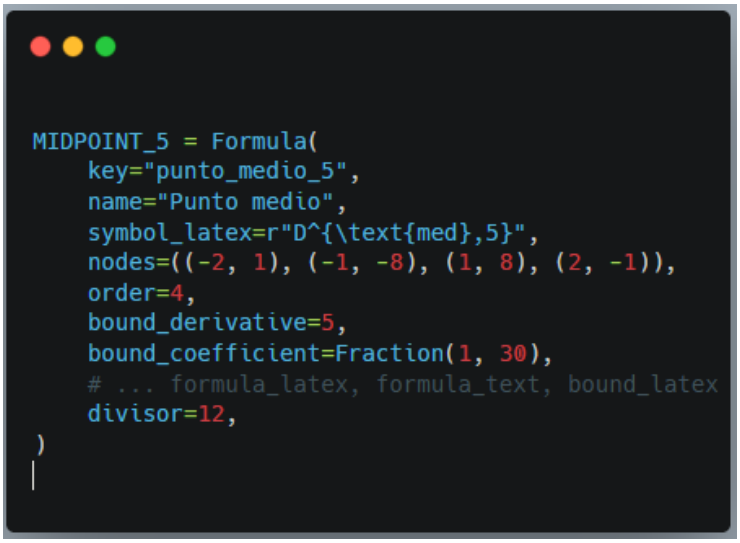
Cada carpeta `frontend/src/methods/<categoría>/index.js` exporta un objeto con una forma fija (creador de estado, formulario, componentes de resultado, PDF, teoría). `SolverView.vue` usa ese objeto como una estrategia: llama a `ui.createStore()`, monta

`ui.form` con `ui.formBindings(store)`, delega en `ui.resultView` o en `ResultsTable` con los *slots* de la categoría. El contrato completo está en la sección 7.4. Una categoría puede sobrescribir partes por método mediante `methods[slug]`, que `methodUi()` combina con la base.

Motor de fórmulas de diferenciación

Las fórmulas de diferencias finitas son datos, no código. Cada `Formula` declara sus nodos (o_i, w_i), el divisor q, el orden p y la cota $c\,h^p \max|f^{(n)}|$:

`backend/numeric_methods/solvers/differentiation/formulas.py`



```python
MIDPOINT_5 = Formula(
    key="punto_medio_5",
    name="Punto medio",
    symbol_latex=r"D^{\text{med},5}",
    nodes=((-2, 1), (-1, -8), (1, 8), (2, -1)),
    order=4,
    bound_derivative=5,
    bound_coefficient=Fraction(1, 30),
    # ... formula_latex, formula_text, bound_latex
    divisor=12,
)
```

El motor común (`finite_differences.py`) evalúa $D(h) = \sum_i w_i\, f(x_0 + o_i h)/(qh)$ para cualquier fórmula, calcula la razón esperada 0.5^p entre errores consecutivos y la cota con n y c. Los métodos de 3 y 5 puntos reutilizan el motor y solo agregan su presentación propia (identidades de Richardson, comparación con otras fórmulas).

Motor de reglas con pesos de integración

Del mismo modo, cada regla compuesta es un `CompositeRule` que declara el factor, el peso de los extremos y el ciclo de pesos interiores:

`backend/numeric_methods/solvers/integration/rules.py`



```python
SIMPSON_13 = CompositeRule(
    key="simpson13",
    name="Simpson 1/3",
    factor=Fraction(1, 3),
    ends=1,
    interior=(4, 2),
    segments_multiple=2,
    # ...
    order=4,
    bound_derivative=4,
    bound_divisor=180,
    # ...
)

def apply(rule, values, h):
    """factor · h · Σ wᵢ·f(xᵢ) con los valores f(x₀), …, f(xₙ)."""
    n = len(values) - 1
    return float(rule.factor) * h * sum(w * v for w, v in zip(rule.weights(n), values))
```

`weight(i, n)` devuelve `ends` en $i \in \{0, n\}$ e `interior[(i - 1) % len(interior)]` en el resto: los pesos 1,2, …,2,1 del trapecio, 1,4,2, …,4,1 de Simpson 1/3 y 1,3,3,2, …,3,1 de Simpson 3/8 salen de la misma función. Romberg reutiliza `TRAPEZOID` y `SIMPSON_13` para sus identidades de columna.

Objeto de resultado común

Todos los solvers devuelven un `SolverResult` con una lista de `IterationStep`. Los campos comunes tienen el mismo nombre en los 12 métodos y los propios de cada familia viajan en `meta`, que `to_dict()` aplana al primer nivel e impide que pise un campo común:

`backend/numeric_methods/solvers/base.py`

```python
@dataclass
class SolverResult:
    method: str
    category: str
    converged: bool
    iterations: list
    solution: list
    variables: list
    warnings: list = field(default_factory=list)
    meta: dict = field(default_factory=dict)

    def to_dict(self):
        data = {
            "method": self.method,
            # ... category, converged, iterations, iterations_used, solution,
variables, warnings
        }
        clashes = data.keys() & self.meta.keys()
        if clashes:
            raise ValueError(f"meta no puede redefinir campos comunes:
{sorted(clashes)}")
        data.update(self.meta)
        return data
```

Gracias a este contrato, `ResultsTable.vue`, `ConvergenceChart.vue` y `exportPdf.js` funcionan con cualquier método que use la vista genérica: leen `iterations[].x`, `iterations[].error`, `solution` y `variables`.

Separación entre cálculo y presentación

Todo número que la interfaz muestra lo calcula el backend: soluciones, errores, cotas, residuos de la comprobación, puntos de las gráficas de la aproximación, nodos y pesos de Gauss, e incluso el texto decimal de h con su forma exacta. El frontend formatea y dibuja. La única reconstrucción en el cliente es el paso a paso de Jacobi y Gauss-Seidel (`frontend/src/utils/iterationSteps.js`), que reordena valores que ya vienen en la respuesta (`A`, `b`, `iterations[].x`) sin calcular nada nuevo: sus diferencias son restas de valores mostrados. Esta separación hace que las pruebas del backend cubran la corrección numérica de todo lo que ve el usuario.

4.5 Decisiones de arquitectura y sus consecuencias

Decisión	Alternativa descartada	Motivo	Consecuencia
Sin base de datos de dominio	Guardar ejercicios y resultados por usuario	El sistema es una calculadora didáctica; no hay estado que conservar entre sesiones.	Despliegue trivial, peticiones independientes y escalado horizontal sin coordinación. No hay historial de ejercicios.

Decisión	Alternativa descartada	Motivo	Consecuencia
Cálculo en el servidor	Implementar los métodos en JavaScript	sympy no tiene equivalente en el navegador para despejes, antiderivadas y LaTeX; Python permite pruebas numéricas exhaustivas.	Una sola implementación de cada método, probada con 565 tests. Requiere conexión con el backend para resolver.
PDF generado en el cliente	Endpoint de PDF en el backend	El navegador ya tiene todos los datos y la imagen de la gráfica.	Sin dependencias de renderizado en el servidor; el PDF refleja exactamente lo que se ve. Las fórmulas del PDF se convierten de LaTeX a texto plano (`frontend/src/utils/latexText.js`) porque jsPDF no dibuja LaTeX.
LaTeX generado en el servidor y dibujado en el cliente	Imágenes de fórmulas en el servidor; LaTeX construido en el frontend	El backend conoce el árbol sympy y su forma escrita; KaTeX dibuja rápido y respeta el tema.	Fórmulas nítidas y seleccionables; respuestas JSON más grandes (contienen LaTeX).
Una ruta por método (`/api/solve/<slug>/`)	Una ruta única con el método en el cuerpo	Rutas autodescriptivas y validación por método sin bifurcaciones.	Las rutas se generan al importar `urls.py`; un slug inexistente responde 404 sin llegar a la vista.
Ejemplos en código Python	Fixtures o base de datos	Los ejemplos se revisan en los mismos commits que los solvers y los tests los recorren.	Cambiar un ejemplo requiere reconstruir y redeplegar.
Antiderivada simbólica en un proceso hijo con plazo	Hilo con *timeout* o `signal.alarm`	Un hilo no se puede detener; `signal.alarm` no existe en Windows ni fuera del hilo principal.	Plazo real de 3 s por antiderivada; costo de crear un proceso (*fork* en Linux, *spawn* en Windows).

4.6 Restricciones conocidas

- **Sin persistencia ni cuentas.** No se guardan ejercicios, resultados ni preferencias en el servidor; el tema se guarda en `localStorage` del navegador.

- **Dependencia del backend.** El frontend no resuelve nada sin la API; si el catálogo no carga, la vista muestra un mensaje de error.
- **Tamaño de los sistemas.** Sistemas lineales con $n \geq 3$; el formulario admite hasta 12 variables (`MAX_VARIABLES = 12` en `frontend/src/methods/linear/store.js`), y el reordenamiento automático para lograr dominancia diagonal solo se intenta con $n \leq 8$. Sistemas no lineales de 2 a 6 ecuaciones; la regla de Cramer admite matrices de hasta 8×8.
- **Polinomios de grado 3 a 10** en Bairstow.
- **Despeje automático en Punto Fijo.** Cada ecuación debe tener exactamente un despeje real en su variable y grado estimado ≤ 4; no se pueden escribir las g_i a mano.
- **Límites `a` y `b` numéricos.** El formulario de integración acepta números y fracciones, no expresiones como `pi`.
- **Cuadratura gaussiana solo en modo función.** Los nodos son irracionales, por lo que no hay modo tabla.
- **Cotas por muestreo.** Los máximos $M = \max|f^{(n)}|$ se estiman en una malla (401 o 2001 puntos), no se calculan exactamente.
- **Costo factorial de Cramer.** El determinante por cofactores crece como $n!$; está acotado por `MAX_SIZE = 8`.
- **Paquete JavaScript único.** El bundle principal compilado pesa 2 677,72 kB (750,45 kB con gzip); Vite advierte que supera 500 kB. No se usa división de código por rutas.
- **Interfaz solo en español**, sin internacionalización. El atributo `lang` de `frontend/index.html` es `en`, aunque el contenido está en español.

5. Estructura de carpetas

El árbol refleja los archivos versionados en el commit `2408a2a`. Se omiten `node_modules/`, `dist/`, `backend/.venv/`, `backend/db.sqlite3` y las carpetas `__pycache__/`, que no se versionan. Las carpetas de categoría del frontend se listan con todos sus archivos solo en

```
metodos/
├── .env.example                    # Plantilla de variables de entorno para Docker
├── .gitattributes                  # Fin de línea: *.sh LF, *.bat CRLF
├── .gitignore                      # Ignora .env
├── README.md                       # Documentación general del proyecto
├── docker-compose.yml              # Servicios backend (gunicorn) y frontend
(nginx :8080)
├── iniciar.bat                     # Inicia con Docker, espera y abre el navegador
├── detener.bat                     # docker compose down
├── reconstruir.bat                 # Reconstruye imágenes con el código actual e
inicia
├── docs/
│   ├── manual-tecnico.md           # Este documento
│   └── img/tecnico/LEEME.txt       # Convención de nombres de las capturas
├── backend/
│   ├── .dockerignore               # Excluye .venv, db.sqlite3, .env del contexto
de Docker
│   ├── .gitignore
│   ├── Dockerfile                  # python:3.12-slim + gunicorn 23.0.0, usuario
app
│   ├── manage.py                   # CLI de Django
│   ├── requirements.txt            # Django, DRF, django-cors-headers, sympy
│   ├── core/                       # Proyecto Django
│   │   ├── __init__.py
│   │   ├── asgi.py                 # Punto de entrada ASGI (no se usa en el
despliegue)
│   │   ├── settings.py             # Configuración; lee DJANGO_* del entorno
│   │   ├── urls.py                 # admin/ y api/ → numeric_methods.urls
│   │   └── wsgi.py                 # Punto de entrada WSGI (gunicorn
core.wsgi:application)
│   └── numeric_methods/            # Aplicación principal
│       ├── __init__.py
│       ├── admin.py                # Vacío (sin modelos)
│       ├── apps.py                 # NumericMethodsConfig
│       ├── models.py               # Vacío (sin modelos)
│       ├── migrations/__init__.py
│       ├── registry.py             # CATEGORIES, METHODS, get_method,
public_catalog
│       ├── urls.py                 # Rutas solve/<slug>/ generadas + methods,
examples, preview
│       ├── views.py                # BaseSolveView, MethodsListView,
ExamplesListView, ExpressionPreviewView
│       ├── examples_data.py        # EXAMPLES por slug y all_examples()
│       ├── serializers/            # Validación de entrada por categoría
│       │   ├── __init__.py
│       │   ├── differentiation.py     # Diferencias, 3 y 5 puntos;
validate_points
│       │   ├── expressions.py         # ExpressionPreviewSerializer
│       │   ├── integration.py         # Trapecio, Simpson, Romberg, Gauss
│       │   ├── linear.py              # LinearSystemSerializer
│       │   ├── nonlinear.py           # NonlinearSystemSerializer,
NewtonSystemSerializer
│       │   └── polynomial.py          # PolynomialSerializer
│       ├── expressions/            # Manejo simbólico (sympy)
│       │   ├── __init__.py
│       │   ├── parser.py              # Parseo seguro: lista blanca, entorno
restringido, árbol
│       │   ├── differentiate.py       # derivative, gradient, jacobian
│       │   ├── normalize.py           # evaluate_tree, estimate_degree
│       │   ├── to_latex.py            # Conversión a LaTeX conservando el orden
escrito
│       │   └── to_text.py             # Texto plano reescribible (mensajes y PDF)
│       ├── linalg/
│       │   ├── __init__.py
│       │   └── cramer.py              # determinant (cofactores), is_singular,
cramer
│       ├── solvers/
│       │   ├── __init__.py
│       │   ├── base.py                # SolverResult, IterationStep
│       │   ├── validation.py          # DEFAULT_TOLERANCE,
DEFAULT_MAX_ITERATIONS, MIN_ITERATIONS
│       │   ├── linear/                # jacobi.py, gauss_seidel.py, runner.py,
dominance.py, validation.py
│       │   ├── nonlinear/             # fixed_point.py, newton.py, common.py,
isolation_steps.py,
│       │                              # derivative_steps.py, verification.py
│       │   ├── polynomial/            # bairstow.py, synthetic.py, validation.py,
latex.py
│       │   ├── differentiation/       # formulas.py, finite_differences.py,
three_point.py,
│       │                              # five_point.py, display.py, chart.py
│       │   └── integration/           # rules.py, exact.py, trapezoid.py,
simpson.py,
│       │                              # romberg.py, gauss.py
│       └── tests/                  # 20 módulos test_*.py, 565 tests
└── frontend/
    ├── .dockerignore
    ├── .env.development            # VITE_API_BASE_URL=http://127.0.0.1:8000/api
    ├── .gitignore
    ├── .vscode/extensions.json     # Recomienda Vue.volar
    ├── Dockerfile                  # node:20-alpine (build) + nginx:alpine
    ├── README.md                   # README de la plantilla de Vite
    ├── index.html                  # Documento único; script de tema antes del
primer pintado
    ├── nginx.conf                  # SPA fallback, proxy /api/, gzip, caché
    ├── package.json                # Dependencias y scripts dev/build/preview
    ├── package-lock.json           # Versiones exactas resueltas
    ├── vite.config.js              # Plugin de Vue
    ├── public/
    │   ├── alternativa-integral.ico  # Favicon
    │   └── icons.svg
    └── src/
        ├── main.js                 # Carga el catálogo, crea rutas y monta la app
        ├── App.vue                 # Cabecera, menú, botón de tema, <RouterView>,
pie
        ├── style.css               # Tipografía, espaciado y estilos base
        ├── assets/
        │   ├── theme.css           # Paleta clara y oscura en variables CSS
        │   └── vite.svg
        ├── api/client.js           # Axios: fetchMethods, fetchExamples,
solveSystem, previewExpression
        ├── router/index.js         # Rutas fijas y addMethodRoutes()
        ├── views/
        │   ├── SolverView.vue      # Vista «Resolver» genérica
        │   └── TheoryView.vue      # Vista «Teoría» genérica
        ├── components/             # 15 componentes compartidos
        │   └── theory/             # AlgorithmSteps, TheoryLayout,
WorkedIterations
        ├── composables/
        │   ├── useExpressionPreviews.js  # Vista previa en vivo de ecuaciones
        │   └── useTheme.js         # Tema claro/oscuro
        ├── utils/
        │   ├── exportPdf.js        # PDF genérico y primitivas de maquetación
        │   ├── iterationSteps.js # formatNumber, paso a paso de Jacobi/Gauss-
Seidel
        │   ├── latexFormulas.js  # LaTeX de fórmulas y sustituciones
        │   ├── latexText.js      # LaTeX → texto plano para el PDF
        │   └── numberInput.js    # Decimales y fracciones en los campos
        └── methods/
            ├── registry.js         # Catálogo + interfaz por categoría
            ├── routeNames.js       # solveRouteName, theoryRouteName
            ├── linear/             # Jacobi y Gauss-Seidel
            │   ├── index.js               # Contrato de la categoría (documentado
aquí)
            │   ├── store.js               # createLinearSystemStore
            │   ├── LinearConfigFields.vue, LinearConfigExtras.vue
            │   ├── LinearIterationDetail.vue, LinearResultSummary.vue,
LinearVerification.vue
            │   ├── pdf.js, verification.js
            │   └── theory/                # LinearTheoryPage.vue
(THEORY_SECTIONS) y secciones
            ├── nonlinear/          # Punto Fijo y Newton (store.js,
newtonStore.js, pdf.js, newtonPdf.js...)
            ├── polynomial/         # Bairstow (PolynomialInput, PolynomialResult,
SyntheticTable...)
            ├── differentiation/  # Diferencias, 3 y 5 puntos
(DifferentiationResult, Chart, explain...)
            └── integration/        # Trapecio, Simpson, Romberg, Gauss (*Result,
*Pdf, *Explain...)
```

Estructura del proyecto en VS Code

Responsabilidades por zona

Raíz del repositorio:

Elemento	Responsabilidad
`docker-compose.yml`	Define los servicios, el puerto publicado 8080, las variables de entorno del backend y el *healthcheck*.
`.env.example`	Plantilla de `.env` con `DJANGO_DEBUG`, `DJANGO_SECRET_KEY` y `DJANGO_ALLOWED_HOSTS`.
`iniciar.bat`, `detener.bat`, `reconstruir.bat`	Operación con doble clic en Windows.
`.gitattributes`	Fija CRLF para `.bat` y LF para `.sh`.
`docs/`	Documentación técnica y capturas.

Backend (`backend/`):

Carpeta	Responsabilidad
`core/`	Proyecto Django: configuración, URL raíz y puntos de entrada WSGI/ASGI.
`numeric_methods/`	Aplicación: registro, vistas, rutas y ejemplos.
`numeric_methods/serializers/`	Validación de la entrada HTTP y mensajes de error por campo.
`numeric_methods/expressions/`	Interpretación segura de texto matemático y conversiones simbólicas.
`numeric_methods/linalg/`	Álgebra lineal directa (determinantes y Cramer) usada por Newton y Bairstow.
`numeric_methods/solvers/`	Implementación de los métodos, sin dependencia de Django.
`numeric_methods/tests/`	Pruebas unitarias, de integración de la API y de coherencia con la teoría.

Frontend (`frontend/`):

Carpeta	Responsabilidad
`public/`	Archivos que se copian tal cual a `dist/` (favicon, iconos).
`src/api/`	Única capa que conoce las URL de la API.
`src/router/`	Rutas fijas y rutas generadas desde el catálogo.
`src/views/`	Vistas genéricas «Resolver» y «Teoría».
`src/methods/`	Registro de interfaces y una carpeta por categoría con su formulario, resultado, PDF y teoría.

Carpeta	Responsabilidad
`src/components/`	Componentes reutilizados por varias categorías.
`src/composables/`	Estado reactivo reutilizable (tema, vistas previas).
`src/utils/`	Funciones puras de formato, LaTeX y PDF.
`src/assets/`	Paleta de colores del tema.

6. Backend

6.1 Configuración de Django

Responsabilidad. `backend/core/` contiene el proyecto Django: la configuración (`settings.py`), la URL raíz (`urls.py`) y los puntos de entrada `wsgi.py` (usado por gunicorn) y `asgi.py` (generado por `startproject`, no se usa en el despliegue).

Variables de entorno. `settings.py` toma de serie los valores del modo desarrollo; Docker los sobrescribe con variables de entorno. Las listas se leen con una función auxiliar que separa por comas:

`backend/core/settings.py`



```
SECRET_KEY = os.environ.get(
    'DJANGO_SECRET_KEY',
    'django-insecure-...',   # valor de desarrollo, recortado aquí
)

DEBUG = os.environ.get('DJANGO_DEBUG', '1') == '1'

def _env_list(name):
    """Lista separada por comas de una variable de entorno ('' -> [])."""
    return [item.strip() for item in os.environ.get(name, '').split(',') if
item.strip()]

ALLOWED_HOSTS = _env_list('DJANGO_ALLOWED_HOSTS')
CSRF_TRUSTED_ORIGINS = _env_list('DJANGO_CSRF_TRUSTED_ORIGINS')
# ...
CORS_ALLOWED_ORIGINS = _env_list('DJANGO_CORS_ALLOWED_ORIGINS') or [
    'http://localhost:5173',
    'http://127.0.0.1:5173',
]

REST_FRAMEWORK = {
    'DEFAULT_RENDERER_CLASSES': [
        'rest_framework.renderers.JSONRenderer',
    ],
}
entiation/  # Diferencias, 3 y 5 puntos (DifferentiationResult, Chart,
explain...)
            └── integration/      # Trapecio, Simpson, Romberg, Gauss (*Result,
*Pdf, *Explain...)
```

Ajustes relevantes.

Ajuste	Valor	Efecto
`DEBUG`	`True` si `DJANGO_DEBUG` vale `'1'` o no está definida; `False` con cualquier otro valor	En desarrollo, páginas de error detalladas. En Docker vale `0`.
`ALLOWED_HOSTS`	Vacío por defecto; `*` en Docker	Con `DEBUG=True` y lista vacía, Django acepta `localhost`, `127.0.0.1` y `[::1]`. En Docker, `*` porque el `Host` llega desde nginx con lo que escribió el navegador, y el backend no publica puertos.
`CORS_ALLOWED_ORIGINS`	`http://localhost:5173`, `http://127.0.0.1:5173`	Solo necesario en desarrollo; en Docker el navegador habla con un único origen.
`CORS` middleware	`corsheaders.middleware.CorsMiddleware`, antes de `CommonMiddleware`	Añade `Access-Control-Allow-Origin` a las respuestas para los orígenes permitidos.
`REST_FRAMEWORK.DEFAULT_RENDERER_CLASSES`	Solo `JSONRenderer`	Las respuestas son siempre JSON; se desactiva la

Ajuste	Valor	Efecto
		API navegable de DRF.
`DATABASES`	SQLite en `backend/db.sqlite3`	Solo tablas internas de Django (sección 9).
`INSTALLED_APPS`	Apps de Django (admin, auth, contenttypes, sessions, messages, staticfiles), `rest_framework`, `corsheaders`, `numeric_methods`	
`LANGUAGE_CODE`, `TIME_ZONE`	`en-us`, `UTC`	Valores de `startproject`; los mensajes de la API están escritos en español en el código.

URL raíz.

`backend/core/urls.py`



```python
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('numeric_methods.urls')),
]
```

El sitio `admin/` existe porque `startproject` lo incluye, pero no tiene modelos registrados ni usuarios creados, y en Docker no es alcanzable: nginx solo reenvía `/api/` (una petición a `http://localhost:8080/admin/` la atiende el *fallback* de la SPA).

6.2 Registro de métodos

Responsabilidad. `backend/numeric_methods/registry.py` declara las categorías y los métodos. Es el único archivo del backend que enumera los métodos.

Interfaz pública.

Elemento	Tipo	Descripción
`CategorySpec`	`dataclass(frozen=True)`	`label`, `description`, `order`, `latex` (expresión

Elemento	Tipo	Descripción
		que encabeza la columna del menú).
`CATEGORIES`	`dict[str, CategorySpec]`	Las 5 categorías: `linear_system` (1), `nonlinear_system` (2), `polynomial` (3), `differentiation` (4), `integration` (5).
`MethodSpec`	`dataclass(frozen=True)`	`slug`, `name`, `category`, `serializer`, `solver`, `summary`, `keywords`.
`METHODS`	`dict[str, MethodSpec]`	Los 12 métodos, indexados por slug.
`get_method(slug)`	Función	Devuelve el `MethodSpec` o `None`.
`public_catalog()`	Función	Lista plana de diccionarios sin `serializer` ni `solver`, ordenada por `order` de categoría y, dentro de cada una, por el orden de `METHODS` (`sorted` es estable).

Generación del catálogo.

`backend/numeric_methods/registry.py`

```python
def public_catalog():
    specs = sorted(METHODS.values(), key=lambda spec:
CATEGORIES[spec.category].order)
    return [
        {
            "slug": spec.slug,
            "name": spec.name,
            "category": spec.category,
            "category_label": CATEGORIES[spec.category].label,
            "summary": spec.summary,
            "keywords": list(spec.keywords),
            "category_description": CATEGORIES[spec.category].description,
            "category_latex": CATEGORIES[spec.category].latex,
            "category_order": CATEGORIES[spec.category].order,
        }
        for spec in specs
    ]
```

`summary` y `keywords` alimentan el buscador del menú del frontend («derivada», «área», «raphson»…).

6.3 Capa de vistas y rutas

Responsabilidad. Traducir HTTP a llamadas al registro, al serializer y al solver. Hay cuatro vistas, todas `APIView` de DRF.

`backend/numeric_methods/urls.py`

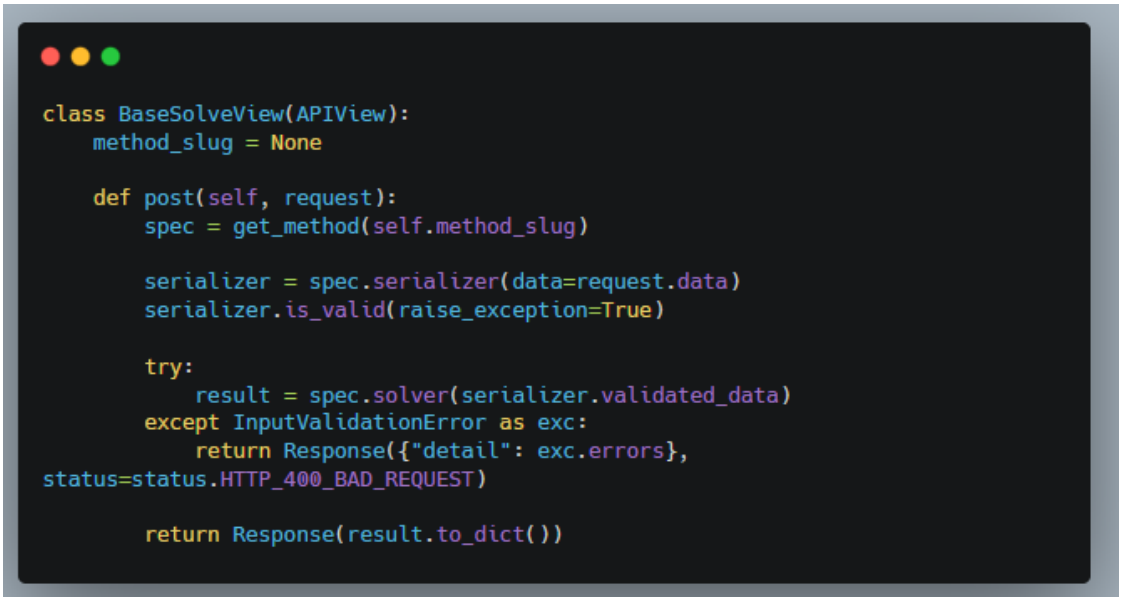


```python
# Una ruta de resolución por método registrado: /api/solve/<slug>/
solve_patterns = [
    path(f"solve/{slug}/", BaseSolveView.as_view(method_slug=slug),
name=f"solve-{slug}")
    for slug in METHODS
]

urlpatterns = [
    *solve_patterns,
    path("methods/", MethodsListView.as_view(), name="methods-list"),
    path("examples/", ExamplesListView.as_view(), name="examples-list"),
    # Con o sin barra final: un POST no puede redirigirse a la versión con
barra.
    re_path(r"^expressions/preview/?$", ExpressionPreviewView.as_view(),
name="expression-preview"),
]
```

Hay 15 patrones: 12 de resolución y 3 auxiliares. `as_view(method_slug=slug)` fija el atributo de clase `method_slug` en cada instancia de vista, así una sola clase atiende los 12 métodos:

`backend/numeric_methods/views.py`



```python
class BaseSolveView(APIView):
    method_slug = None

    def post(self, request):
        spec = get_method(self.method_slug)

        serializer = spec.serializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        try:
            result = spec.solver(serializer.validated_data)
        except InputValidationError as exc:
            return Response({"detail": exc.errors},
status=status.HTTP_400_BAD_REQUEST)

        return Response(result.to_dict())
```

Vista	Ruta	Método HTTP	Responsabilidad
`BaseSolveView`	`solve/<slug>/`	POST	Valida con el serializer del método, ejecuta el solver y serializa el `SolverResult`.
`MethodsListView`	`methods/`	GET	Devuelve `public_catalog()`.
`ExamplesListView`	`examples/`	GET	Con `?method=<slug>`, los ejemplos de ese método (404 si

Vista	Ruta	Método HTTP	Responsabilidad
			no existen); sin parámetro, `all_examples()`.
`ExpressionPreviewView`	`expressions/preview` (barra final opcional)	POST	Parsea una ecuación y devuelve `{"latex": ...}`; no resuelve nada.

Las vistas solo definen `get` o `post`; DRF responde **405** con `{"detail": "Method \"GET\" not allowed."}` a cualquier otro verbo.

6.4 Serializers

Responsabilidad. Validar la forma y el rango de la entrada antes de llegar al solver, aplicar valores por defecto y producir mensajes de error en español, agrupados por campo. Los serializers de las categorías con expresiones (no lineales, diferenciación, integración, preview) llaman al parser seguro para rechazar ecuaciones mal escritas con un 400 antes de ejecutar el método.

Archivo	Clases	Validaciones principales
`serializers/linear.py`	`LinearSystemSerializer`	`A` cuadrada con $n \geq 3$; `b` y `x0` de longitud n; diagonal sin ceros; `tolerance` ≥ 0; `max_iterations` ≥ 1; `auto_reorder` (por defecto `true`).
`serializers/nonlinear.py`	`NonlinearSystemSerializer`, `NewtonSystemSerializer`	2 a 6 ecuaciones; una variable por ecuación, nombres válidos y no reservados; `x0` de longitud n; cada ecuación parseable. Newton además rechaza una variable que no aparece en ninguna ecuación o una ecuación sin variables (jacobiano singular en todo punto).
`serializers/polynomial.py`	`PolynomialSerializer`	Exactamente uno de `polynomial` (texto) o `coefficients` (lista); tolerancia en % mayor que 0; `r0`, `s0` finitos con magnitud ≤ 10^{100}; grado

Archivo	Clases	Validaciones principales
		3 a 10; coeficiente principal no nulo. Tras validar, `coefficients` siempre contiene los coeficientes.
`serializers/differentiation.py`	`DifferentiationSerializer`, `ThreePointSerializer`, `FivePointSerializer`, `validate_points`	Modo `function`: `function` parseable en `x`, `x0` finito, `h0 > 0`, tolerancia > 0, refinamientos entre 6 y 30. Modo `table`: 2/3/5 a 20 puntos, x estrictamente crecientes; en 3 y 5 puntos, espaciado uniforme.
`serializers/integration.py`	`IntegrationSerializer`, `SimpsonSerializer`, `RombergSerializer`, `GaussSerializer`	Modo `function`: $a < b$, `n0` de 1 a 1000, tolerancia > 0, refinamientos entre 6 y 22. Simpson: `n0_13` par (2 a 1000), `n0_38` múltiplo de 3 (3 a 999). Romberg: 6 a 20 filas; tabla de 3, 5, 9, 17 o 33 puntos. Gauss: solo modo función, N de 1 a 50, máximo de puntos de 6 a 20.
`serializers/expressions.py`	`ExpressionPreviewSerializer`	`equation` como texto; `variables` como lista de hasta 20 nombres válidos. Calcula `latex` con `equation_to_latex`.

Herencia. Los serializers especializados reutilizan la validación de su base: `ThreePointSerializer` extiende `DifferentiationSerializer` y añade la comprobación de espaciado uniforme; `FivePointSerializer` extiende a `ThreePointSerializer`; Simpson, Romberg y Gauss extienden `IntegrationSerializer` cambiando atributos de clase (`min_points`, `max_points`, `max_rows`, `rows_error`) y añadiendo reglas propias.

Ejemplo de validación cruzada (varios campos a la vez, con un error por campo):

`backend/numeric_methods/serializers/linear.py`



```python
def validate(self, data):
    A = data.get("A")
    b = data.get("b")
    x0 = data.get("x0")
    errors = {}
    # ...
    n = len(A)

    if n < MIN_VARIABLES:
        errors.setdefault("A", []).append(
            f"El sistema debe tener al menos {MIN_VARIABLES} variables (n >=
{MIN_VARIABLES})."
        )
    # ... A cuadrada, longitud de b y de x0
    if errors:
        raise serializers.ValidationError(errors)
    # ... diagonal principal sin ceros
    return data
```

Formato de los errores. `is_valid(raise_exception=True)` lanza `ValidationError`; DRF la convierte en una respuesta **400** cuyo cuerpo es un objeto `{campo: [mensajes]}`. Los serializers construyen las listas de mensajes de forma plana (no indexadas por posición de la lista), que es lo que el frontend sabe mostrar.

6.5 Solvers

Contrato. Un solver es una función `solve_<método>(data) -> SolverResult` que recibe `serializer.validated_data`. Si la entrada es sintácticamente válida pero el método no puede aplicarse (por ejemplo, un despeje ambiguo), lanza una subclase de `InputValidationError`. Ningún solver importa Django.

`base.py`: contrato de resultado

`IterationStep` es una fila de la tabla: `iteration`, `x` (valores en el orden de `variables`), `error` (`None` si no es calculable) y `extra` (detalle propio del método; se serializa solo si no está vacío). `SolverResult` reúne `method`, `category`, `converged`, `iterations`, `solution`, `variables`, `warnings` y `meta`. La propiedad `iterations_used` es `len(iterations)`. El extracto está en la sección 4.4.

`validation.py`: constantes comunes

`backend/numeric_methods/solvers/validation.py`



```python
DEFAULT_TOLERANCE = 1e-6
DEFAULT_MAX_ITERATIONS = 100
MIN_ITERATIONS = 6

class InputValidationError(Exception):
    def __init__(self, errors):
        self.errors = errors if isinstance(errors, list) else [errors]
        super().__init__("; ".join(self.errors))
```

`MIN_ITERATIONS` impone el mínimo de 6 iteraciones o filas antes de declarar convergencia. En diferenciación e integración se reexporta como `MIN_REFINEMENTS` y `MIN_ROWS`.

Subclases por familia: `MatrixValidationError` (lineales), `NonlinearValidationError` (no lineales), `PolynomialValidationError` (Bairstow), `DifferentiationValidationError` (diferenciación) e `IntegrationValidationError` (integración).

Sistemas lineales (`solvers/linear/`)

Archivo	Contenido
`jacobi.py`, `gauss_seidel.py`	Algoritmos puros: `solve(A, b, x0, tolerance, max_iterations)` → diccionario con `solution`, `iterations`, `converged`, `iterations_used`.
`runner.py`	`run_linear_system`: verificación de dominancia, reordenamiento, advertencias, comprobación y empaquetado en `SolverResult`; `solve_jacobi` y `solve_gauss_seidel`.
`dominance.py`	`find_dominant_ordering`: busca la permutación de filas que maximiza el margen mínimo de dominancia ($n \leq 8$, búsqueda exhaustiva).
`validation.py`	`MIN_VARIABLES = 3`, `validate_dimensions`, `validate_diagonal_nonzero`, `check_diagonal_dominance`.

Decisiones numéricas:

- **Error:** norma infinito de la diferencia entre iteraciones, $\max_i |x_i^{(k+1)} - x_i^{(k)}|$.
- **Criterio de parada:** `error < tolerance and k >= MIN_ITERATIONS`; o `max_iterations` (si se pide menos de 6, se eleva a 6); o un valor no finito (divergencia), que corta la iteración con `error = None`.
- **Tolerancia por defecto:** 10^{-6}; máximo por defecto: 100.
- **Dominancia diagonal:** se comprueba siempre; si no se cumple y `auto_reorder` es verdadero, se reordenan filas de A junto con b (las columnas no cambian, por lo que x

conserva su orden). Si sigue sin cumplirse, se emite una advertencia: la dominancia es condición suficiente, no necesaria.

- **Respuesta:** `meta` incluye `A` y `b` tal como se usaron (reordenados), `row_order` y `verification`. La comprobación sustituye la solución en el sistema **original**; si todas las componentes se ven enteras con 6 decimales, se comprueba con los enteros (`rounded_to_integers`), y un residuo $|r_i| \leq 10^{-6} \cdot \max\left(1, |b_i|, \sum |a_{ij} x_j|\right)$ se muestra como «$\approx b_i$».

`backend/numeric_methods/solvers/linear/jacobi.py`

```python
for k in range(1, max_iterations + 1):
    x_new = [0.0] * n
    for i in range(n):
        s = sum(A[i][j] * x[j] for j in range(n) if j != i)
        x_new[i] = (b[i] - s) / A[i][i]

    if not all(math.isfinite(v) for v in x_new):
        iterations.append({"iteration": k, "x": x_new, "error": None})
        converged = False
        break

    error = max(abs(x_new[i] - x[i]) for i in range(n))
    iterations.append({"iteration": k, "x": x_new, "error": error})
    x = x_new

    if error < tolerance and k >= MIN_ITERATIONS:
        converged = True
        break
```

Sistemas no lineales (`solvers/nonlinear/`)

Archivo	Contenido
`fixed_point.py`	Punto Fijo con actualización secuencial (análogo no lineal de Gauss-Seidel).
`newton.py`	Newton multivariable con jacobiano simbólico y regla de Cramer.
`common.py`	`MIN_EQUATIONS = 2`, `MAX_EQUATIONS = 6`, `to_real` (complejo, infinito o NaN → `None`).
`isolation_steps.py`	Pasos del despeje de cada ecuación para la interfaz.
`derivative_steps.py`	Derivadas parciales término a término para el jacobiano mostrado.
`verification.py`	Sustitución de la solución en las ecuaciones originales.

Decisiones numéricas:

- **Punto Fijo, despeje automático.** g_i se obtiene una sola vez con `sympy.solve(f_i, x_i)` y se acepta solo si hay exactamente un despeje real; con varios despejes reales se responde 400 explicando el motivo y listándolos. Antes de llamar a `solve` se estima el

grado de la ecuación sin expandir (`estimate_degree`) y se rechaza si supera `MAX_SOLVE_DEGREE = 4`, porque `solve` puede tardar segundos en grados altos.

- **Newton.** En cada iteración evalúa $F\left(\mathbf{x}^{(k)}\right)$ y $J\left(\mathbf{x}^{(k)}\right)$, resuelve $J\,\Delta\mathbf{x} = -F$ por Cramer y actualiza $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}$. Un jacobiano singular detiene el método como no convergente.
- **Evaluación numérica.** g_i, F y J se convierten con `lambdify` (módulos `math` y `mpmath`) una sola vez; nunca se sustituye simbólicamente en cada iteración.
- **Error y parada:** norma infinito del cambio ($\| \Delta\mathbf{x} \|_\infty$ en Newton); `error < tolerance and k >= MIN_ITERATIONS`, o `max_iterations`, o un valor complejo, infinito o fuera de dominio.
- **Valores por defecto:** tolerancia 10^{-6}, máximo 100, $\mathbf{x}^{(0)} = \mathbf{0}$ si no se envía.

Polinomios (`solvers/polynomial/`)

Archivo	Contenido
`bairstow.py`	Iteración por factor, deflación, cierres directos (cuadrático o lineal), raíces nulas, comprobación.
`synthetic.py`	Divisiones sintéticas b y c (`b_table`, `c_table`).
`validation.py`	Grado 3 a 10, coeficientes finitos, lectura de texto a coeficientes (`coefficients_from_text`), `MAX_EXPAND_DEGREE = 200`.
`latex.py`	LaTeX del polinomio, de los factores y de la factorización.

Decisiones numéricas:

- **Sistema 2×2 por iteración** resuelto con Cramer: $c_2\Delta r + c_3\Delta s = -b_1$, $c_1\Delta r + c_2\Delta s = -b_0$.
- **Error relativo porcentual** con el valor ya actualizado: $\varepsilon_r = |\Delta r/r| \cdot 100$, $\varepsilon_s = |\Delta s/s| \cdot 100$; si r o s es menor que `ZERO_THRESHOLD = 1e-12`, se usa el error absoluto y se advierte.
- **Parada por factor:** $\varepsilon_r \le \varepsilon_s^{\text{tol}}$ y $\varepsilon_s \le \varepsilon_s^{\text{tol}}$ con al menos 6 iteraciones, o el máximo de iteraciones **por factor**.
- **Valores por defecto:** tolerancia 0.0001 %, $r_0 = s_0 = -1$, máximo 100 por factor. Cada factor arranca con los r_0 y s_0 del usuario.
- **Raíces complejas** sin aritmética compleja en la iteración: si $\Delta = r^2 + 4s < 0$, parte real $r/2$ e imaginaria $\sqrt{-\Delta}/2$. La comprobación final sí usa `complex` y es informativa.

Diferenciación numérica (`solvers/differentiation/`)

Archivo	Contenido
`formulas.py`	Las 8 fórmulas como datos (sección 4.4) y los diccionarios de opciones por método.
`finite_differences.py`	Motor común (modos función y tabla, cota, comprobación) y `solve_finite_differences`.
`three_point.py`, `five_point.py`	Métodos de 3 y 5 puntos sobre el mismo motor: identidades de Richardson, comparaciones, columna «Recomendada».
`display.py`	Reglas de presentación de h y de los decimales del paso a paso.
`chart.py`	Datos de la gráfica de la aproximación (polinomio interpolante de Lagrange, tangente, vista ilustrativa 2×, 5×, 10×).

Decisiones numéricas:

- **Refinamiento:** la fila k usa $h_k = h_0/2^{k-1}$. Por cada fórmula: aproximación D_k, error estimado $|D_k - D_{k-1}|$, error real $|D_k - f'(x_0)|$ (la derivada exacta se obtiene con sympy), cota $c\,h^p M_k$ y razón entre errores reales consecutivos (esperada 0.5^p).
- **Parada:** error estimado ≤ tolerancia **en todas las fórmulas pedidas**, con al menos 6 filas, o el máximo de refinamientos.
- **Valores por defecto:** diferencias progresiva y regresiva, tolerancia 10^{-4}, máximo 25 refinamientos (6 a 30), $h_0 = 0.1$; 3 y 5 puntos, tolerancia 10^{-6}.
- **Cota:** $M_k = \max|f^{(n)}|$ se estima muestreando 401 puntos del intervalo de la fila (`SAMPLES = 401`).
- **Modo tabla:** en cada x_i, cada fórmula usa sus vecinos con su propio h; con una función para comparar se añaden derivada exacta, error real y cota.

Integración numérica (`solvers/integration/`)

Archivo	Contenido
`rules.py`	Trapecio, Simpson 1/3 y Simpson 3/8 como `CompositeRule` (sección 4.4).
`exact.py`	Integral exacta por antiderivada (sympy) o de referencia (`mpmath.quad` a 30 dígitos), con límite de tiempo.
`trapezoid.py`	Trapecio simple y compuesto; recurrencia que reutiliza puntos evaluados; modo tabla con espaciado no uniforme.
`simpson.py`	Reglas 1/3 y 3/8; en tabla con n impar, combinación 1/3 + 3/8.
`romberg.py`	Tabla triangular con extrapolación de Richardson; comparación de costo.
`gauss.py`	Nodos y pesos de Gauss-Legendre por Newton sobre la recurrencia de Legendre; versión compuesta.

Decisiones numéricas:

Método	Fila k	Criterio de parada	Tolerancia y máximo por defecto	Error y cota
Trapecio	$n_k = n_0 \cdot 2^{k-1}$ (tope 2^{21} segmentos)	$\lvert T_k - T_{k-1}\rvert \le$ tol, $k \ge 6$	10^{-6}; 20 (6 a 22)	E_t, ε_t, E_a de Chapra, cota $(b-a)h^2M_2/12$ con M_2 muestreado en 2001 puntos
Simpson	$n_k = n_0 \cdot 2^{k-1}$ por regla	Todas las reglas pedidas cumplen en la misma fila, $k \ge 6$	10^{-6}; 20	Cota $(b-a)h^4M_4/180$ (1/3) o $/80$ (3/8)
Romberg	Fila i: trapecio con 2^{i-1} segmentos y $R_{i,k}$ por Richardson	$\lvert R_{i,i} - R_{i-1,i-1}\rvert \le$ tol, $i \ge 6$	10^{-6}; 15 filas (6 a 20)	ε_a porcentual de Chapra
Gauss	$m = k$ puntos con N fijo	$\lvert G_m - G_{m-1}\rvert \le$ tol, $m \ge 6$	10^{-6}; 12 puntos (6 a 20)	Cota con M_{2m} solo para $m \le 5$

Integral exacta o de referencia. `exact.py` intenta `sympy.integrate(f, x)` en un proceso hijo con plazo `TIME_LIMIT = 3.0` s. Si la antiderivada F no contiene `Integral`, solo usa funciones de la lista blanca y $F(a)$, $F(b)$ son reales finitos, la integral es «exacta»: $F(b) - F(a)$. Si no, se calcula una referencia con `mpmath.quad` a 30 dígitos. Las antiderivadas se guardan en una caché por expresión de hasta 256 entradas por proceso.

`backend/numeric_methods/solvers/integration/exact.py`

```python
def _integrate_with_timeout(expr, symbol, timeout=TIME_LIMIT):
    """F, None (sympy falló) o TIMED_OUT."""
    context = multiprocessing.get_context("spawn" if os.name == "nt" else
"fork")
    channel = context.Queue()
    process = context.Process(target=_worker, args=(expr, symbol, channel),
daemon=True)
    process.start()
    try:
        channel.get(timeout=STARTUP_LIMIT)
        status, result = channel.get(timeout=timeout)
        return result if status == "ok" else None
    except queue_module.Empty:
        return TIMED_OUT
    finally:
        if process.is_alive():
            process.kill()
```

El plazo empieza cuando el hijo avisa que está listo (`STARTUP_LIMIT = 30.0` s de espera máxima para el arranque), así el arranque de *spawn* en Windows, que importa sympy, no se descuenta de los 3 s.

6.6 Manejo simbólico de expresiones

Responsabilidad. `backend/numeric_methods/expressions/` convierte el texto del usuario en árboles sympy validados y los convierte de vuelta a LaTeX o a texto. Es la única puerta de entrada de texto matemático al sistema.

Archivo	Interfaz pública	Descripción
`parser.py`	`parse_expression`, `parse_equation`, `parse_function`, `parse_system`, `make_symbols`, `validate_expression_tree`, `ExpressionError`, `ALLOWED_FUNCTIONS`, `ALLOWED_CONSTANTS`, límites `MAX_*`	Parseo seguro con tres capas de defensa (sección 10.2). Acepta `^` como potencia, multiplicación implícita (`3x`, `2(x+1)`), ecuaciones `lhs = rhs` y la forma con encabezado `f1(x, y) = ...`.
`differentiate.py`	`derivative`, `gradient`, `jacobian`	Derivadas parciales con símbolos normalizados por nombre (`real=True`, igual que el parser).
`normalize.py`	`evaluate_tree`, `estimate_degree`	Reconstrucción del árbol con la evaluación normal de sympy; cota superior del grado sin expandir.
`to_latex.py`	`expression_to_latex`, `equation_to_latex`, `substitution_template_latex`, `group_log_bases`	LaTeX con `order="none"` (respeta el orden escrito) y corrección de fracciones en árboles no evaluados.
`to_text.py`	`expression_to_text`	Texto reescribible en el formulario: `^` en lugar de `**`, `ln(x)` para el natural, `abs(x)`.

Funciones y constantes admitidas. `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh`, `cosh`, `tanh`, `exp`, `log` (base 10; `log(x, b)` en base b), `ln` (natural), `sqrt`, `abs`; constantes `pi` y `e`.

Derivación. `jacobian` devuelve `sp.Matrix(exprs).jacobian(símbolos)`. Los símbolos se reconstruyen con `make_symbols`, de modo que coinciden con los del parser: un `Symbol("x")` sin `real=True` sería una variable distinta y su derivada saldría 0.

Integración simbólica. No está en `expressions/`: la realiza `backend/numeric_methods/solvers/integration/exact.py` con el límite de tiempo descrito en 6.5. Las derivadas de orden alto de la cota de Gauss (`derivatives`) usan el mismo mecanismo de proceso hijo y plazo.

Límites de tiempo. El parser no ejecuta operaciones costosas: rechaza por adelantado lo que podría colgar el proceso (números enormes, exponentes > 1000, anidamiento excesivo). Los únicos límites de tiempo explícitos son los de `exact.py` (3 s). Los despejes de Punto Fijo se acotan por grado (≤ 4) y la expansión de polinomios en Bairstow por grado estimado (≤ 200).

6.7 Álgebra lineal

Responsabilidad. `backend/numeric_methods/linalg/cramer.py` resuelve sistemas lineales directos pequeños en Python puro con floats. Lo usan Newton (sistema $J\,\Delta\mathbf{x} = -F$ en cada iteración) y Bairstow (sistema 2×2 en cada iteración).

Elemento	Descripción
`determinant(matrix)`	Expansión por cofactores a lo largo de la primera fila; omite los términos con coeficiente 0.
`replace_column(matrix, column, vector)`	Copia de la matriz con una columna sustituida.
`is_singular(matrix, det=None)`	`abs(det) <= 1e-12 · cota de Hadamard` (producto de las normas de las filas).
`cramer(matrix, vector)`	Devuelve `CramerSolution(determinant, column_determinants, column_matrices, solution)`; lanza `SingularMatrixError`.
`MAX_SIZE = 8`	Tamaño máximo admitido.

`backend/numeric_methods/linalg/cramer.py`

```python
def _cofactor_expansion(matrix, n):
    if n == 1:
        return matrix[0][0]
    if n == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    total = 0.0
    for j, value in enumerate(matrix[0]):
        if value == 0:
            continue  # el término se anula: no hace falta calcular el menor
        minor = [row[:j] + row[j + 1:] for row in matrix[1:]]
        sign = -1.0 if j % 2 else 1.0
        total += sign * value * _cofactor_expansion(minor, n - 1)
    return total
```

Por qué Cramer y cofactores, y no eliminación gaussiana. El curso resuelve a mano el sistema de cada iteración de Newton con la regla de Cramer: $D = \det J$, $D_i = \det J_i$, $\Delta x_i = D_i/D$. El sistema devuelve exactamente esos valores intermedios (`D`, `D_i` y las matrices J_i en

`IterationStep.extra`) para que el estudiante pueda comparar su cálculo paso a paso; la eliminación gaussiana no produce esos números. El costo $O(n!)$ de los cofactores es irrelevante para los tamaños del curso (2 a 6 incógnitas, tope de 8) y queda acotado por `MAX_SIZE`. El criterio de singularidad es relativo a la escala (cota de Hadamard), no un umbral absoluto.

6.8 Datos de ejemplos

Responsabilidad. `backend/numeric_methods/examples_data.py` contiene los ejemplos precargados como estructuras Python. No hay base de datos ni archivos externos.

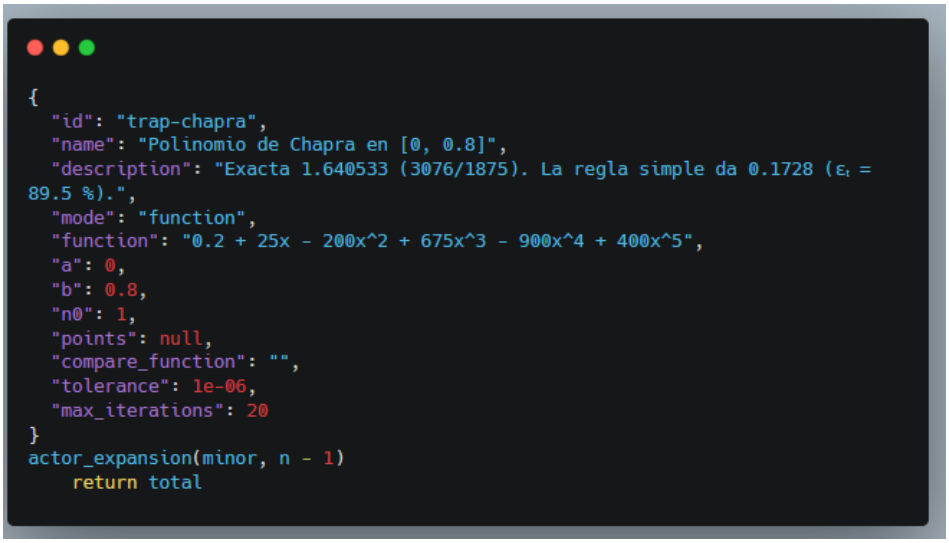
Estructura. Cada ejemplo es un diccionario con `id` (único), `name`, `description` y los mismos campos que acepta el serializer del método, incluidos `tolerance` y `max_iterations`. Varias familias usan funciones auxiliares que completan los campos por defecto (`_polynomial_example`, `_difference_example`, `_three_point_example`, `_integration_example`, `_simpson_example`, `_romberg_example`, `_gauss_example`). El diccionario `EXAMPLES` asocia cada slug con su lista; Jacobi y Gauss-Seidel comparten `LINEAR_SYSTEM_EXAMPLES`.

Slug	Lista	Ejemplos
`jacobi`	`LINEAR_SYSTEM_EXAMPLES`	6
`gauss-seidel`	`LINEAR_SYSTEM_EXAMPLES` (compartida)	6
`punto-fijo`	`NONLINEAR_SYSTEM_EXAMPLES`	4
`newton`	`NEWTON_EXAMPLES`	3
`bairstow`	`POLYNOMIAL_EXAMPLES`	9
`diferencias-finitas`	`DIFFERENCE_EXAMPLES`	8
`tres-puntos`	`THREE_POINT_EXAMPLES`	8
`cinco-puntos`	`FIVE_POINT_EXAMPLES`	8
`trapecio`	`TRAPEZOID_EXAMPLES`	8
`simpson`	`SIMPSON_EXAMPLES`	8
`romberg`	`ROMBERG_EXAMPLES`	8
`gauss`	`GAUSS_EXAMPLES`	8
Total		**84 asignaciones; 78 ejemplos distintos**

Cómo se sirven. `GET /api/examples/?method=<slug>` devuelve `EXAMPLES[slug]`; sin parámetro, `all_examples()` devuelve los 78 sin repetir, en orden de aparición. El frontend los

carga en el selector al entrar en un método y, al elegir uno, llama a `store.loadExample(example)`.

Ejemplo real (primer ejemplo del trapecio, tal como lo devuelve la API):



```
{
  "id": "trap-chapra",
  "name": "Polinomio de Chapra en [0, 0.8]",
  "description": "Exacta 1.640533 (3076/1875). La regla simple da 0.1728 (εt =
89.5 %).",
  "mode": "function",
  "function": "0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5",
  "a": 0,
  "b": 0.8,
  "n0": 1,
  "points": null,
  "compare_function": "",
  "tolerance": 1e-06,
  "max_iterations": 20
}
actor_expansion(minor, n - 1)
    return total
```

6.9 Manejo de errores

Origen del fallo	Mecanismo	Respuesta HTTP	Cuerpo
JSON mal formado	Parser de DRF	400	`{"detail": "JSON parse error - ..."}`
Campo con tipo o rango inválido, o ecuación no parseable	`serializer.is_valid(raise_exception=True)`	400	`{"<campo>": ["mensaje", ...]}`
Entrada válida que el método no puede tratar (despeje ambiguo, grado fuera de rango detectado en el solver…)	El solver lanza `InputValidationError`; `BaseSolveView` la captura	400	`{"detail": ["mensaje", ...]}`
No convergencia, divergencia, jacobiano singular	El solver devuelve `converged: false` con `warnings`	200	Resultado completo con advertencias
Ejemplos de un slug inexistente	`ExamplesListView`	404	`{"detail": "No hay ejemplos para el método '<slug>'."`

Origen del fallo	Mecanismo	Respuesta HTTP	Cuerpo
			`}`
Ruta inexistente (`/api/solve/<slug-desconocido>/`)	Resolución de URL de Django	404	Página HTML de Django
Verbo HTTP no admitido	DRF	405	`{"detail": "Method \"GET\" not allowed."}`
Excepción no prevista	Django	500	Con `DEBUG=0`, página HTML genérica; con `DEBUG=1`, traza

La no convergencia **no** es un error HTTP: es un resultado válido del método, y la interfaz lo muestra con su tabla y sus advertencias. El frontend unifica los dos formatos de 400 en `handleSolve` (`frontend/src/views/SolverView.vue`): si `detail` es una lista o un texto, lo muestra; si no, concatena los mensajes de cada campo. Sin respuesta del servidor, muestra «No fue posible conectar con el servidor. Verifica que el backend esté corriendo.».

7. Frontend

7.1 Arranque de la aplicación

`frontend/src/main.js`



```
import { createApp } from 'vue'
import 'katex/dist/katex.min.css'
import './assets/theme.css'
import './style.css'
import App from './App.vue'
import router, { addMethodRoutes } from './router'
import { loadMethodRegistry, methodRegistry } from './methods/registry'

// Las rutas de resolución salen del catálogo de métodos, así que se carga
// antes de la primera navegación. Si falla, la app arranca igual y la vista
// Resolver muestra el error.
await loadMethodRegistry()
addMethodRoutes(methodRegistry.methods)

createApp(App).use(router).mount('#app')

actor_expansion(minor, n - 1)
    return total
```

El catálogo se descarga **antes** de montar la aplicación porque las rutas `/resolver/<slug>` y `/teoria/<slug>` no existen hasta que se conoce la lista de métodos. Si el usuario abre directamente `/resolver/simpson`, el enrutador debe tener esa ruta registrada en la primera navegación; de lo contrario, la ruta comodín lo redirigiría a `/`. `loadMethodRegistry()` nunca rechaza: si la API falla, deja `status = 'error'` y la aplicación arranca igual para mostrar el mensaje. El `await` de nivel superior es válido porque Vite compila módulos ES modernos.

7.2 Enrutamiento

`frontend/src/router/index.js` usa `createWebHistory()` (URL sin `#`), por lo que el servidor debe devolver `index.html` para cualquier ruta de la SPA (en Docker lo hace nginx con `try_files`; en desarrollo, Vite).

Ruta	Nombre	Componente	Comportamiento
`/`	`home`	`SolverView`	Redirige al primer método del catálogo (`/resolver/jacobi`).
`/teoria`	`theory`	`TheoryView`	Redirige a la teoría del primer método con teoría; convierte anclas antiguas `#metodo-<slug>`.
`/resolver/<slug>`	`solve-<slug>`	`SolverView`	Generada por `addMethodRoutes`; `props: { method: slug }`, `meta.method`.
`/teoria/<slug>`	`theory-<slug>`	`TheoryView`	Generada solo si la categoría declara teoría para el método (`hasTheory`); los 12 la tienen.
`/:pathMatch(.*)*`			Redirige a `/`.

Los nombres de las rutas generadas salen de `frontend/src/methods/routeNames.js` (`solveRouteName`, `theoryRouteName`), un módulo sin dependencias que usan tanto el registro como el contenido de cada categoría. Todas las rutas de resolución usan el mismo componente, de modo que al cambiar entre métodos de una misma categoría Vue reutiliza la instancia y conserva los datos escritos. `scrollBehavior` compensa la cabecera fija (77 px) al navegar a un ancla.

7.3 Cliente HTTP

`frontend/src/api/client.js`



```
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://127.0.0.1:8000/api',
  headers: {
    'Content-Type': 'application/json',
  },
})

export function solveSystem(method, payload) {
  return apiClient
    .post(`/solve/${encodeURIComponent(method)}/`, payload)
    .then((res) => res.data)
}
```

Función	Petición
`fetchMethods()`	`GET /methods/`
`fetchExamples(method)`	`GET /examples/?method=<slug>`
`solveSystem(method, payload)`	`POST /solve/<slug>/`
`previewExpression(equation, variables, { signal })`	`POST /expressions/preview`; `signal` permite cancelarla con `AbortController`.

URL base por entorno. Vite sustituye `import.meta.env.VITE_API_BASE_URL` en tiempo de compilación:

Entorno	Fuente	Valor
Desarrollo (`npm run dev`)	`frontend/.env.development`	`http://127.0.0.1:8000/api`
Docker	`ARG/ENV VITE_API_BASE_URL=/api` en `frontend/Dockerfile`	`/api` (mismo origen, nginx hace de proxy)
Sin variable	Valor de respaldo en `client.js`	`http://127.0.0.1:8000/api`

7.4 Registro de interfaces y contrato de categoría

`frontend/src/methods/registry.js` combina el catálogo del backend con la interfaz de cada categoría:

`frontend/src/methods/registry.js`



```
// markRaw: contienen componentes, que no deben volverse objetos reactivos.
const CATEGORY_UI = {
  linear_system: markRaw(linearSystem),
  nonlinear_system: markRaw(nonlinearSystem),
  polynomial: markRaw(polynomial),
  differentiation: markRaw(differentiation),
  integration: markRaw(integration),
}

function methodUi(category, slug) {
  const base = CATEGORY_UI[category]
  const own = base.methods?.[slug]
  return own ? markRaw({ ...base, ...own }) : base
}
```

Un método cuya categoría no tenga interfaz registrada se descarta con un aviso en la consola. Funciones exportadas: `loadMethodRegistry`, `methodRegistry` (estado de solo lectura con `status` y `methods`), `getMethod(slug)`, `hasTheory(method)`, `methodsByCategory(filter)`, `siblingMethods(slug)`, `solveRouteName`, `theoryRouteName`.

Contrato de categoría. Documentado en el encabezado de `frontend/src/methods/linear/index.js` y verificado contra los usos en `SolverView.vue` y `ResultsTable.vue`:

Campo	Obligatorio	Tipo	Uso
`createStore`	Sí	`() => store`	Crea el estado del formulario. El store expone `state`, `loadExample(example)`, `hasInvalidValues()` y `buildPayload()`.
`configTitle`, `formTitle`	Sí	Texto	Títulos de las tarjetas de configuración y de datos.
`configFields`	Sí (puede ser `null`)	Componente	Campos dentro de la cuadrícula de configuración.
`configExtras`	Sí (puede ser `null`)	Componente	Bloque debajo de la cuadrícula.
`form`	Sí	Componente	Formulario de datos.
`formBindings(store)`	Sí	Función	Props y eventos del formulario.
`invalidMessage`	Sí	Texto	Error cuando `hasInvalidValues()` es verdadero.
`solvedSystem(payload, response)`	Sí	Función	Datos de entrada tal como se

Campo	Obligatorio	Tipo	Uso
			resolvieron (por ejemplo, `A` y `b` reordenados).
`resultSummary`	Sí (puede ser `null`)	Componente	Bloque extra en el resumen de `ResultsTable`.
`iterationDetail`	Sí (puede ser `null`)	Componente	Detalle expandible de cada iteración.
`pdfReport({ result, system })`	Sí (puede ser `null`)	Función	Partes del PDF propias de la categoría.
`theoryPage`	Sí	Componente	Página de teoría; recibe la prop `method`.
`theorySections`	Sí	`{ slug: componente }`	Métodos con teoría; uno sin entrada no aparece en «Teoría ▾».
`methods[slug]`	No	Objeto	Campos que sobrescriben la base para un método.
`beforeIterations`	No	Componente	Sección fija entre la gráfica y la tabla.
`afterIterations`	No	Componente	Sección final (por ejemplo, la comprobación).
`resultView`	No	Componente	Vista completa del resultado en lugar de `ResultsTable`; recibe `result`, `system`, `methodName` y exporta su propio PDF.
`toleranceLabel`, `defaultTolerance`	No	Texto, número	Etiqueta y valor inicial de la tolerancia.
`maxIterationsLabel`, `defaultMaxIterations`	No	Texto, número	Etiqueta y valor inicial del máximo.
`hidesIterationFields(store)`	No	Función	`true` para ocultar tolerancia y máximo (modo tabla).

Uso por categoría:

Categoría	Vista de resultado	Sobrescrituras por método
`linear_system`	`ResultsTable` + `LinearResultSummary`, `LinearIterationDetail`, `LinearVerification`	Ninguna (Jacobi y Gauss-Seidel comparten la interfaz)
`nonlinear_system`	`ResultsTable` + resumen, detalle y comprobación	`newton`: store, formulario, `beforeIterations` (jacobiano), detalle y PDF propios
`polynomial`	`resultView: PolynomialResult`	Ninguna
`differentiation`	`resultView: DifferentiationResult`	Fórmulas y valores por defecto de

Categoría	Vista de resultado	Sobrescrituras por método
		3 y 5 puntos
`integration`	`resultView: IntegrationResult`	`simpson`, `romberg`, `gauss`: store, configuración y vista propios

7.5 Vistas genéricas

`frontend/src/views/SolverView.vue`. Atiende los 12 métodos. Responsabilidades:

- obtener `getMethod(props.method).ui` y los métodos hermanos de la categoría para el selector «Método»;
- recrear el store solo cuando cambia `ui.createStore` (no al pasar de Jacobi a Gauss-Seidel), aplicando la tolerancia y el máximo por defecto de la categoría;
- cargar los ejemplos del método y atender `?ejemplo=<id>` (carga y resuelve);
- `handleSolve()`: valida en el cliente, arma el payload (`buildPayload()` + `tolerance` + `max_iterations`), llama a `solveSystem` y guarda `result` y `solvedSystem`;
- descartar respuestas tardías: cada petición lleva un número (`solveRequestId`) que «Limpiar» incrementa;
- «Limpiar»: confirma con `ConfirmDialog` si hay datos o resultado, recrea el store, restablece valores por defecto, vuelve a montar el formulario (`formKey`) y lleva el foco al primer campo;
- mostrar `ui.resultView` o `ResultsTable` con los *slots* de la categoría.

`frontend/src/views/TheoryView.vue`. Monta `activeMethod.ui.theoryPage` con la prop `method`; si el método no existe o no tiene teoría, muestra un mensaje de error. Las páginas de teoría de cada categoría comparten `components/theory/TheoryLayout.vue`, `AlgorithmSteps.vue` y `WorkedIterations.vue`. Los ejemplos resueltos de diferenciación e integración leen archivos JSON con respuestas reales del solver (por ejemplo, `frontend/src/methods/integration/theory/trapezoidExample.json`).

7.6 Componentes compartidos

Componente (`frontend/src/components/`)	Responsabilidad	Dónde se usa
`NavMenu.vue`	Menú de métodos con panel, buscador y modo móvil.	`App.vue`
`ResultsTable.vue`	Resultado genérico: resumen, gráfica, tabla paginada de iteraciones, PDF; extensible	`SolverView.vue` (lineales y no lineales)

Componente (`frontend/src/components/`)	Responsabilidad	Dónde se usa
	por *slots*.	
`ConvergenceChart.vue`	Gráfica Chart.js de error por iteración (escala logarítmica por defecto) o de aproximaciones; exporta imagen con paleta clara.	`ResultsTable`, diferenciación, integración, polinomios, teoría de diferenciación
`TrapezoidChart.vue`	Curva $f(x)$ y paneles de la regla de integración sombreados.	Integración y su teoría
`MathFormula.vue`	Único punto de la aplicación que invoca KaTeX (`throwOnError: false`, `trust: false`).	Todas las categorías y la teoría
`MatrixInput.vue`	Entrada de A, b y $\mathbf{x}^{(0)}$.	Lineales
`NumericFieldsInput.vue`	Fila de campos numéricos con decimales y fracciones.	No lineales ($\mathbf{x}^{(0)}$), Bairstow (r_0, s_0, coeficientes)
`FunctionTableInput.vue`	Formulario de dos modos: función o tabla de datos.	Diferenciación e integración
`EquationPreview.vue`	Vista previa LaTeX o error de una ecuación.	No lineales, Bairstow, `FunctionTableInput`
`MathSymbolToolbar.vue`	Barra de símbolos que inserta funciones y operadores en el campo.	No lineales, Bairstow, `FunctionTableInput`
`MathSyntaxHelp.vue`	Referencia rápida de la sintaxis que acepta el parser.	No lineales, `FunctionTableInput`
`DerivationSteps.vue`	Desglose término a término de una derivada o integral.	Diferenciación, integración
`GlossaryBlock.vue`	Glosario plegable de símbolos o columnas.	Diferenciación, integración
`InfoTip.vue`	Tooltip accesible por ratón, teclado y toque.	Diferenciación, integración
`ConfirmDialog.vue`	Diálogo de confirmación con `<dialog>` nativo y `showModal()`.	`SolverView.vue` («Limpiar»)
`theory/TheoryLayout.vue`	Contenedor y estilos comunes de las páginas de teoría.	Las 5 páginas de teoría
`theory/AlgorithmSteps.vue`	Pasos del algoritmo como tarjetas numeradas.	Teoría de las 5 categorías
`theory/WorkedIterations.vue`	Sustituciones de cada iteración del ejemplo resuelto.	Teoría de lineales y no lineales

7.7 Composables

`useTheme()` (`frontend/src/composables/useTheme.js`). Estado de módulo compartido por toda la aplicación: `theme` (`'light'` o `'dark'`, solo lectura) y `toggleTheme()`. La clave de almacenamiento es `THEME_STORAGE_KEY = 'theme'`. Detalles en 7.9.

`useExpressionPreviews(getTexts, getVariables)` (`frontend/src/composables/useExpressionPreviews.js`). Para cada fila de ecuación consulta `POST /api/expressions/preview` y expone `previews`, con `status` (`empty`, `pending`, `valid`, `invalid`), `latex`, `error` y `stale`. Aplica:

- *debounce* por fila de `PREVIEW_DEBOUNCE_MS = 450` ms;
- caché por contenido (texto + variables) de hasta 200 entradas;
- cancelación de la petición anterior al programar una nueva;
- descarte de respuestas obsoletas (se guardan en caché pero no se muestran);
- sin parpadeo: mientras llega la respuesta nueva se sigue mostrando la anterior marcada como `stale`.

7.8 Utilidades

Archivo (`frontend/src/utils/`)	Exporta	Responsabilidad
`exportPdf.js`	`exportResultToPdf`, `MARGIN`, `CONTENT_WIDTH`, `ensureSpace`, `sectionTitle`, `paragraph`	PDF A4 genérico (título, datos de entrada aportados por la categoría, resultado, advertencias, gráfica, tabla completa de iteraciones) y primitivas que reutilizan los `pdf.js` de cada categoría. El archivo se descarga como `<fileStem>-AAAA-MM-DD.pdf`.
`iterationSteps.js`	`MAX_DECIMALS`, `formatNumber`, `tableValue`, `H_APPROX_NOTE`, `baseVectorFor`, `buildIterationDetail`	Formato numérico y paso a paso de Jacobi/Gauss-Seidel a partir de la respuesta.
`latexFormulas.js`	Constructores de LaTeX	Fórmulas y sustituciones a partir de `buildIterationDetail`; no calcula nada.
`latexText.js`	Conversión LaTeX → texto	Texto plano para el PDF (`\frac`, `\textcolor`, potencias, subíndices, `\sum`…). No es un parser general.
`numberInput.js`	Parseo de campos	Acepta decimales y fracciones (`6/7`); el valor se guarda sin redondear.

Reglas de presentación numérica del proyecto.

1. **El redondeo es solo de presentación.** Los valores viajan y se guardan con doble precisión; `numberInput.js` no redondea lo que escribe el usuario y `formatNumber` solo formatea.
2. **Seis decimales, sin ceros finales.** `formatNumber` usa `toFixed(6)`, elimina ceros y punto final, y convierte `-0` en `0`; `null` se muestra como – y un no finito como ∞.
3. **Enteros sin decimales.** `1.000000` se muestra `1`. En los sistemas lineales, si toda la solución se ve entera con 6 decimales, el backend hace la comprobación con los enteros, para que la sustitución que se ve sea exacta.
4. **Notación científica solo en magnitudes extremas.** `formatNumber` pasa a notación exponencial con 2 decimales a partir de 10^7 (`SCIENTIFIC_THRESHOLD = 1e7`), para que los casos de divergencia no rompan la tabla; `backend/numeric_methods/solvers/polynomial/latex.py` usa el mismo umbral. En el resto de valores, incluidos los de `backend/numeric_methods/solvers/differentiation/display.py` (`fixed`: «decimal fijo sin ceros finales ni notación científica»), no se usa notación científica.
5. **Coherencia entre lo mostrado y lo calculado.** El paso h y los puntos $x_0 \pm h$ se envían exactos (hasta 12 decimales, o como $h_0/2^{k-1}$ con ≈ 6 decimales); en el paso a paso, los valores de f llevan $d = \min(15,\ 6 + \lceil -\log_{10} h \rceil)$ decimales para que el cociente visible sea coherente con los valores visibles. En las tablas, una h no exacta se muestra como «≈ valor» con la forma exacta en el atributo `title` (`tableValue`).

7.9 Sistema de temas

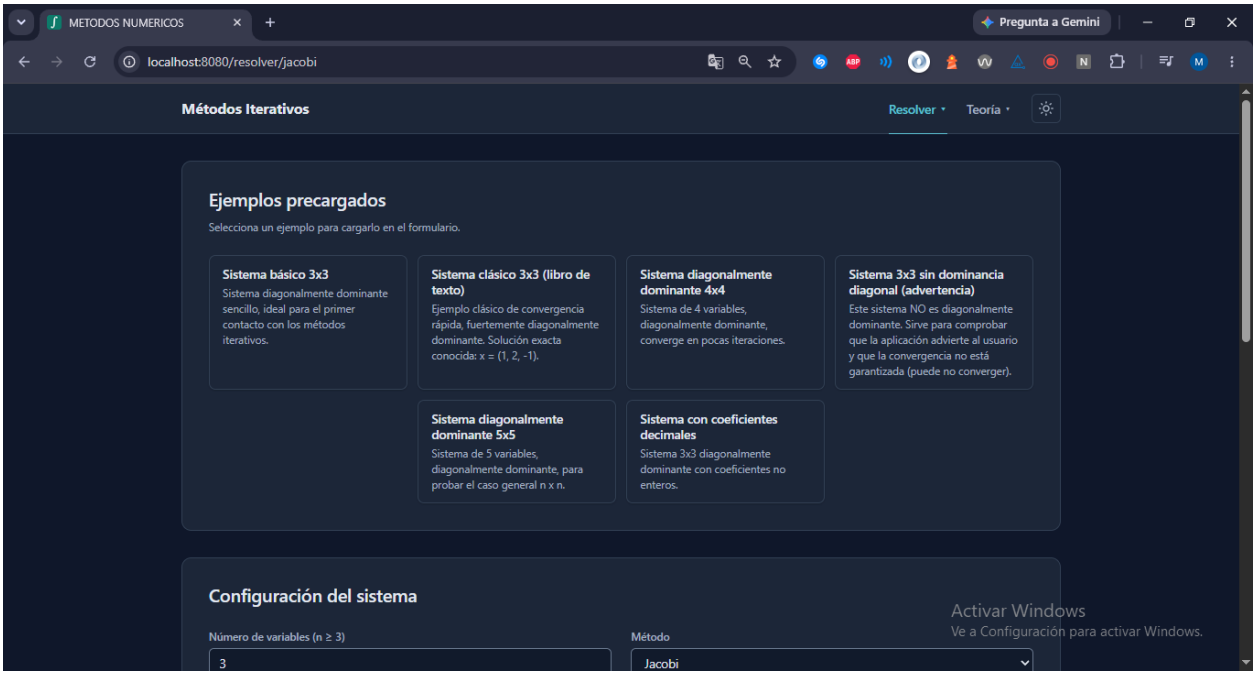
Variables CSS. `frontend/src/assets/theme.css` es el único archivo con colores de la interfaz. Define los mismos *tokens* para el tema claro (`:root, [data-theme='light']`) y para el oscuro: superficies (`--color-page`, `--color-surface`, `--color-input`…), texto, bordes, color de marca (`--color-accent: #0e5a6d` en claro), estados, significado matemático (`--color-value-old` para el valor anterior, `--color-value-new` para el recalculado), series de las gráficas (`--color-series-1` a `--color-series-6`) y `--color-tolerance`. `style.css` y los componentes solo consumen variables.

Detección y persistencia.

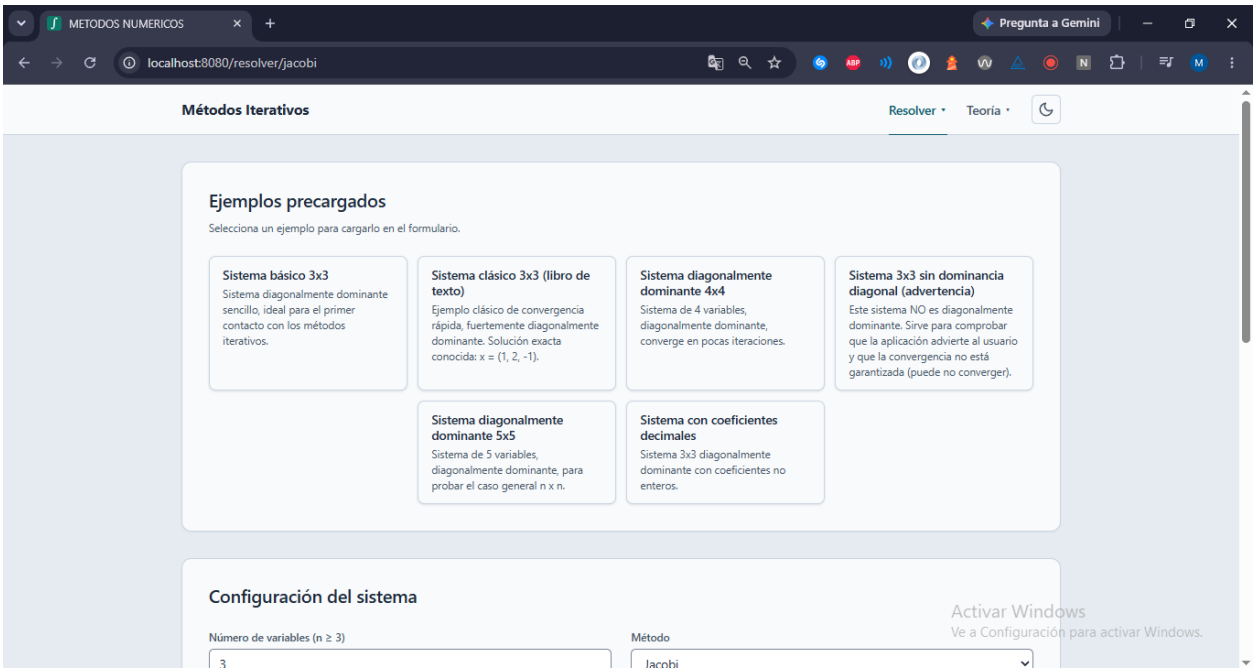
1. Un script en línea en `frontend/index.html`, antes del primer pintado, lee `localStorage.getItem('theme');` si no es `light` ni `dark`, usa `matchMedia('(prefers-color-scheme: dark)')`, y fija `data-theme` en `<html>`. Así la página no destella en blanco al cargar en oscuro.
2. `useTheme.js` lee ese valor inicial, escucha cambios de la preferencia del sistema (solo los aplica si el usuario no eligió un tema) y, al pulsar el botón, guarda la elección en `localStorage` (si el almacenamiento está bloqueado, el cambio vale para la visita).
3. Durante el cambio se añade la clase `theme-transition` 200 ms para animar los colores solo en ese momento.

Por qué el PDF siempre sale en claro. Un PDF se imprime o se lee sobre fondo blanco. jsPDF dibuja con colores fijos en RGB (`exportPdf.js`), y la gráfica que se inserta como imagen se genera con la paleta clara aunque la pantalla esté en oscuro: `ConvergenceChart.vue`, `TrapezoidChart.vue` y `DifferentiationChart.vue` crean un elemento temporal con `data-theme="light"` (`source.dataset.theme = forcedTheme`), leen sus variables con `getComputedStyle` y dibujan la imagen con esos colores. `theme.css` aplica la paleta clara a cualquier elemento con ese atributo precisamente para permitirlo.

CAPTURA T-03 — Modo claro y modo oscuro



Modo oscuro



modo claro

7.10 Menú de navegación

`frontend/src/components/NavMenu.vue` construye el menú a partir del registro (`methodsByCategory`), sin ninguna lista de métodos escrita a mano.

- **Estructura.** «Resolver ▾» y «Teoría ▾» abren el mismo panel en dos modos (los enlaces llevan a `/resolver/<slug>` o a `/teoria/<slug>`). En escritorio (≥ 1024 px, `DESKTOP_QUERY`) el panel muestra las categorías en columnas, ordenadas por `category_order`, con la expresión LaTeX de cada categoría como encabezado; en pantallas menores, un botón «Métodos» lo abre a pantalla completa con las categorías como acordeones.
- **Buscador.** Normaliza el texto (minúsculas, sin tildes) y exige que todas las palabras aparezcan en el nombre, el resumen, la categoría o los sinónimos (`keywords`) del método. Resalta las coincidencias y, si no hay resultados, sugiere «integral», «derivada» o «sistema». El resumen muestra «12 métodos en 5 categorías».
- **Accesibilidad.** Enlace «Saltar al contenido» en `App.vue`; atajo de teclado `/` para abrir el buscador; `Escape` cierra el panel y devuelve el foco al botón que lo abrió; `aria-expanded` en los disparadores y en los acordeones; `role="dialog"` en el modo móvil; navegación con flechas entre los elementos de la cuadrícula; foco en `<main>` tras navegar.

- **Crecimiento.** Un método o una categoría nuevos aparecen solos: el menú lee nombre, resumen, sinónimos y datos de la categoría del catálogo. En modo «Teoría» solo se listan los métodos para los que `hasTheory` es verdadero.

8. API REST

8.1 Convenciones

Aspecto	Convención
Prefijo	Todas las rutas cuelgan de `/api/`. En desarrollo: `http://127.0.0.1:8000/api/`; en Docker: `http://localhost:8080/api/`.
Formato de petición	JSON (`Content-Type: application/json`). DRF también acepta formularios, pero el frontend solo envía JSON.
Formato de respuesta	Siempre JSON (`JSONRenderer` es el único *renderer*), salvo los 404 de rutas inexistentes y los 500, que genera Django en HTML.
Codificación	UTF-8. Los mensajes y etiquetas contienen tildes y símbolos Unicode (ε_t, x_0, $\approx$).
Barra final	Obligatoria en `solve/<slug>/`, `methods/` y `examples/`; opcional en `expressions/preview`.
Estado	Sin estado: no hay sesión, *cookies* ni autenticación. Cada petición es independiente.
Campos desconocidos	Se ignoran. Los ejemplos precargados incluyen claves como `n` o `mode` que algunos serializers no definen.
Valores no finitos	El backend los convierte en `null` para producir JSON estricto.

Códigos de estado usados:

Código	Significado en el sistema
200	Petición resuelta. Incluye los casos en que el método **no converge** (`converged: false`).
400	Entrada inválida (serializer) o método no aplicable a la entrada (solver).
404	Ejemplos de un slug inexistente o ruta inexistente.
405	Verbo HTTP no admitido en la ruta.
500	Error no previsto del servidor.

8.2 Catálogo de endpoints

La API tiene **15 endpoints**: 12 de resolución (uno por método registrado) y 3 auxiliares.

Método HTTP	Ruta	Descripción	Respuestas

Método HTTP	Ruta	Descripción	Respuestas
GET	`/api/methods/`	Catálogo de métodos con los datos de su categoría.	200
GET	`/api/examples/?method=<slug>`	Ejemplos precargados de un método; sin `method`, todos sin repetir.	200, 404
POST	`/api/expressions/preview`	Interpreta una ecuación y devuelve su LaTeX.	200, 400
POST	`/api/solve/jacobi/`	Sistema lineal por Jacobi.	200, 400
POST	`/api/solve/gauss-seidel/`	Sistema lineal por Gauss-Seidel.	200, 400
POST	`/api/solve/punto-fijo/`	Sistema no lineal por Punto Fijo.	200, 400
POST	`/api/solve/newton/`	Sistema no lineal por Newton.	200, 400
POST	`/api/solve/bairstow/`	Raíces de un polinomio por Bairstow.	200, 400
POST	`/api/solve/diferencias-finitas/`	$f'(x_0)$ con diferencias progresiva y regresiva.	200, 400
POST	`/api/solve/tres-puntos/`	$f'(x_0)$ con la fórmula de 3 puntos.	200, 400
POST	`/api/solve/cinco-puntos/`	$f'(x_0)$ con la fórmula de 5 puntos.	200, 400
POST	`/api/solve/trapecio/`	Integral con el método del trapecio.	200, 400
POST	`/api/solve/simpson/`	Integral con Simpson 1/3 y 3/8.	200, 400
POST	`/api/solve/romberg/`	Integral con Romberg.	200, 400
POST	`/api/solve/gauss/`	Integral con cuadratura gaussiana.	200, 400

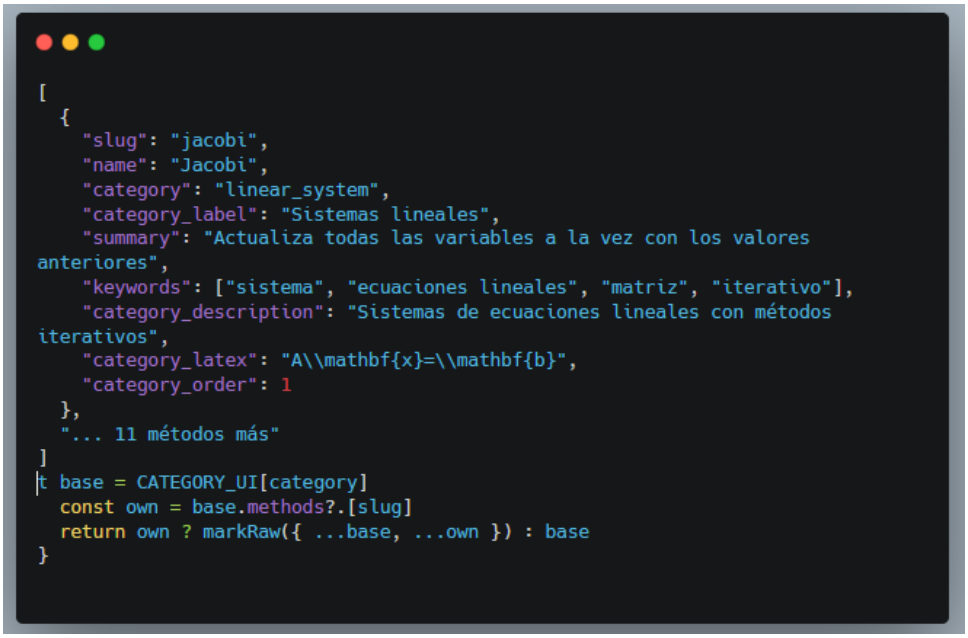
Todas las rutas `solve/` responden 405 a GET. `/api/methods/` y `/api/examples/` responden 405 a POST.

8.3 Especificación de cada endpoint

Los ejemplos JSON de esta sección son respuestas reales obtenidas del contenedor Docker el 13 de noviembre de 2026. Las respuestas largas están recortadas y se indica con `"..."` o con un comentario en el texto.

`GET /api/methods/`

Sin parámetros. Devuelve una lista de 12 objetos (4 337 bytes), ordenada por categoría.



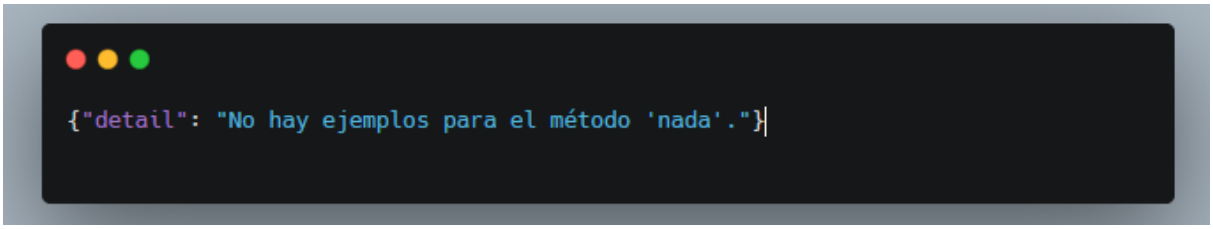
```
[
  {
    "slug": "jacobi",
    "name": "Jacobi",
    "category": "linear_system",
    "category_label": "Sistemas lineales",
    "summary": "Actualiza todas las variables a la vez con los valores
anteriores",
    "keywords": ["sistema", "ecuaciones lineales", "matriz", "iterativo"],
    "category_description": "Sistemas de ecuaciones lineales con métodos
iterativos",
    "category_latex": "A\\mathbf{x}=\\mathbf{b}",
    "category_order": 1
  },
  "... 11 métodos más"
]
t base = CATEGORY_UI[category]
  const own = base.methods?.[slug]
  return own ? markRaw({ ...base, ...own }) : base
}
```

`GET /api/examples/`

Parámetro de consulta	Obligatorio	Descripción
`method`	No	Slug del método. Sin él, se devuelven los 78 ejemplos distintos (24 743 bytes).

Respuesta 200: lista de ejemplos con la estructura de la sección 6.8. `?method=trapecio` devuelve 8 ejemplos (2 593 bytes).

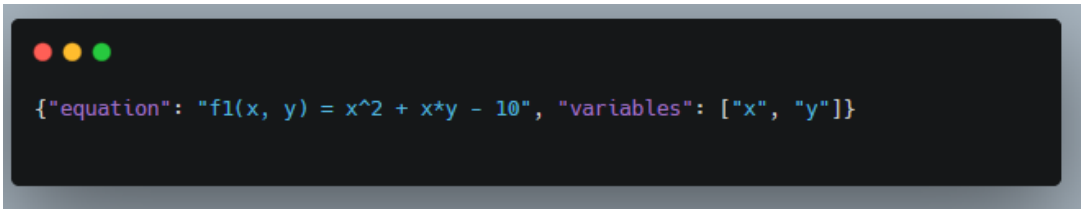
Respuesta 404:



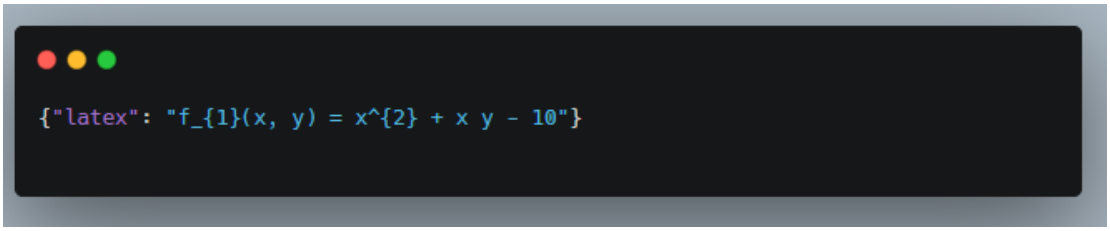
```
{"detail": "No hay ejemplos para el método 'nada'."}
```

`POST /api/expressions/preview`

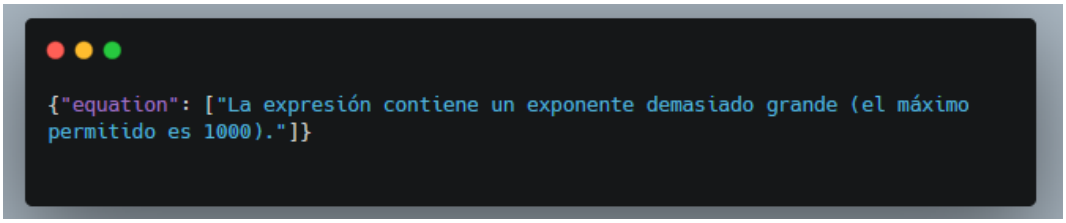
Campo	Tipo	Obligatorio	Descripción
`equation`	texto	Sí	Ecuación o expresión: `x^2 + y = 3`, `f1(x, y) = ...` o `x*y - 1`.
`variables`	lista de texto	Sí	Variables válidas, como máximo 20.



Respuesta 200:

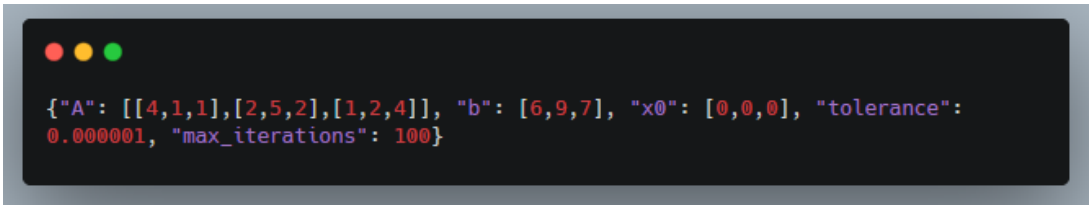


Respuesta 400 (exponente que supera el límite)



`POST /api/solve/jacobi/` y `POST /api/solve/gauss-seidel/`

Campo	Tipo	Obligatorio	Por defecto	Restricciones
`A`	lista de listas de números	Sí		Cuadrada, $n \geq 3$, diagonal sin ceros
`b`	lista de números	Sí		Longitud n
`x0`	lista de números o `null`	No	ceros	Longitud n
`tolerance`	número	No	`0.000001`	≥ 0
`max_iterations`	entero	No	`100`	≥ 1 (se eleva a 6 si es menor)
`auto_reorder`	booleano	No	`true`	Reordena filas para lograr dominancia diagonal

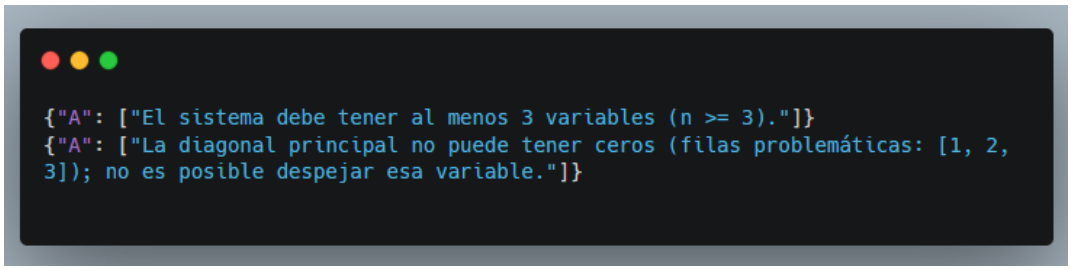


Respuesta 200 (5 350 bytes; se muestran la iteración final y la primera ecuación de la comprobación):



```
{
  "method": "jacobi",
  "category": "linear_system",
  "converged": true,
  "iterations": [
    {"iteration": 1, "x": [1.5, 1.8, 1.75], "error": 1.8},
    {"iteration": 2, "x": [0.6125, 0.5, 0.4750000000000001], "error": 1.3},
    "...",
    {"iteration": 40, "x": [0.9999997626391195, 0.9999996726391748,
0.999999675278147], "error": 8.040037821066548e-07}
  ],
  "iterations_used": 40,
  "solution": [0.9999997626391195, 0.9999996726391748, 0.999999675278147],
  "variables": ["x1", "x2", "x3"],
  "warnings": [],
  "n": 3,
  "is_diagonally_dominant": true,
  "A": [[4.0, 1.0, 1.0], [2.0, 5.0, 2.0], [1.0, 2.0, 4.0]],
  "b": [6.0, 9.0, 7.0],
  "reordered": false,
  "row_order": null,
  "verification": {
    "solution": [1.0, 1.0, 1.0],
    "equations": [
      {"row": 0,
       "terms": [{"coefficient": 4.0, "variable": "x1", "value": 1.0, "term":
4.0},
                 {"coefficient": 1.0, "variable": "x2", "value": 1.0, "term":
1.0},
                 {"coefficient": 1.0, "variable": "x3", "value": 1.0, "term":
1.0}],
       "lhs": 6.0, "b": 6.0, "residual": 0.0, "is_zero": true},
      "..."
    ],
    "max_residual": 0.0,
    "reordered": false,
    "rounded_to_integers": true
  }
}
```


Errores posibles (400):



```
{"A": ["El sistema debe tener al menos 3 variables (n >= 3)."]}
{"A": ["La diagonal principal no puede tener ceros (filas problemáticas: [1, 2,
3]); no es posible despejar esa variable."]}
```


`POST /api/solve/punto-fijo/` y **`POST /api/solve/newton/`**

Campo	Tipo	Obligatorio	Por defecto	Restricciones
`equations`	lista de texto	Sí		2 a 6 ecuaciones parseables
`variables`	lista de texto	Sí		Una por ecuación; nombres válidos, no reservados, sin repetir. En Punto Fijo, la ecuación i se despeja para la variable i; en Newton fijan el orden de las columnas del jacobiano.

Campo	Tipo	Obligatorio	Por defecto	Restricciones
`x0`	lista de números o `null`	No	ceros	Longitud n
`tolerance`	número	No	`0.000001`	≥ 0
`max_iterations`	entero	No	`100`	≥ 1

Petición (ejemplo `trigonometrico-2x2` de Punto Fijo):

```
{"variables": ["x", "y"], "equations": ["3*x - cos(y) - 1 = 0", "4*y - sin(x) - 2 = 0"], "x0": [0, 0], "tolerance": 0.000001, "max_iterations": 100}
```

Respuesta 200: campos comunes (6 iteraciones, `variables: ["x", "y"]`, solución $\approx (0.600449,\ 0.641253)$) y campos propios `n`, `x0`, `variables_latex`, `equations`, `failure` y `verification`. Newton añade `jacobian`, y cada `IterationStep.extra` contiene F, J, D, D_i, las matrices J_i y $\Delta\mathbf{x}$.

Error 400 del solver (despeje ambiguo), con el formato `detail`:

```
{"detail": ["La ecuación 1 admite 2 despejes reales de x (x = -sqrt(4 - y^2); x = sqrt(4 - y^2)), así que el despeje automático es ambiguo. Reescribe la ecuación para que x tenga un único despeje, por ejemplo con x sola en el lado izquierdo: x = ..."]}
```

`POST /api/solve/bairstow/`

Campo	Tipo	Obligatorio	Por defecto	Restricciones
`polynomial`	texto	Uno de los dos		Polinomio en `x`, grado 3 a 10
`coefficients`	lista de números	Uno de los dos		Orden descendente $a_n, \ldots, a_0$; $a_n \neq 0$
`r0`, `s0`	número	No	`-1`, `-1`	Finitos, magnitud $\leq 10^{100}$
`tolerance`	número	No	`0.0001`	En porcentaje; > 0
`max_iterations`	entero	No	`100`	Por factor; ≥ 1

```
{"polynomial": "x^3 - 2x^2 - 5x + 6", "r0": -1, "s0": -1, "tolerance": 1, "max_iterations": 100}
```

Respuesta 200: `variables: ["r", "s"]`, y los campos propios `polynomial`, `r0`, `s0`, `tolerance_percent`, `zero_roots`, `factors` (cada factor con sus iteraciones y tablas sintéticas), `roots`, `factorization`, `notes` y `failure`.

POST /api/solve/diferencias-finitas/, /tres-puntos/, /cinco-puntos/

Campo	Tipo	Por defecto	Restricciones
`mode`	`"function"` o `"table"`	`"function"`	
`formula`	texto	`"both"` (diferencias) o `"all"` (3 y 5 puntos)	Diferencias: `forward`, `backward`, `both`. 3 puntos: `punto_medio`, `extremo_adelante`, `extremo_atras`, `all`. 5 puntos: `punto_medio_5`, `extremo_adelante_5`, `extremo_atras_5`, `all`.
`function`	texto		Modo función: obligatorio, variable `x`
`x0`	número		Modo función: obligatorio, finito
`h0`	número	`0.1`	> 0
`tolerance`	número	`0.0001` (diferencias); `0.000001` (3 y 5 puntos)	> 0
`max_iterations`	entero	`25`	6 a 30 refinamientos
`points`	lista de pares `[x, y]`		Modo tabla: 2 (diferencias), 3 o 5 a 20 puntos; x estrictamente creciente; 3 y 5 puntos exigen espaciado uniforme
`compare_function`	texto	`""`	Modo tabla, opcional

```
{"mode": "function", "formula": "all", "function": "x*exp(x)", "x0": 2,
"h0": 0.1, "tolerance": 0.000001, "max_iterations": 25}
```

Respuesta 200 (`tres-puntos`): 11 filas, `variables: ["punto_medio", "extremo_adelante", "extremo_atras"]` (una columna por fórmula), solución ≈ 22.167168 en las tres. Campos propios: `mode`, `x0`, `x0_display`, `h0`, `tolerance`, `max_iterations`, `exact`, `differentiable`, `verification`, `comparison`, `rounding_text`, `min_rows`, `third_derivative`, `chart`, `formula`, `formulas`, `function`, `sampling_points`.

POST /api/solve/trapecio/, /simpson/, /romberg/, /gauss/

Campo	Tipo	Por defecto	Restricciones
`mode`	`"function"` o `"table"`	`"function"`	Gauss solo admite `function`
`function`	texto		Modo función: obligatorio, variable `x`
`a`, `b`	número		Modo función: obligatorios, finitos, $a < b$
`n0`	entero	`1`	1 a 1000 (trapecio); ignorado en Romberg y Gauss

Campo	Tipo	Por defecto	Restricciones
`tolerance`	número	`0.000001`	> 0
`max_iterations`	entero	`20` (trapecio, Simpson); `15` (Romberg); `12` (Gauss)	6 a 22; Romberg 6 a 20 filas; Gauss 6 a 20 puntos
`points`	lista de pares		Trapecio 2 a 30 (espaciado libre); Simpson 3 a 30 uniformes; Romberg 3, 5, 9, 17 o 33 uniformes
`compare_function`	texto	`""`	Modo tabla, opcional
`rule`	texto	`"both"`	Solo Simpson: `simpson13`, `simpson38`, `both`
`n0_13`, `n0_38`	entero	`2`, `3`	Solo Simpson: par 2 a 1000; múltiplo de 3, 3 a 999
`subintervals`	entero	`1`	Solo Gauss: 1 a 50

Petición (ejemplo 1 del trapecio):

```
{"mode": "function", "function": "0.2 + 25x - 200x^2 + 675x^3 - 900x^4 +
400x^5", "a": 0, "b": 0.8, "n0": 1, "tolerance": 0.000001,
"max_iterations": 20}
```

Respuesta 200. Pesa 571 060 bytes, casi todos en `iterations`, porque cada fila lleva su paso a paso completo; se muestran los campos de primer nivel y se recortan los objetos grandes:

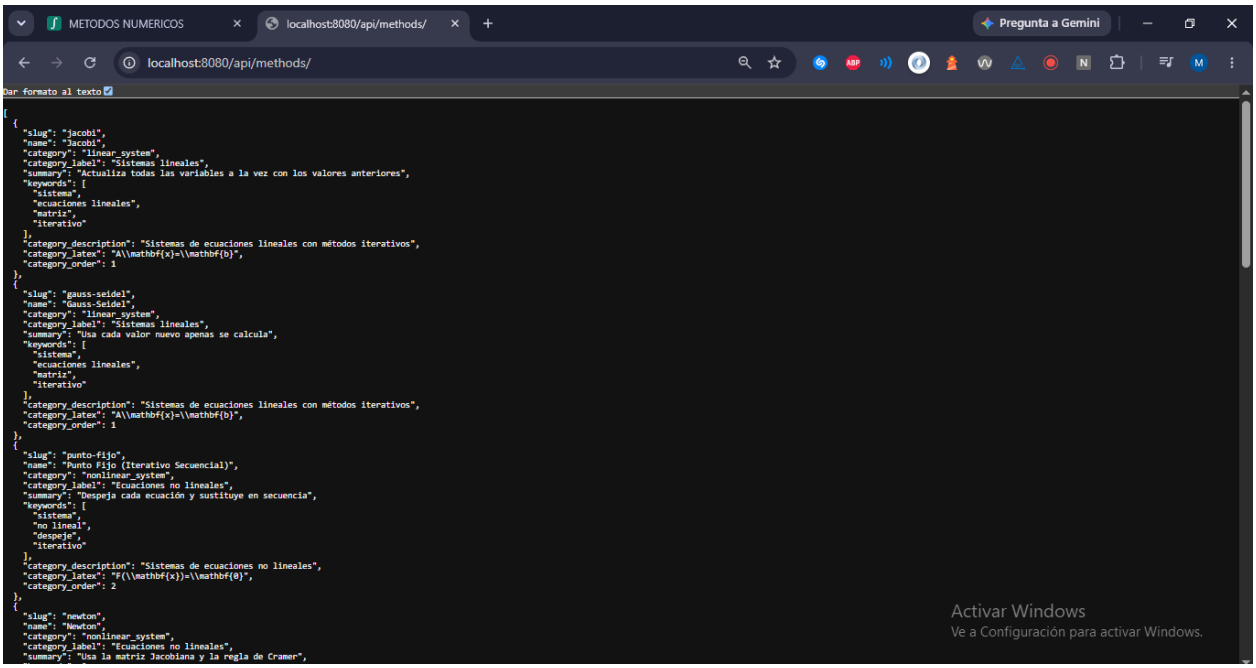

```
{
  "method": "trapecio",
  "category": "integration",
  "converged": true,
  "iterations": [
    {"iteration": 1, "x": [0.17280000000001566], "error": null,
     "extra": {"n": 1, "h": {"value": 0.8, "exact": true, "text": "0.8",
"latex": "0.8", "approx": null},
               "T": 0.17280000000001566, "true_error": 1.4677333333333178,
               "relative_error": 89.46684005201465, "approx_error":
2.560000000000013,
               "bound": 17.06666666666667, "...": "..."}},
    "...",
    {"iteration": 13, "x": [1.6405331807454469], "error":
4.5776361345950534e-07, "extra": {"n": 4096, "...": "..."}}
  ],
  "iterations_used": 13,
  "solution": [1.6405331807454469],
  "variables": ["T"],
  "warnings": [],
  "mode": "function",
  "a": 0.0,
  "b": 0.8,
  "n0": 1,
  "tolerance": 1e-06,
  "max_iterations": 20,
```

```
  "integral": {"kind": "exact", "value": 1.6405333333333334, "label":
"exacta", "special": null, "timed_out": false,
               "antiderivative": {"latex": "0.2 x + 12.5 x^{2} -
\\frac{200}{3} x^{3} + 168.75 x^{4} - 180 x^{5} + \\frac{200}{3} x^{6}",
"...": "..."}},
  "min_rows": 6,
  "rule": {"key": "trapecio", "name": "Trapecio", "factor": [1, 2],
"ends": 1, "interior": [2], "order": 2, "expected_ratio": 0.25, "...":
"..."},
  "notes": [],
  "sampling_points": 2001,
  "...": "a_display, b_display, width_display, function, mean_second, M2,
curve, chart_rows, verification"
}
```


Error 400 (expresión con un nombre fuera de la lista blanca):

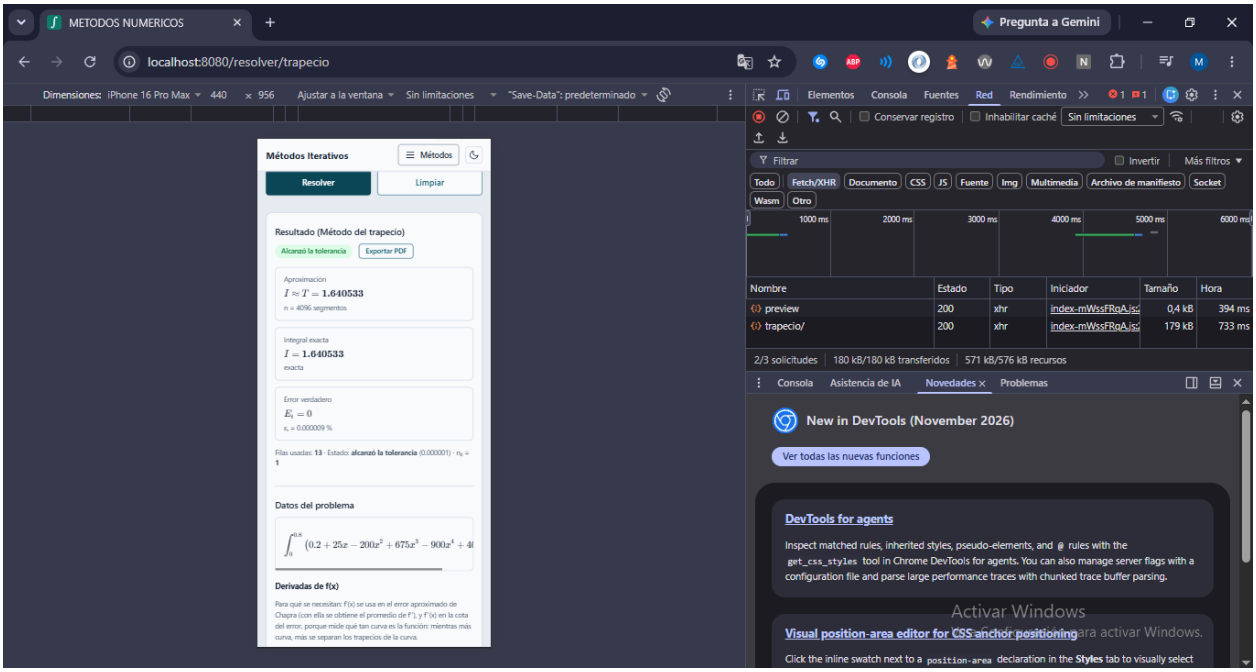

```
{"function": ["f(x): '__import__' no es una variable declarada ni una
función conocida."]}
```


CAPTURA T-04 — Catálogo de métodos en el navegador



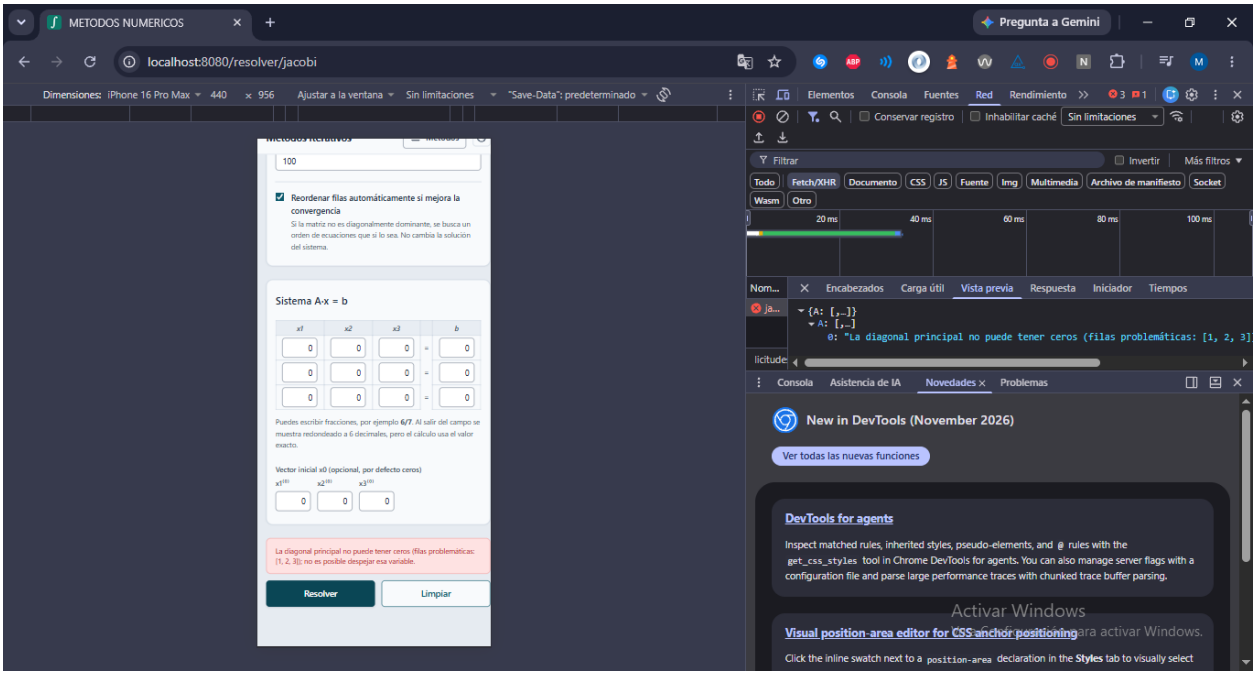
Catálogo de métodos en el navegador

CAPTURA T-05 — Petición POST de resolución en DevTools



Petición POST de resolución en DevTools

CAPTURA T-06 — Petición rechazada con 400



Petición rechazada con 400

8.4 Contrato de respuesta común

Campos comunes (`SolverResult.to_dict()`, presentes en los 12 métodos):

Campo	Tipo	Descripción

Campo	Tipo	Descripción
`method`	texto	Slug del método.
`category`	texto	Clave de la categoría.
`converged`	booleano	`true` si se alcanzó la tolerancia respetando el mínimo de iteraciones.
`iterations`	lista	Filas `{iteration, x, error[, extra]}`. `x` sigue el orden de `variables`; `error` puede ser `null`.
`iterations_used`	entero	Longitud de `iterations`.
`solution`	lista	Valores finales, en el orden de `variables`.
`variables`	lista de texto	Nombres de las columnas de `x` y de `solution`.
`warnings`	lista de texto	Advertencias (sin dominancia diagonal, no convergencia, error absoluto en Bairstow…).

Significado de `variables` por categoría:

Categoría	`variables`	`solution`
Lineales	`["x1", ..., "xn"]`	Vector solución
No lineales	Nombres declarados por el usuario	Punto solución
Polinomios	`["r", "s"]`	r y s del último factor iterado
Diferenciación	Claves de las fórmulas calculadas (`forward`, `punto_medio`…)	Última aproximación de cada fórmula
Integración	`["T"]`, `["R"]`, `["G"]` o los nombres de las reglas de Simpson	Última aproximación

Campos propios por categoría (claves de `meta`, observadas en respuestas reales del modo función):

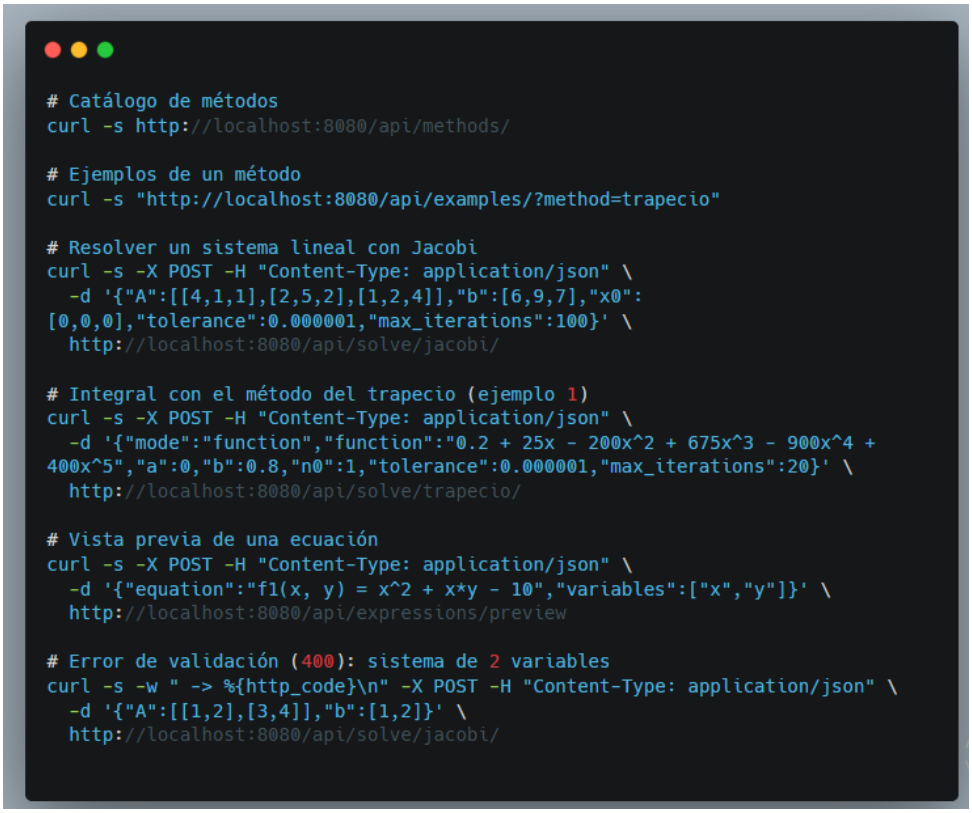
Categoría / método	Campos propios
Lineales	`n`, `is_diagonally_dominant`, `A`, `b`, `reordered`, `row_order`, `verification`
Punto Fijo	`n`, `x0`, `variables_latex`, `equations`, `failure`, `verification`
Newton	Los de Punto Fijo y `jacobian`
Bairstow	`polynomial`, `r0`, `s0`, `tolerance_percent`, `zero_roots`, `factors`, `roots`, `factorization`, `notes`, `failure`
Diferencias	`mode`, `x0`, `x0_display`, `h0`, `tolerance`, `max_iterations`, `exact`, `differentiable`, `verification`, `min_rows`, `curvature`, `chart`, `formula`, `formulas`, `function`, `sampling_points`
3 puntos	Los de diferencias sin `curvature`, más `comparison`, `rounding_text`, `third_derivative`

Categoría / método	Campos propios
5 puntos	Los de 3 puntos sin `third_derivative`, más `comparison_title`, `derivative_pattern`, `fifth_derivative`
Trapecio	`mode`, `a`, `b`, `a_display`, `b_display`, `width_display`, `n0`, `tolerance`, `max_iterations`, `function`, `integral`, `min_rows`, `mean_second`, `M2`, `curve`, `chart_rows`, `notes`, `verification`, `rule`, `sampling_points`
Simpson	`mode`, `a`, `b`, `a_display`, `b_display`, `width_display`, `rule_choice`, `tolerance`, `max_iterations`, `function`, `integral`, `min_rows`, `mean_fourth`, `M4`, `curve`, `rules`, `notes`, `verification`, `sampling_points`
Romberg	`mode`, `a`, `b`, `a_display`, `b_display`, `width_display`, `tolerance`, `max_iterations`, `function`, `integral`, `min_rows`, `columns`, `evaluations`, `cost`, `curve`, `chart_rows`, `notes`, `verification`
Gauss	`mode`, `a`, `b`, `a_display`, `b_display`, `width_display`, `subintervals`, `H_display`, `half_display`, `center_display`, `half_width_display`, `centers`, `undefined_ends`, `tolerance`, `max_iterations`, `function`, `derivatives_timed_out`, `integral`, `min_rows`, `max_bound_points`, `evaluations`, `cost`, `curve`, `chart_rows`, `notes`, `sampling_points`, `verification`

El modo tabla de diferenciación e integración devuelve un conjunto de campos distinto (puntos, segmentos y valores por punto); el componente de resultado de cada método distingue los modos por `mode`.

8.5 Ejemplos de uso

Los comandos siguientes se ejecutaron tal cual en Git Bash contra el contenedor Docker (`http://localhost:8080`). En modo desarrollo, sustituir la base por `http://127.0.0.1:8000`. En Windows PowerShell 5.1, `curl` es un alias de `Invoke-WebRequest`: debe escribirse `curl.exe`, y las comillas del JSON requieren otro escapado, por lo que se recomienda Git Bash.



```
# Catálogo de métodos
curl -s http://localhost:8080/api/methods/

# Ejemplos de un método
curl -s "http://localhost:8080/api/examples/?method=trapecio"

# Resolver un sistema lineal con Jacobi
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"A":[[4,1,1],[2,5,2],[1,2,4]],"b":[6,9,7],"x0":
[0,0,0],"tolerance":0.000001,"max_iterations":100}' \
  http://localhost:8080/api/solve/jacobi/

# Integral con el método del trapecio (ejemplo 1)
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"mode":"function","function":"0.2 + 25x - 200x^2 + 675x^3 - 900x^4 +
400x^5","a":0,"b":0.8,"n0":1,"tolerance":0.000001,"max_iterations":20}' \
  http://localhost:8080/api/solve/trapecio/

# Vista previa de una ecuación
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"equation":"f1(x, y) = x^2 + x*y - 10","variables":["x","y"]}' \
  http://localhost:8080/api/expressions/preview

# Error de validación (400): sistema de 2 variables
curl -s -w " -> %{http_code}\n" -X POST -H "Content-Type: application/json" \
  -d '{"A":[[1,2],[3,4]],"b":[1,2]}' \
  http://localhost:8080/api/solve/jacobi/
```

9. Modelo de datos

El sistema no persiste datos de dominio. `backend/numeric_methods/models.py` no declara modelos y `backend/numeric_methods/migrations/` solo contiene `__init__.py`. Esto implica:

- **Sin base de datos de dominio:** no hay tablas de ejercicios, resultados ni usuarios propios.
- **Sin autenticación:** la API es pública; no hay sesiones, *tokens* ni permisos por usuario.
- **Sin estado entre peticiones:** el resultado depende solo del cuerpo de la petición. Dos peticiones iguales producen la misma respuesta, lo que permite repetir, cachear o balancear peticiones sin coordinación. La única memoria entre peticiones es la caché de antiderivadas de `exact.py` (hasta 256 entradas por proceso de gunicorn), que no altera los resultados.

Tablas que Django crea de todos modos. `INSTALLED_APPS` incluye `django.contrib.admin`, `auth`, `contenttypes` y `sessions`, que traen sus propias migraciones. `python manage.py migrate` crea sus tablas en `backend/db.sqlite3` (usuarios, grupos, permisos, tipos de contenido, sesiones y registro del admin). Ninguna se usa para la lógica del sistema; se ejecuta `migrate` para que Django arranque sin advertencias de migraciones pendientes y para que el

sitio `admin/` sea funcional si algún día se usa. En Docker, el `CMD` del `backend/Dockerfile` ejecuta `python manage.py migrate --noinput` en cada arranque del contenedor; el archivo vive dentro del contenedor y se pierde al recrearlo, sin consecuencias.

Dónde viven los datos que sí existen.

Dato	Ubicación	Persistencia
Catálogo de métodos y categorías	`backend/numeric_methods/registry.py`	Código fuente
Ejemplos precargados	`backend/numeric_methods/examples_data.py`	Código fuente
Ejemplos resueltos de la teoría	`frontend/src/methods/*/theory/*.json`	Código fuente (verificados por tests del backend)
Tema elegido por el usuario	`localStorage`, clave `theme`	Navegador del usuario
Datos del formulario y resultado en pantalla	Memoria de la SPA	Hasta recargar la página
PDF exportado	Descarga del navegador	Archivo del usuario

10. Seguridad

10.1 Superficie de ataque

La API es pública y sin autenticación. El riesgo principal no está en el acceso, sino en el **contenido**: los usuarios envían texto que el servidor interpreta como matemáticas (`function`, `equations`, `polynomial`, `compare_function`, `equation`). La función de sympy que convierte texto en expresión, `parse_expr`, evalúa el texto con `eval()` de Python incluso con `evaluate=False`; con su configuración por defecto, una entrada como `__import__("os").getcwd()` se ejecutaría. Un segundo riesgo es la **denegación de servicio**: entradas cortas que cuestan muy poco al parsear pero cuelgan el proceso al evaluarlas o manipularlas (`9**9**9`, `x**(10**50)`, despejes de grado alto, antiderivadas que no terminan).

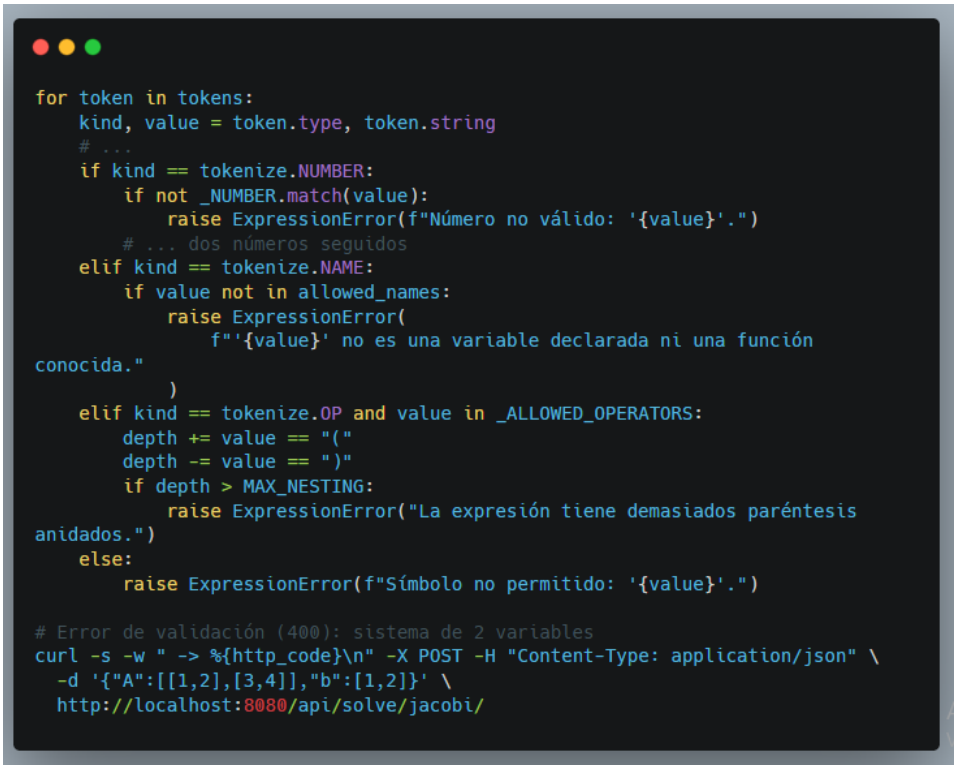
Entradas numéricas (matrices, puntos, límites) se validan como tipos numéricos de DRF y con comprobaciones de finitud y magnitud; nunca se interpretan como código.

10.2 Defensas del parser

`backend/numeric_methods/expressions/parser.py` aplica tres capas, en este orden.

Capa 1: validación léxica con lista blanca, antes de llamar a sympy. El texto se tokeniza con `tokenize` de Python y solo se aceptan números, los operadores `+ - * / ** ^ ( ) ,` y nombres de una lista blanca (variables declaradas, funciones y constantes permitidas). Se rechazan atributos (`.`), cadenas, corchetes, `lambda`, nombres desconocidos, saltos de línea, la barra invertida y los caracteres de control o invisibles. También se rechazan dos números seguidos (`007`, `1.5.2`), que la multiplicación implícita convertiría silenciosamente en productos.

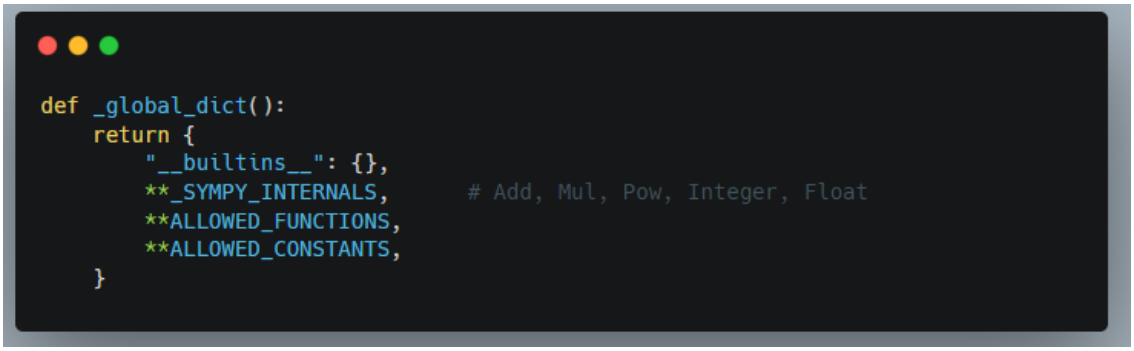
`backend/numeric_methods/expressions/parser.py`



```
for token in tokens:
    kind, value = token.type, token.string
    # ...
    if kind == tokenize.NUMBER:
        if not _NUMBER.match(value):
            raise ExpressionError(f"Número no válido: '{value}'.")
        # ... dos números seguidos
    elif kind == tokenize.NAME:
        if value not in allowed_names:
            raise ExpressionError(
                f"'{value}' no es una variable declarada ni una función
conocida."
            )
    elif kind == tokenize.OP and value in _ALLOWED_OPERATORS:
        depth += value == "("
        depth -= value == ")"
        if depth > MAX_NESTING:
            raise ExpressionError("La expresión tiene demasiados paréntesis
anidados.")
    else:
        raise ExpressionError(f"Símbolo no permitido: '{value}'.")

# Error de validación (400): sistema de 2 variables
curl -s -w " -> %{http_code}\n" -X POST -H "Content-Type: application/json" \
  -d '{"A":[[1,2],[3,4]],"b":[1,2]}' \
  http://localhost:8080/api/solve/jacobi/
```

Capa 2: entorno restringido para `parse_expr`. El `eval` interno solo ve los constructores imprescindibles de sympy, las funciones y constantes de la lista blanca y un `__builtins__` vacío, que impide que Python inyecte `open`, `eval` o `__import__`. `Symbol` no está en el diccionario: si estuviera, la transformación `auto_symbol` podría crear símbolos con cualquier nombre. Las variables declaradas van en `local_dict`.



```python
def _global_dict():
    return {
        "__builtins__": {},
        **_SYMPY_INTERNALS,      # Add, Mul, Pow, Integer, Float
        **ALLOWED_FUNCTIONS,
        **ALLOWED_CONSTANTS,
    }
```

Capa 3: validación del árbol ya parseado. `validate_expression_tree` recorre el árbol y solo admite símbolos declarados, números racionales o de coma flotante finitos, `pi`, `e`, sumas, productos, potencias y las clases de función de la lista blanca. Calcula con floats de Python el valor de cada subárbol puramente numérico, sin sympy ni enteros exactos, de modo que comprobar `9**9**9` es instantáneo (desborda y se rechaza).

Otras defensas relacionadas.

- **KaTeX sin `trust`.** `frontend/src/components/MathFormula.vue` es el único punto que invoca KaTeX, con `trust: false` y `throwOnError: false`: los comandos LaTeX que podrían insertar enlaces o HTML no se ejecutan.
- **Usuario sin privilegios en Docker.** El backend corre como el usuario `app` (UID 1000), no como `root`.
- **Backend no expuesto.** En Docker, el puerto 8000 no se publica; solo nginx llega a gunicorn.
- **Sin evaluación simbólica repetida.** Las iteraciones evalúan funciones compiladas con `lambdify` sobre floats.

10.3 Límites contra denegación de servicio

Límite	Valor	Dónde	Qué evita
Longitud de la expresión	300 caracteres (`MAX_LENGTH`)	`parser.py`	Entradas arbitrariamente largas
Anidamiento de paréntesis	30 (`MAX_NESTING`)	`parser.py`	Recursión profunda en el parser
Profundidad del árbol	40 niveles (`MAX_TREE_DEPTH`)	`parser.py`	Recursión en la validación y en sympy
Magnitud de los números	10^{100} (`MAX_MAGNITUDE`)	`parser.py`; también `r0`, `s0`, coeficientes, puntos, límites	Desbordamientos y cálculos

Límite	Valor	Dónde	Qué evita
			con enteros gigantes
Exponente numérico	1000 (`MAX_EXPONENT`)	`parser.py`	`x**(10**50)` con enteros exactos
División literal entre cero	Rechazada	`parser.py` (`_inspect`)	Valores no finitos
Variables en la vista previa	20 (`MAX_PREVIEW_VARIABLES`)	`serializers/expressions.py`	Trabajo excesivo por pulsación de tecla
Grado para despejar (Punto Fijo)	4 (`MAX_SOLVE_DEGREE`)	`solvers/nonlinear/fixed_point.py`	`sympy.solve` de varios segundos
Grado para expandir (Bairstow)	200 (`MAX_EXPAND_DEGREE`)	`solvers/polynomial/validation.py`	Expandir `((x^1000)^1000)`
Ecuaciones por sistema	6 (`MAX_EQUATIONS`)	`solvers/nonlinear/common.py`	Despejes y determinantes en cascada
Tamaño de Cramer	8 × 8 (`MAX_SIZE`)	`linalg/cramer.py`	Costo factorial
Reordenamiento por permutaciones	$n \leq 8$ (`MAX_BRUTE_FORCE_SIZE`)	`solvers/linear/dominance.py`	$n!$ permutaciones
Antiderivada y derivadas de la cota de Gauss	3 s (`TIME_LIMIT`), proceso hijo que se mata	`solvers/integration/exact.py`	`sympy.integrate` que no termina
Segmentos del trapecio y Simpson	2^{21} (`MAX_SEGMENTS`)	`solvers/integration/trapezoid.py`	Evaluaciones de f desmesuradas
Comparación de costo de Romberg y Gauss	2^{22} evaluaciones (`COST_LIMIT`)	`solvers/integration/romberg.py`	Simulaciones de costo desmesuradas
Refinamientos y filas	30 (diferenciación), 22 (trapecio, Simpson), 20 (Romberg, Gauss)	Serializers	Tablas desmesuradas
Tiempo por petición en	120 s (`gunicorn --timeout 120`, `proxy_read_timeout`	`backend/Dockerfile`, `frontend/nginx.conf`	Workers bloqueados

Límite	Valor	Dónde	Qué evita
Docker	`120s`)		indefinidamente
Tamaño del cuerpo	2,5 MB (valor por defecto de Django, `DATA_UPLOAD_MAX_MEMORY_SIZE`)	Django	Cuerpos enormes

10.4 Configuración segura en producción

Variable	Recomendación	Motivo
`DJANGO_DEBUG`	`0` (es el valor por defecto en `docker-compose.yml`)	Con `DEBUG` activo, Django muestra trazas, configuración y rutas en las páginas de error. Cualquier valor distinto de `'1'` lo desactiva.
`DJANGO_SECRET_KEY`	Una clave larga y aleatoria, definida en `.env` (no versionado)	El valor por defecto de `settings.py` es público en el repositorio y el de `docker-compose.yml` también. Aunque el sistema no usa sesiones ni firmas, Django la usa para firmar datos internos.
`DJANGO_ALLOWED_HOSTS`	En una red local, `*` es aceptable porque solo nginx llega al backend. Expuesto en internet, el dominio real.	Evita ataques de cabecera `Host`.
`DJANGO_CORS_ALLOWED_ORIGINS`	No definirla en Docker (mismo origen). En desarrollo, el valor por defecto.	Ampliar CORS solo hace falta si otro origen consume la API.
`DJANGO_CSRF_TRUSTED_ORIGINS`	No necesaria para la API	Solo aplica a vistas con sesión (admin).

Generar una clave secreta:

```
python -c "import secrets; print(secrets.token_urlsafe(50))"
```

10.5 Pendientes de seguridad

Si el sistema se publica en internet, faltan:

1. **HTTPS.** nginx escucha solo en HTTP (puerto 80). Hace falta un certificado (por ejemplo, terminación TLS en un proxy inverso) y, en Django, `SECURE_PROXY_SSL_HEADER`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE` y HSTS.

2. **Limitación de tasa.** No hay *rate limiting* por IP. Cada petición de integración puede consumir hasta 3 s de CPU en un proceso hijo; con 3 workers, pocas peticiones simultáneas saturan el servicio. Se recomienda `limit_req` en nginx o la limitación de DRF.
3. **Cabeceras de seguridad en nginx.** Faltan `Content-Security-Policy`, `X-Content-Type-Options`, `Referrer-Policy` y `X-Frame-Options` para los archivos estáticos.
4. **Clave secreta obligatoria.** `settings.py` tiene un valor por defecto; en producción debería fallar el arranque si `DJANGO_SECRET_KEY` no está definida.
5. **Sitio `admin/`.** Está en `core/urls.py` aunque no se usa; conviene retirarlo si se expone el backend directamente.
6. **Límite de tiempo global por petición.** Los despejes de Punto Fijo y las derivadas simbólicas fuera de `exact.py` se acotan por grado, no por tiempo; el único límite global es el `--timeout 120` de gunicorn.
7. **Auditoría de dependencias.** No hay un proceso automatizado (`pip-audit`, `npm audit`) en el repositorio.

11. Despliegue

11.1 Requisitos previos

Requisito	Versión mínima	Modo	Fuente del requisito
Python	3.11 (desarrollo verificado con 3.11.0)	Desarrollo	Sintaxis `float | None` de `solvers/base.py` (3.10+); Django 5.2 exige 3.10+
pip	Incluido en Python	Desarrollo	
Node.js	20.19 o 22.12	Desarrollo	Campo `engines` de Vite 8.3.0 y de `@vitejs/plugin-vue` 6.0.8
npm	Incluido en Node.js	Desarrollo	
Docker Desktop (en Windows, con WSL 2)	Con Docker Compose v2 (subcomando `docker compose`)	Docker	`docker-compose.yml` y scripts `.bat`
Navegador	Chrome, Edge o Firefox actuales	Ambos	ES modules, `<dialog>`, `await` de nivel superior en el código compilado
Puertos libres	8000 y 5173 (desarrollo); 8080 (Docker)	Ambos	

11.2 Modo desarrollo

El modo desarrollo usa dos procesos: Django con `runserver` (recarga al guardar código Python) y Vite con recarga en caliente del frontend. No usa Docker ni `.env`.

Backend (primera terminal):

```
cd backend
python -m venv .venv                # solo la primera vez
.venv\Scripts\activate.bat          # Windows (CMD)
.venv\Scripts\Activate.ps1          # Windows (PowerShell)
source .venv/Scripts/activate       # Windows (Git Bash)
source .venv/bin/activate           # Linux / macOS
pip install -r requirements.txt
python manage.py migrate            # tablas internas de Django
python manage.py runserver 8000
```

La API queda en `http://127.0.0.1:8000/api/`.

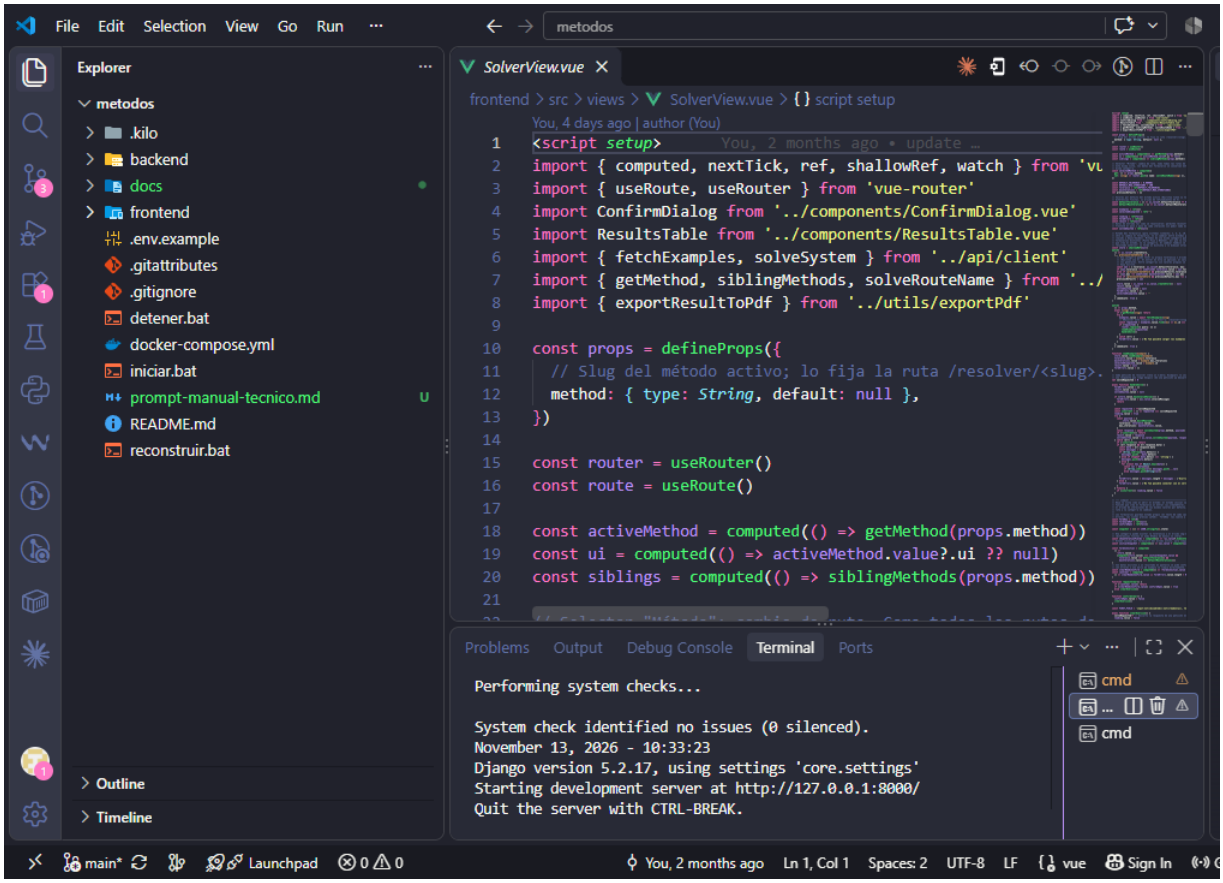
Frontend (segunda terminal):

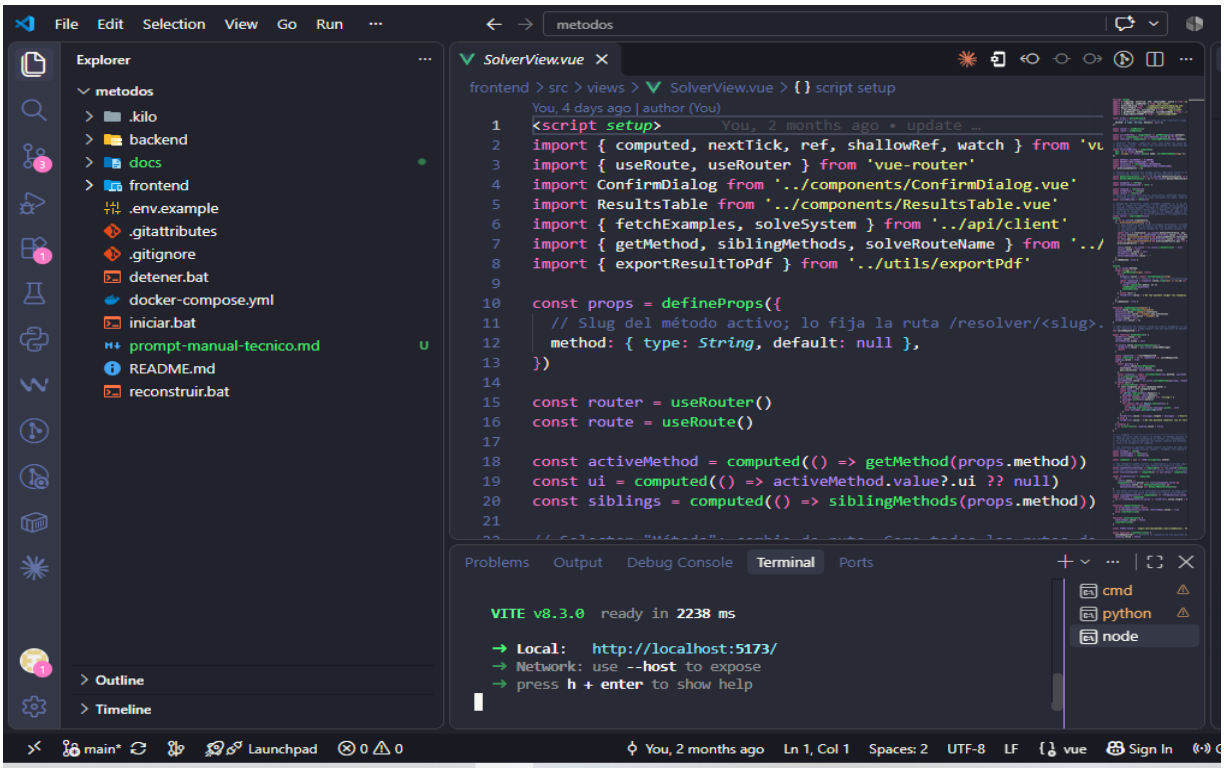
```
cd frontend
npm install                         # solo la primera vez o al cambiar
package.json
npm run dev
```

La aplicación queda en `http://localhost:5173`. El frontend lee `VITE_API_BASE_URL` de `frontend/.env.development` (`http://127.0.0.1:8000/api`) y el backend permite ese origen por CORS.

Proceso	Comando	URL
Backend	`python manage.py runserver 8000`	`http://127.0.0.1:8000/api/`
Frontend	`npm run dev`	`http://localhost:5173`

CAPTURA T-07 — Modo desarrollo con dos terminales





Modo desarrollo con dos terminales

11.3 Modo Docker

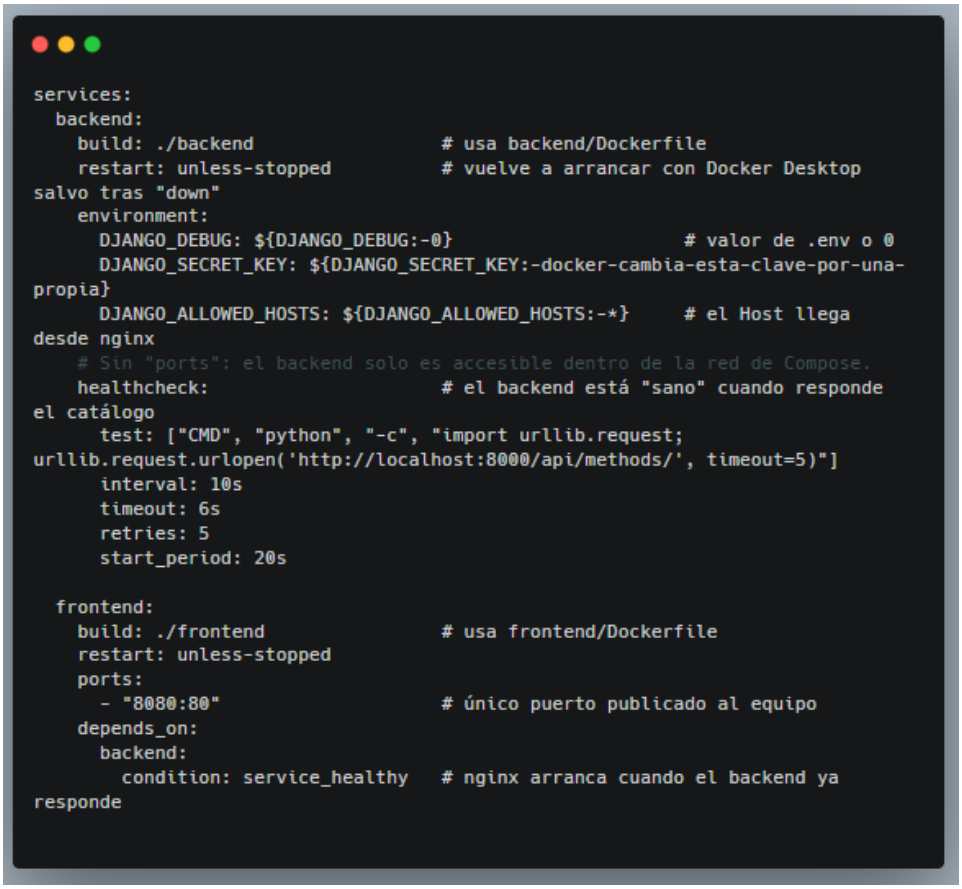
Un solo comando construye las imágenes y levanta los dos contenedores:

`docker compose up -d --build`

La primera construcción descarga imágenes base y dependencias y compila el frontend; tarda varios minutos. Las siguientes reutilizan la caché de capas. La aplicación queda en `http://localhost:8080`.

`docker-compose.yml` comentado.

`docker-compose.yml`

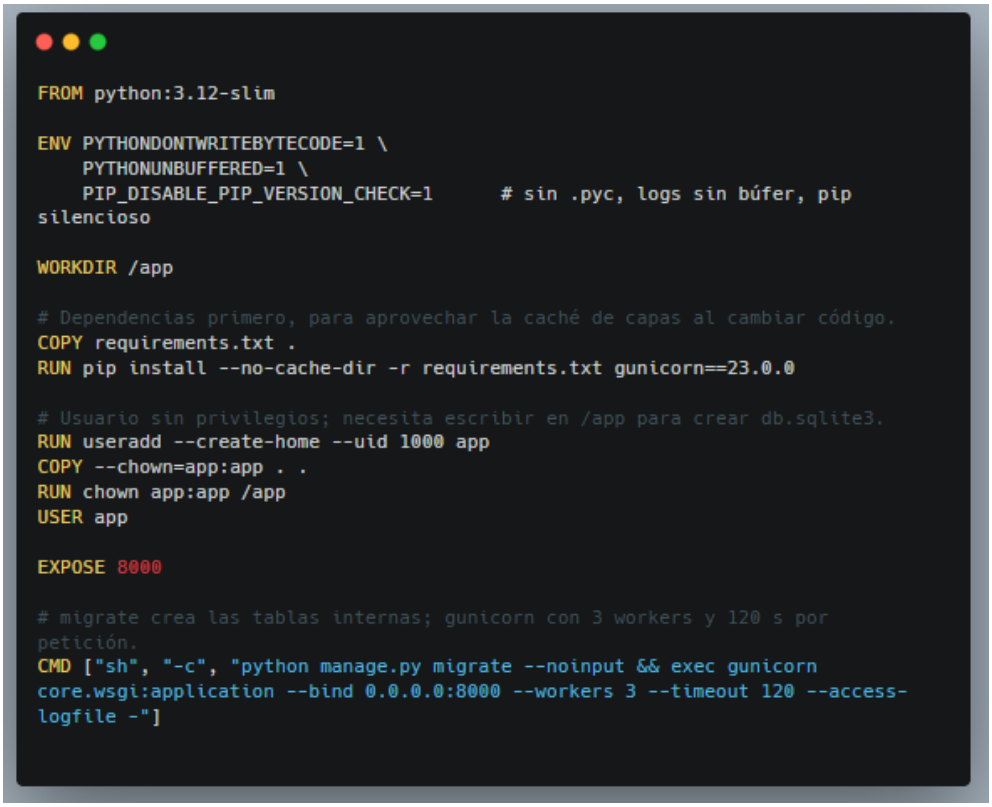


```yaml
services:
  backend:
    build: ./backend                # usa backend/Dockerfile
    restart: unless-stopped         # vuelve a arrancar con Docker Desktop
salvo tras "down"
    environment:
      DJANGO_DEBUG: ${DJANGO_DEBUG:-0}                    # valor de .env o 0
      DJANGO_SECRET_KEY: ${DJANGO_SECRET_KEY:-docker-cambia-esta-clave-por-una-
propia}
      DJANGO_ALLOWED_HOSTS: ${DJANGO_ALLOWED_HOSTS:-*}     # el Host llega
desde nginx
    # Sin "ports": el backend solo es accesible dentro de la red de Compose.
    healthcheck:                    # el backend está "sano" cuando responde
el catálogo
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:8000/api/methods/', timeout=5)"]
      interval: 10s
      timeout: 6s
      retries: 5
      start_period: 20s

  frontend:
    build: ./frontend               # usa frontend/Dockerfile
    restart: unless-stopped
    ports:
      - "8080:80"                   # único puerto publicado al equipo
    depends_on:
      backend:
        condition: service_healthy  # nginx arranca cuando el backend ya
responde
```

`backend/Dockerfile` comentado.

`backend/Dockerfile`



```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1      # sin .pyc, logs sin búfer, pip
silencioso

WORKDIR /app

# Dependencias primero, para aprovechar la caché de capas al cambiar código.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt gunicorn==23.0.0

# Usuario sin privilegios; necesita escribir en /app para crear db.sqlite3.
RUN useradd --create-home --uid 1000 app
COPY --chown=app:app . .
RUN chown app:app /app
USER app

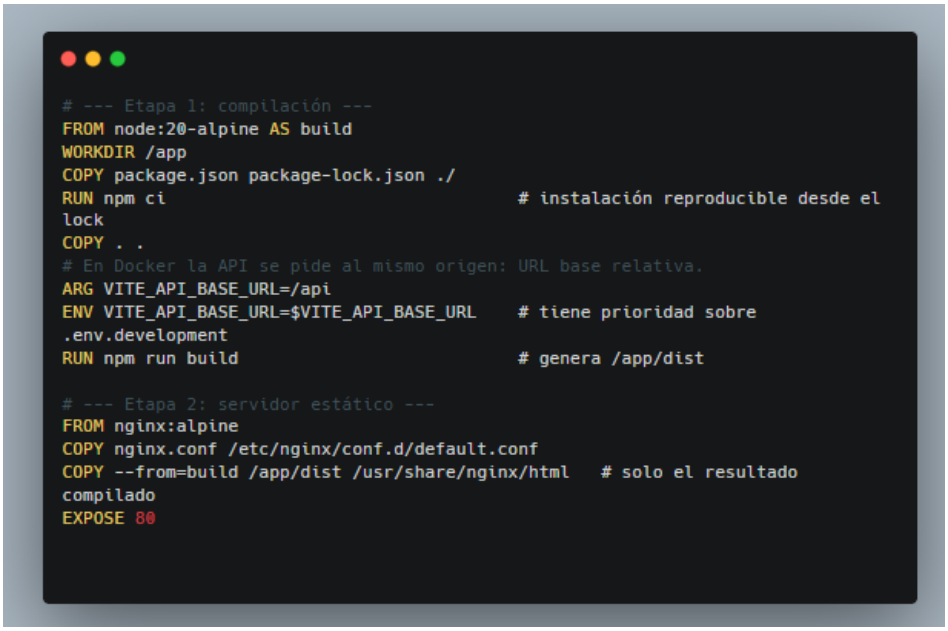
EXPOSE 8000

# migrate crea las tablas internas; gunicorn con 3 workers y 120 s por
petición.
CMD ["sh", "-c", "python manage.py migrate --noinput && exec gunicorn
core.wsgi:application --bind 0.0.0.0:8000 --workers 3 --timeout 120 --access-
logfile -"]
```

`backend/.dockerignore` excluye `.venv`, `__pycache__`, `*.pyc`, `db.sqlite3`, `.git`, `.env` y `.env.*` del contexto de construcción. Como la imagen del backend no incluye el frontend, las pruebas que comparan con los JSON de la teoría se omiten si se ejecutaran dentro del contenedor (`skipUnless(THEORY_EXAMPLE.exists(), ...)`).

`frontend/Dockerfile` comentado.

`frontend/Dockerfile`



```
# --- Etapa 1: compilación ---
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci                                  # instalación reproducible desde el
lock
COPY . .
# En Docker la API se pide al mismo origen: URL base relativa.
ARG VITE_API_BASE_URL=/api
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL    # tiene prioridad sobre
.env.development
RUN npm run build                           # genera /app/dist

# --- Etapa 2: servidor estático ---
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html   # solo el resultado
compilado
EXPOSE 80
```

La imagen final no contiene Node.js ni `node_modules`: solo nginx y los archivos de `dist/`.

`frontend/nginx.conf` comentado.

```
frontend/nginx.conf
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # Compresión de texto a partir de 1 KB.
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/css application/javascript text/javascript application/json
image/svg+xml;

    # API: proxy al contenedor del backend conservando la ruta (/api/...).
    location /api/ {
        proxy_pass http://backend:8000;          # "backend" es el nombre del
servicio en Compose
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;                 # igual que el timeout de
gunicorn
    }

    # Archivos con hash de Vite: nunca cambian de contenido, caché de un año.
    location /assets/ {
        add_header Cache-Control "public, max-age=31536000, immutable";
        try_files $uri =404;
    }

    # index.html sin caché, para que tras reconstruir se carguen los assets
nuevos.
    location = /index.html {
        add_header Cache-Control "no-cache";
    }

    # Fallback de la SPA: cualquier ruta desconocida devuelve index.html.
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

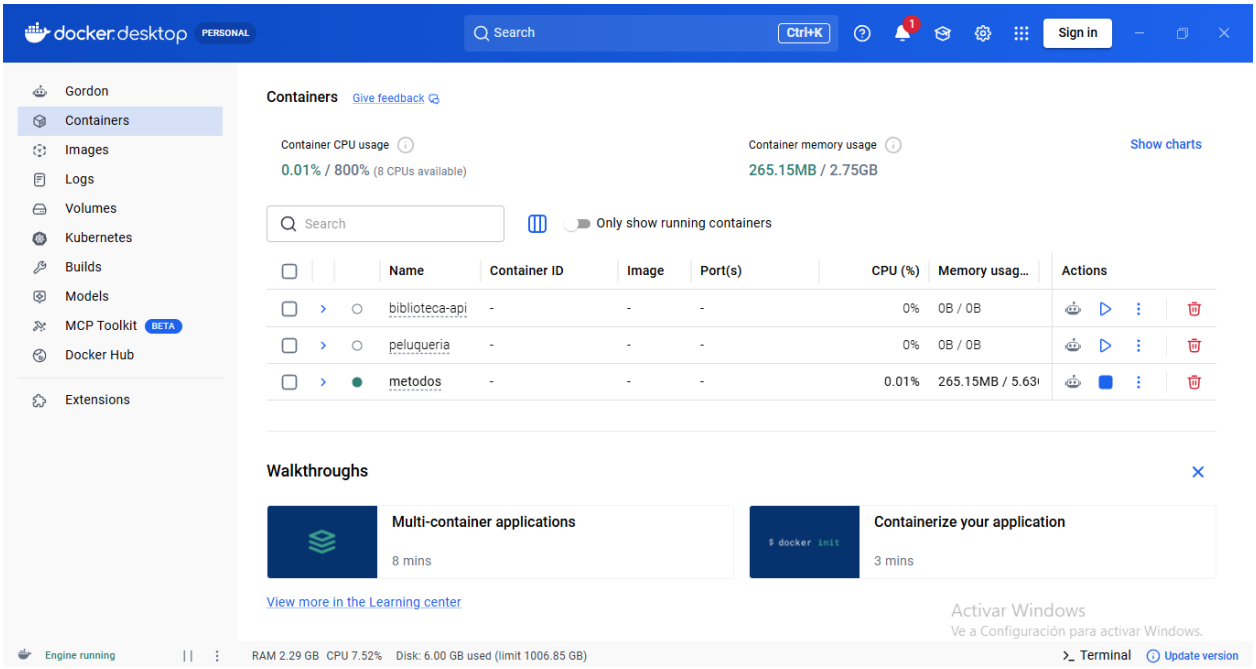
El *fallback* hace que una recarga en `/resolver/simpson` funcione: nginx no tiene ese archivo y entrega `index.html`, y Vue Router muestra la vista. La combinación de `immutable` en

`/assets/` y `no-cache` en `index.html` garantiza que tras una reconstrucción el navegador pida el `index.html` nuevo, que referencia archivos con otro hash.

Operación diaria.

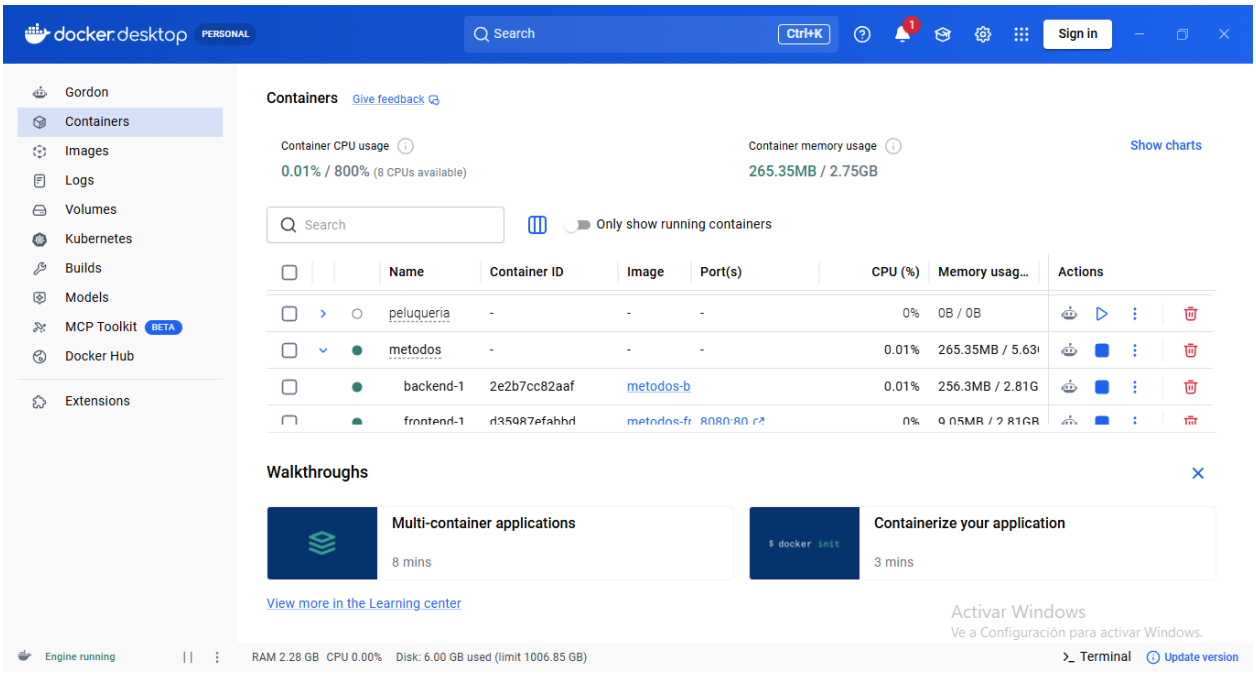
```
docker compose up -d                # iniciar
docker compose ps                   # estado de los contenedores
docker compose logs -f backend      # logs del backend (Ctrl+C para salir)
docker compose logs -f frontend     # logs de nginx
docker compose down                 # detener y eliminar contenedores
```

CAPTURA T-08 — Construcción con Docker Compose



Construcción con Docker Compose

CAPTURA T-09 — Contenedores en Docker Desktop



Contenedores en Docker Desktop

11.4 Scripts de Windows

Los tres scripts están en la raíz, usan finales de línea CRLF (forzados por `.gitattributes`) y se ejecutan con doble clic. Todos empiezan con `cd /d "%~dp0"`, de modo que funcionan aunque se lancen desde otra carpeta.

Script	Pasos
`iniciar.bat`	1. Comprueba que Docker responda (`docker info`); si no, pide abrir Docker Desktop y termina. 2. `docker compose up -d --wait --wait-timeout 300`: inicia y espera hasta 300 s a que los servicios estén sanos. 3. Si falla, sugiere `docker compose logs backend`. 4. Abre `http://localhost:8080` en el navegador.
`detener.bat`	`docker compose down`: detiene y elimina los contenedores (las imágenes se conservan).
`reconstruir.bat`	Igual que `iniciar.bat`, pero con `--build`: reconstruye las imágenes con el código actual antes de iniciar.

`iniciar.bat` no reconstruye: si se cambió código, hay que usar `reconstruir.bat`.

11.5 Variables de entorno

Backend (leídas por `backend/core/settings.py`):

Variable	Valor por defecto en `settings.py` (desarrollo)	Valor por defecto en Docker	Obligatoria	Descripción
`DJANGO_DEBUG`	`1` (activo)	`0`	No	`'1'` activa el modo depuración; cualquier otro valor lo desactiva.
`DJANGO_SECRET_KEY`	Clave `django-insecure-...` del repositorio	`docker-cambia-esta-clave-por-una-propia`	No (recomendada en producción)	Clave criptográfica de Django.
`DJANGO_ALLOWED_HOSTS`	Vacía (con `DEBUG`, Django admite `localhost`, `127.0.0.1`, `[::1]`)	`*`	Sí si `DEBUG=0`	Hosts admitidos, separados por comas.
`DJANGO_CSRF_TRUSTED_ORIGINS`	Vacía	No definida	No	Orígenes de confianza para CSRF, separados por comas.
`DJANGO_CORS_ALLOWED_ORIGINS`	`http://localhost:5173`, `http://127.0.0.1:5173`	No definida (usa el valor por defecto)	No	Orígenes permitidos por CORS, separados por comas.

En Docker se personalizan copiando `.env.example` como `.env` en la raíz (`.env` está en `.gitignore`). Compose sustituye `${VARIABLE:-defecto}` con el valor de `.env` o del entorno. El modo desarrollo no lee `.env`.

Frontend (sustituida por Vite en tiempo de compilación):

Variable	Desarrollo	Docker	Descripción
`VITE_API_BASE_URL`	`http://127.0.0.1:8000/api` (`frontend/.env.development`)	`/api` (`ARG` del `frontend/Dockerfile`)	URL base de la API para Axios.

11.6 Compilación del frontend

```
cd frontend
npm run build
```

`vite build` compila los SFC, empaqueta los módulos y escribe el resultado en `frontend/dist/` (ignorada por Git):

Contenido de `dist/`	Descripción
`index.html`	Documento único, con las referencias a los archivos con hash.
`assets/index-<hash>.js`	Paquete principal de la aplicación: 2 677,72 kB (750,45 kB con gzip).
`assets/index-<hash>.css`	Estilos, incluidos los de KaTeX: 155,89 kB (25,03 kB con gzip).
`assets/html2canvas-<hash>.js`, `assets/index.es-<hash>.js`, `assets/purify.es-<hash>.js`	Módulos cargados bajo demanda por jsPDF.
`assets/KaTeX_*.woff2`, `.woff`, `.ttf`	Fuentes de KaTeX.
`alternativa-integral.ico`, `icons.svg`	Copiados de `public/`.

Las cifras corresponden a la compilación del 13 de noviembre de 2026 (5,54 s). Vite emite una advertencia porque el paquete principal supera 500 kB; no es un error. Para revisar la compilación localmente: `npm run preview`.

11.7 Procedimiento de actualización

Cambio realizado	Modo desarrollo	Modo Docker
Código Python	`runserver` recarga solo	`docker compose up -d --build` o `reconstruir.bat`
Código del frontend (`src/`)	Vite recarga solo	`docker compose up -d --build` o `reconstruir.bat`
`backend/requirements.txt`	`pip install -r requirements.txt` y reiniciar `runserver`	Reconstruir (la capa de dependencias se invalida)
`frontend/package.json`	`npm install` y reiniciar `npm run dev`	Reconstruir (`npm ci` usa `package-lock.json`)
Ejemplos (`examples_data.py`)	Recarga automática	Reconstruir
Variables de `.env`	No aplica	`docker compose up -d` (recrea los contenedores afectados)
Tras actualizar desde Git (`git pull`)	`pip install`, `npm install`, `python manage.py test numeric_methods`	`docker compose up -d --build`

Después de reconstruir, el navegador obtiene el `index.html` nuevo gracias a `Cache-Control: no-cache`. Si muestra la versión anterior, forzar la recarga con Ctrl+F5 (sección 15).

11.8 Acceso desde otro equipo de la red

En modo Docker, el puerto 8080 se publica en todas las interfaces del equipo (`0.0.0.0:8080`). Desde otro dispositivo de la misma red:

1. Obtener la IP del equipo anfitrión con `ipconfig` (Windows), en «Dirección IPv4» del adaptador de la red Wi-Fi o Ethernet.
2. Abrir `http://<IP-del-equipo>:8080` en el otro dispositivo.
3. La primera vez, el Firewall de Windows puede pedir permiso para Docker; hay que permitirlo en redes privadas.

No se requiere cambiar `DJANGO_ALLOWED_HOSTS` (vale `*`) ni CORS (todo pasa por nginx en el mismo origen). En modo desarrollo, el acceso remoto no está configurado: `runserver 8000` escucha solo en `127.0.0.1`, y Vite solo en `localhost`.

12. Pruebas

12.1 Estrategia

Las pruebas automatizadas cubren el **backend**, donde se calcula todo número que ve el usuario (sección 4.4). Se usan `SimpleTestCase` de Django (sin base de datos) para los solvers y `APITestCase` de DRF para los endpoints.

Qué se prueba:

- **Valores calculados a mano o en fracciones exactas.** Primeras iteraciones de Jacobi, Gauss-Seidel, Newton y Bairstow; filas de las tablas de refinamiento comparadas con fracciones exactas (x^2, polinomio de Chapra, x^5); nodos y pesos de Gauss contra sus formas cerradas.
- **Propiedades de los métodos.** Razón de errores esperada (0.5^p), error real ≤ cota (+ redondeo), exactitud de Simpson para cúbicas y de Gauss para t^k con $k \leq 2m-1$, identidades de Richardson, recurrencia del trapecio igual a la fórmula directa.
- **Filas de parada de cada ejemplo precargado** y el mínimo de 6 iteraciones.

- **Casos borde.** Matrices sin dominancia, diagonal con ceros, jacobiano singular, raíces nulas y dobles, despejes ambiguos, tablas no uniformes, singularidades en los extremos o en el interior, límites de tiempo (simulados y reales).
- **Seguridad del parser.** Inyección de código, nombres no permitidos, límites de longitud, anidamiento, magnitud y exponente.
- **Endpoints.** Códigos 200, 400, 404, catálogo, ejemplos, vista previa.
- **Coherencia con la teoría.** Las respuestas guardadas en `frontend/src/methods/*/theory/*.json` deben coincidir con lo que el solver calcula hoy.
- **Presentación numérica.** Forma exacta y aproximada de h, decimales del paso a paso.

Qué no se prueba automáticamente, y por qué:

- **El frontend.** No hay pruebas unitarias ni de extremo a extremo de Vue: el proyecto no incluye Vitest ni Playwright como dependencias. La lógica del frontend es de presentación; se verifica con la rutina manual de 12.4.
- **El despliegue Docker.** Se verifica con el *healthcheck* de Compose y la rutina manual.
- **La generación del PDF.** Depende del navegador; se revisa a mano.

12.2 Cómo ejecutarlas

```
cd backend
python manage.py test numeric_methods             # toda la suite
python manage.py test numeric_methods -v 2        # con el nombre de cada test
python manage.py test numeric_methods.tests.test_trapezoid   # un solo módulo
```

Con el entorno virtual activado. La suite completa tarda 81,175 s en el equipo de desarrollo (Windows, Python 3.11.0); la duración varía con el equipo porque algunas pruebas lanzan procesos hijo para el límite de tiempo de la integración simbólica (en Windows se crean con *spawn*, más lento que el *fork* de Linux). Resultado esperado:

```
Ran 565 tests in 81.175s

OK
```

12.3 Cobertura por archivo

Cifras obtenidas con `python manage.py test numeric_methods -v 2` y contrastadas con el número de métodos `test_*` de cada archivo. Ninguna prueba se omitió en la ejecución local.

Archivo (`backend/numeric_methods/tests/`)	Tests	Qué cubre
`test_api.py`	13	Endpoints de los métodos lineales y de ejemplos.
`test_bairstow.py`	30	Divisiones sintéticas, primera iteración de Chapra a mano, ejemplos, raíces nulas, raíz doble, cierres directos, fallos, endpoint y comprobación de raíces.
`test_differentiation.py`	37	Diferencias progresiva y regresiva: fórmulas exactas, Burden y Faires, filas de parada, mínimo de 6 filas, no convergencia, razón ≈ 0.5, error ≤ cota, h exacto, errores 400, modo tabla, comprobación, explicaciones y teoría.
`test_differentiation_chart.py`	16	Gráfica de la aproximación de los 3 métodos (`chart.py`): pendiente del interpolante = D, tangente, ventanas, frases de lectura, `ln(x)` con ventana que cruza 0.
`test_dominance.py`	8	Dominancia diagonal y reordenamiento de filas.
`test_expressions.py`	49	Parser seguro, derivación y LaTeX.
`test_five_point.py`	41	Fórmula de 5 puntos: fracciones exactas, Burden y Faires, identidad de Richardson, razón ≈ 0.0625, $f^{(5)}$, modo tabla, errores 400, explicaciones, endpoint.
`test_fixed_point.py`	41	Punto Fijo: despeje automático, rechazos, actualización secuencial, endpoint y comprobación.
`test_gauss.py`	28	Cuadratura gaussiana: nodos y pesos exactos, simetría, exactitud polinómica, ejemplos, compuesta, cota, límite de tiempo de las derivadas, errores 400, teoría.
`test_h_display.py`	10	Forma exacta y decimal de h en tablas y paso a paso (diferenciación, trapecio, Simpson, Romberg, Gauss compuesta).
`test_isolation_steps.py`	20	Despeje paso a paso de Punto Fijo.
`test_newton.py`	38	Newton y regla de Cramer: iteraciones a mano, jacobiano singular, endpoint y comprobación.
`test_newton_steps.py`	16	Derivadas parciales paso a paso y sustitución en F.
`test_preview.py`	31	Endpoint de vista previa, logaritmos y límites de seguridad.
`test_registry.py`	15	Registro de métodos, catálogo, ejemplos por método, fusión de `meta` y campos comunes de la respuesta.
`test_romberg.py`	35	Romberg: tabla triangular en fracciones exactas,

Archivo (`backend/numeric_methods/tests/`)	Tests	Qué cubre
		identidades de columna, filas de parada, advertencia de función poco suave, comparación de costo, modo tabla, errores 400, teoría.
`test_simpson.py`	35	Simpson: fracciones exactas, exactitud para cúbicas, identidad $(4T_{2n} - T_n)/3$, reutilización de evaluaciones, modo tabla con combinación 1/3 + 3/8, errores 400, teoría.
`test_solvers.py`	23	Jacobi y Gauss-Seidel y su comprobación (residuos exactos, solución entera, orden original tras reordenar, no convergencia).
`test_three_point.py`	36	Fórmula de 3 puntos: fracciones exactas, Burden y Faires, identidades, modo tabla, errores 400, explicaciones, teoría.
`test_trapezoid.py`	43	Trapecio: fracciones exactas, recurrencia, filas de parada, error ≤ cota, integral exacta o de referencia, límite de tiempo, modo tabla, errores 400, explicaciones, teoría.
Total (20 archivos)	**565**	

12.4 Verificación manual

En el navegador (modo desarrollo o Docker):

1. Abrir la aplicación; la raíz debe redirigir a `/resolver/jacobi` y el pie decir «Sistemas lineales · Jacobi».
2. Abrir «Resolver ▾»: deben aparecer 5 categorías y 12 métodos. Buscar «integral» y comprobar que quedan los 4 de integración.
3. En cada categoría, cargar un ejemplo y pulsar «Resolver»: el resultado debe mostrar «Convergió», la gráfica y la tabla. Abrir el detalle de una iteración.
4. Exportar el PDF de un resultado y abrirlo: debe tener datos de entrada, resultado, gráfica (en colores claros aunque la pantalla esté en oscuro) y tabla.
5. Provocar un error (por ejemplo, un 0 en la diagonal de Jacobi): debe mostrarse el mensaje del backend.
6. Escribir una ecuación en Punto Fijo: la vista previa debe actualizarse al dejar de escribir.
7. Pulsar «Limpiar» con un resultado en pantalla: debe pedir confirmación y dejar el formulario como al abrir el método.
8. Cambiar el tema con el botón de la barra; recargar la página y comprobar que se conserva.

9. Abrir «Teoría ▾» y una página de teoría de cada categoría.
10. Abrir la consola del navegador (F12 → Consola): no debe haber errores.

En Docker, además:

1. `docker compose ps`: `backend` en estado `healthy`, `frontend` en `running` con `0.0.0.0:8080->80/tcp`.
2. Recargar con F5 en `http://localhost:8080/resolver/simpson`: debe cargar la vista (fallback de nginx).
3. `curl -sI http://localhost:8080/` debe incluir `Cache-Control: no-cache`.
4. `curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/api/methods/` debe imprimir `200`.

13. Rendimiento y límites

Límites numéricos.

Método	Entrada	Iteraciones o filas	Otros límites
Jacobi, Gauss-Seidel	$n \geq 3$; formulario hasta 12 variables	Mínimo 6, por defecto 100, sin máximo en el backend	Reordenamiento automático con $n \leq 8$
Punto Fijo	2 a 6 ecuaciones	Mínimo 6, por defecto 100	Grado ≤ 4 por ecuación en su variable
Newton	2 a 6 ecuaciones	Mínimo 6, por defecto 100	Cramer hasta 8 × 8
Bairstow	Grado 3 a 10	Mínimo 6, por defecto 100 **por factor**	Grado estimado ≤ 200 antes de expandir
Diferencias, 3 y 5 puntos	Tabla de 2/3/5 a 20 puntos	6 a 30 refinamientos (por defecto 25)	M con 401 muestras
Trapecio	Tabla de 2 a 30 puntos; n_0 de 1 a 1000	6 a 22 (por defecto 20)	$n_k \leq 2^{21}$; M_2 con 2001 muestras (401 por segmento en tabla)
Simpson	Tabla de 3 a 30 puntos; n_0 2–1000 (1/3), 3–999 (3/8)	6 a 22 (por defecto 20)	$n_k \leq 2^{21}$; M_4 con 2001 muestras
Romberg	Tabla de 3, 5, 9, 17 o 33 puntos	6 a 20 filas (por defecto 15)	Comparación de costo hasta 2^{22} evaluaciones
Gauss	Solo función; N de 1 a 50	m de 6 a 20 puntos (por defecto 12)	Cota solo para $m \leq 5$
Todas las	300 caracteres, 30		Números ≤ 10^{100},

Método	Entrada	Iteraciones o filas	Otros límites
expresiones	paréntesis, profundidad 40		exponentes ≤ 1000

Operaciones costosas y cómo se acotan.

Operación	Dónde	Acotación
Integración simbólica (`sympy.integrate`)	`solvers/integration/exact.py`	Proceso hijo con plazo de 3 s; si se agota, integral de referencia con `mpmath.quad`; caché de 256 antiderivadas por proceso.
Derivadas de orden alto (hasta $f^{(10)}$ para la cota de Gauss con $m = 5$)	`exact.derivatives`	Mismo proceso hijo y plazo; la cota se omite desde $m = 6$ y se marca «no disponible» si se agota el tiempo.
Despeje simbólico (`sympy.solve`)	`solvers/nonlinear/fixed_point.py`	Grado estimado ≤ 4 antes de llamar.
Muestreo para las cotas	Diferenciación (401), trapecio y Simpson (2001)	Evaluación con `lambdify` (floats), una vez por fila o por intervalo.
Determinantes por cofactores	`linalg/cramer.py`	$n \leq 8$; omite términos nulos.
Búsqueda de permutaciones dominantes	`solvers/linear/dominance.py`	$n \leq 8$; descarte temprano si alguna posición no admite fila.
Evaluación de f en refinamientos sucesivos	Trapecio, Romberg	Recurrencia que reutiliza los puntos ya evaluados; tope de 2^{21} segmentos.

Tamaño de las respuestas. Medido para los 84 pares método–ejemplo contra el contenedor Docker el 13 de noviembre de 2026 (todos respondieron 200):

Método	Mínimo	Máximo	Ejemplo más grande
`trapecio`	23 391 B	571 060 B	`trap-chapra`
`romberg`	10 167 B	556 213 B	`romb-raiz`
`simpson`	27 388 B	311 164 B	`simp-chapra`
`tres-puntos`	71 060 B	264 983 B	`tp-burden-xexp`
`cinco-puntos`	76 113 B	238 516 B	
`diferencias-finitas`	43 395 B	178 318 B	
`gauss`	33 357 B	62 301 B	

Método	Mínimo	Máximo	Ejemplo más grande
`bairstow`	6 882 B	43 397 B	
`newton`	6 789 B	16 514 B	
`jacobi`	2 464 B	13 434 B	
`gauss-seidel`	1 994 B	13 174 B	
`punto-fijo`	3 635 B	6 048 B	

Los tamaños son sin comprimir; nginx aplica gzip a `application/json`. Las respuestas grandes se deben al paso a paso completo de cada fila (LaTeX, sustituciones, puntos de las gráficas).

Tiempos de respuesta. En la misma medición, la petición más lenta tardó 1,58 s (`punto-fijo`, ejemplo `burden-faires-3x3`, por los despejes simbólicos), seguida de `gauss`, ejemplo `gauss-seno-x`, con 1,49 s. La primera petición del trapecio con un polinomio nuevo tardó 1,11 s y las repetidas, 0,24 s, por la caché de antiderivadas. Los tiempos dependen del equipo y del estado de la caché; se dan como referencia, no como garantía.

14. Guía de mantenimiento y extensión

14.1 Agregar un método a una categoría existente

Ejemplo: un método `regla-punto-medio` en la categoría `integration`.

1. **Solver.** Crear `backend/numeric_methods/solvers/integration/<archivo>.py` con `solve_<método>(data) -> SolverResult`. Reutilizar `rules.py` (si es una regla con pesos), `exact.py` y las utilidades del trapecio. Usar `MIN_ITERATIONS` o `MIN_REFINEMENTS` para el mínimo de filas. Lanzar `IntegrationValidationError` para entradas no aplicables.
2. **Serializer.** Si la entrada coincide con la de un método existente, reutilizar su serializer; si no, crear una subclase en `backend/numeric_methods/serializers/integration.py`.
3. **Registro.** Añadir un `MethodSpec` a `METHODS` en `backend/numeric_methods/registry.py`, con `slug`, `name`, `category`, `serializer`, `solver`, `summary` y `keywords`. La ruta `/api/solve/<slug>/`, el catálogo y el menú se actualizan solos.

4. **Ejemplos.** Añadir la lista a `backend/numeric_methods/examples_data.py` y la entrada en `EXAMPLES` (`test_registry.py` exige que todo método tenga ejemplos).
5. **Frontend.** Si el formulario y la vista de resultado de la categoría sirven, no hay que tocar nada. Si el método necesita algo propio, añadir una entrada en `methods: { '<slug>': { ... } }` de `frontend/src/methods/integration/index.js` (store, `configFields`, `resultView`…).
6. **Teoría.** Añadir la sección a `THEORY_SECTIONS` de la página de teoría de la categoría (sección 14.4). Sin ella, el método no aparece en «Teoría ▾».
7. **Pruebas.** Crear `backend/numeric_methods/tests/test_<método>.py` con valores calculados a mano, filas de parada de cada ejemplo, errores 400 y el endpoint.

14.2 Agregar una categoría nueva

1. **Categoría en el backend.** Añadir un `CategorySpec` a `CATEGORIES` en `registry.py` con `label`, `description`, `latex` y `order`.
2. **Paquete de solvers.** Crear `backend/numeric_methods/solvers/<categoría>/` con `__init__.py`, el solver y una subclase de `InputValidationError`. Devolver `SolverResult` con `category="<clave>"` y los datos propios en `meta` (sin repetir campos comunes: `to_dict()` lo rechaza).
3. **Serializer.** Crear `backend/numeric_methods/serializers/<categoría>.py`, devolviendo errores como listas planas por campo.
4. **Métodos y ejemplos.** Registrar los `MethodSpec` y sus ejemplos (pasos 3 y 4 de 14.1).
5. **Interfaz de categoría.** Crear `frontend/src/methods/<categoría>/` con:
   - `store.js`: una función fábrica que devuelva `state`, `loadExample`, `hasInvalidValues` y `buildPayload`;
   - `index.js`: el objeto que cumple el contrato de la sección 7.4;
   - los componentes de formulario, resultado y PDF (o reutilizar `ResultsTable` si la respuesta sigue el contrato común);
   - `theory/<Categoría>TheoryPage.vue` con `export const THEORY_SECTIONS`.
6. **Registro del frontend.** Importar la interfaz en `frontend/src/methods/registry.js` y añadirla a `CATEGORY_UI` con la misma clave que en el backend. Sin este paso, los métodos de la categoría se ignoran con un aviso en la consola.
7. **Pruebas** del backend y verificación manual (12.4).

14.3 Agregar una fórmula de derivación o una regla de integración

Fórmula de derivación. En `backend/numeric_methods/solvers/differentiation/formulas.py`:

1. Declarar una `Formula` con `key`, `name`, `symbol_latex`, `nodes` (pares desplazamiento–peso en el orden del numerador), `divisor`, `order`, `bound_derivative`, `bound_coefficient` (`Fraction`), `formula_latex`, `formula_text` y `bound_latex`.
2. Añadirla a `FORMULAS` y al diccionario de opciones del método (`CHOICES`, `THREE_POINT_CHOICES` o `FIVE_POINT_CHOICES`, o uno nuevo).
3. Si es un método nuevo, crear su serializer con `formula = ChoiceField(choices=list(<OPCIONES>))` y su solver sobre el motor de `finite_differences.py` (como hacen `three_point.py` y `five_point.py`).
4. Probar con un polinomio de grado `order` (error 0) y otro de grado `order + 1` (error igual a la cota).

Regla de integración. En `backend/numeric_methods/solvers/integration/rules.py`:

1. Declarar un `CompositeRule` con `factor` (`Fraction`), `ends`, `interior` (ciclo de pesos), `segments_multiple`, `order`, `bound_derivative`, `bound_divisor` y los textos LaTeX.
2. Aplicarla con `apply(rule, values, h)`; `weights(n)` genera los pesos sin código adicional.
3. Usarla desde un solver existente (como `simpson.py` usa `SIMPSON_13` y `SIMPSON_38`) o desde uno nuevo, y exponerla con `public_rule(rule)`.
4. Probar la exactitud para polinomios del grado adecuado y la razón 0.5^{order}.

14.4 Agregar una página de teoría

1. Crear el componente de la sección en `frontend/src/methods/<categoría>/theory/<Método>Section.vue`, usando `components/theory/AlgorithmSteps.vue`, `WorkedIterations.vue` y `MathFormula.vue`. Los textos largos pueden ir en un `*Content.js` aparte, como en las categorías existentes.
2. Registrarla en `THEORY_SECTIONS` de `<Categoría>TheoryPage.vue`: `'<slug>': <Método>Section`. La ruta `/teoria/<slug>` y la entrada en «Teoría ▾» se crean solas (`hasTheory`).

3. Si el ejemplo resuelto debe mostrar números del solver, guardar la respuesta real en `theory/<método>Example.json` y añadir una prueba en el backend que compare ese JSON con la respuesta actual (patrón de `test_trapezoid.py`, con `skipUnless` para entornos sin frontend).
4. Para un botón «Resolver este ejemplo», enlazar a `/resolver/<slug>?ejemplo=<id>`; `SolverView.vue` carga y resuelve ese ejemplo.

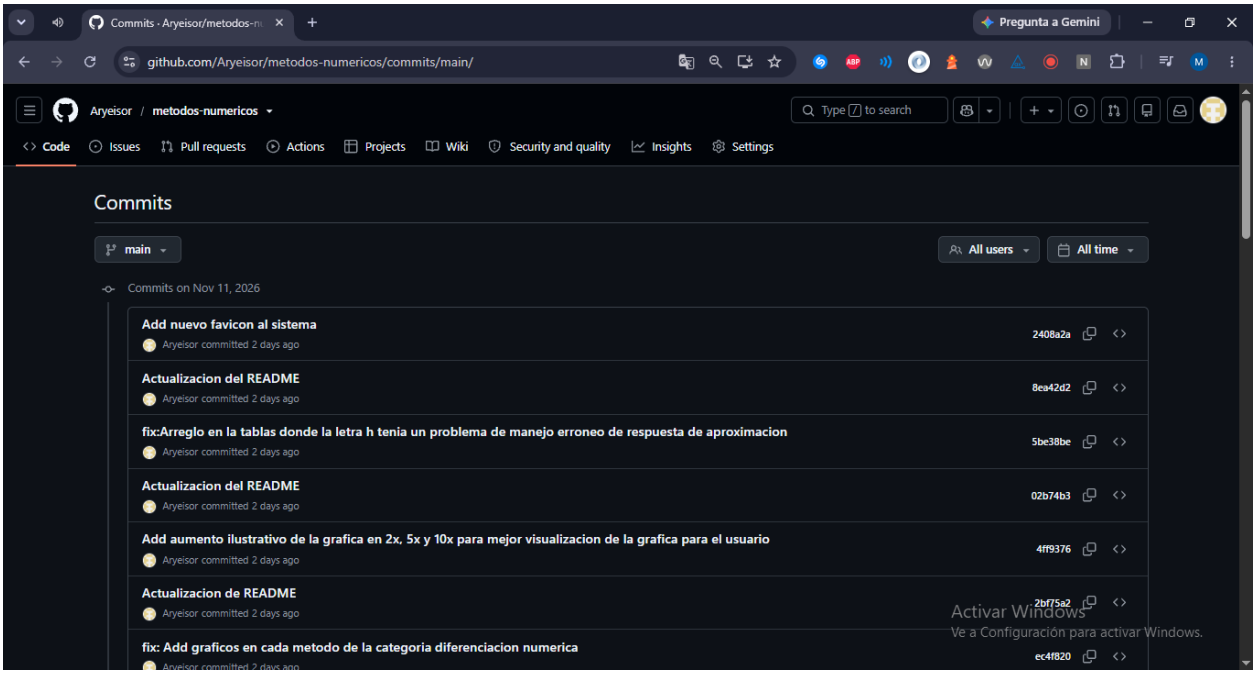
14.5 Agregar ejemplos precargados

1. Añadir un diccionario a la lista del método en `backend/numeric_methods/examples_data.py` (o usar su función auxiliar), con `id` único en todo el archivo, `name`, `description` y los campos del serializer, incluidos `tolerance` y `max_iterations`.
2. Si el ejemplo es de Jacobi o Gauss-Seidel, aparece en ambos (lista compartida).
3. Comprobar que el serializer lo acepta: `curl` o el selector de ejemplos.
4. Si las pruebas verifican las filas de parada de todos los ejemplos del método, añadir la cifra esperada en el test correspondiente.

14.6 Convenciones de código

Aspecto	Convención observada
Nombres en el código	Inglés para identificadores (`solve_trapezoid`, `buildPayload`, `createStore`); español para slugs, rutas de la interfaz (`/resolver`, `/teoria`) y textos.
Idioma de los textos	Español en la interfaz, en los mensajes de error de la API, en comentarios y docstrings.
Python	`snake_case`; constantes en `MAYÚSCULAS` al principio del módulo; `dataclass` inmutables para datos declarativos; solvers sin dependencias de Django.
JavaScript y Vue	`camelCase`; componentes en `PascalCase.vue`; `<script setup>`; un `index.js` por categoría con el contrato.
Estilos	Colores solo en `frontend/src/assets/theme.css` como variables CSS.
Errores	Listas planas de mensajes por campo; mensajes que explican la causa y sugieren la corrección.
Presentación numérica	6 decimales sin ceros finales; enteros sin decimales; – para valores no disponibles; notación científica solo desde 10^7; redondeo solo al mostrar (sección 7.8).
Commits	Mensajes en español, descriptivos; prefijos ocasionales `Add`, `fix:`.

CAPTURA T-10 — Historial de commits en GitHub



Historial de commits en GitHub

14.7 Lista de verificación antes de dar por terminado un cambio

- `python manage.py test numeric_methods` termina en `OK`.
- Los ejemplos precargados del método afectado se resuelven sin error en el navegador.
- Los mensajes de error nuevos están en español y aparecen en la interfaz.
- La respuesta sigue el contrato común (`method`, `category`, `converged`, `iterations`, `iterations_used`, `solution`, `variables`, `warnings`) y no redefine esos campos en `meta`.
- Los números nuevos se calculan en el backend, no en el frontend.
- La vista se ve bien en modo claro y oscuro, y el PDF sale en claro.
- La consola del navegador no muestra errores.

  `npm run build` termina sin errores.
- `docker compose up -d --build` levanta los dos servicios y el backend queda `healthy`.
- Si se cambió la respuesta de un método con teoría, se regeneraron los JSON de `theory/` y sus pruebas pasan.
- Se actualizaron el `README.md` y este manual si cambió algo documentado.

15. Solución de problemas

Síntoma	Causa probable	Solución
`docker compose up` falla con «port is already allocated» o «bind: address already in use» en 8080	Otro programa usa el puerto 8080	Cerrar ese programa, o cambiar `"8080:80"` por otro puerto (por ejemplo `"8090:80"`) en `docker-compose.yml` y abrir `http://localhost:8090`.
`error during connect` o `cannot find the file specified` al usar `docker`; `iniciar.bat` dice «Docker Desktop no esta iniciado»	Docker Desktop cerrado o arrancando	Abrir Docker Desktop, esperar a «Engine running» y repetir.
El navegador muestra la versión anterior tras cambiar código	No se reconstruyó la imagen, o el navegador conserva un `index.html` en caché	Ejecutar `reconstruir.bat` (o `docker compose up -d --build`); después, recargar con Ctrl+F5.
Un `.bat` no se ejecuta o muestra caracteres extraños; un `.sh` falla en el contenedor con `\r`	Finales de línea incorrectos (CRLF/LF) tras editar el archivo	`.bat` debe tener CRLF y `.sh` LF (`.gitattributes` lo fija). En VS Code, cambiar «LF»/«CRLF» en la barra de estado y guardar; o `git add --renormalize .`
`ModuleNotFoundError` en `runserver` o falla `npm run dev` tras actualizar el repositorio	Dependencias desactualizadas	`pip install -r requirements.txt` con el entorno virtual activado; `npm install` en `frontend/`. En Docker, reconstruir.
`npm run dev` falla con un error de versión de Node o de sintaxis	Node.js anterior a 20.19	Instalar Node.js 20.19+ o 22.12+.
En desarrollo, la consola muestra «blocked by CORS policy»	El frontend se abrió desde un origen distinto de `localhost:5173`/`127.0.0.1:5173`, o se cambió el puerto de Vite	Abrir `http://localhost:5173`, o añadir el origen con `DJANGO_CORS_ALLOWED_ORIGINS` (separado por comas) antes de lanzar `runserver`.
La interfaz dice «No fue posible conectar con el servidor»	Backend detenido o URL base incorrecta	En desarrollo, iniciar `runserver 8000`; comprobar `VITE_API_BASE_URL`. En Docker, `docker compose ps` y `docker compose logs backend`.
La vista dice «No fue posible cargar los métodos desde el servidor»	El catálogo no cargó al arrancar	Iniciar el backend y recargar la página (el catálogo se pide una vez al cargar).
`iniciar.bat` termina con «La aplicacion no	El backend no pasó el *healthcheck* en 300 s	`docker compose logs backend`; revisar errores de migración o de

Síntoma	Causa probable	Solución
pudo iniciarse»		importación.
`DisallowedHost` / 400 «Bad Request» del backend	El host no está en `DJANGO_ALLOWED_HOSTS` con `DEBUG=0`	Añadir el host a la variable o usar `*` detrás de nginx.
Una integral tarda varios segundos la primera vez	Integración simbólica en un proceso hijo (hasta 3 s; en Windows, más el arranque de *spawn*)	Comportamiento esperado; las repeticiones usan la caché.
Desde otro equipo no abre `http://<IP>:8080`	Firewall de Windows o red pública	Permitir Docker en redes privadas; comprobar la IP con `ipconfig`.

16. Glosario

Término	Definición
Antiderivada	Función F tal que $F' = f$; permite calcular la integral exacta $F(b) - F(a)$.
Bairstow	Método que extrae factores cuadráticos $x^2 - rx - s$ de un polinomio iterando sobre r y s.
Bundle	Archivo JavaScript que agrupa los módulos de la aplicación tras la compilación.
Caché de capas	Mecanismo de Docker que reutiliza los pasos de construcción que no cambiaron.
Catálogo	Lista de métodos que devuelve `GET /api/methods/`.
Comprobación	Sustitución de la solución en el problema original para mostrar el residuo.
Contrato de categoría	Forma fija del objeto que exporta `frontend/src/methods/<categoría>/index.js`.
Cota de error	Máximo teórico del error de truncamiento de una fórmula.
Cuadratura gaussiana	Integración que evalúa f en las raíces del polinomio de Legendre con pesos óptimos.
Debounce	Retraso que agrupa eventos rápidos (pulsaciones) en una sola acción.
Despeje	Reescritura de una ecuación para aislar una variable ($x_i = g_i(\dots)$).
Diferencia finita	Aproximación de la derivada por cocientes de diferencias de valores de la función.
Dominancia diagonal	Propiedad $\lvert a_{ii}\rvert > \sum_{j\neq i} \lvert a_{ij}\rvert$ en cada fila; condición suficiente de convergencia de Jacobi y Gauss-Seidel.
Error estimado	Diferencia entre dos aproximaciones consecutivas.
Error verdadero	Diferencia entre el valor exacto o de referencia y la aproximación.
Extrapolación de	Combinación de dos aproximaciones con pasos distintos que cancela el

Término	Definición
Richardson	término principal del error.
Fallback de SPA	Regla del servidor que devuelve `index.html` para cualquier ruta de la aplicación.
Gauss-Seidel	Método iterativo para sistemas lineales que usa cada valor nuevo apenas se calcula.
gzip	Compresión que nginx aplica a las respuestas de texto.
Healthcheck	Comando que Docker ejecuta periódicamente para saber si un servicio está listo.
HMR	*Hot Module Replacement*: recarga de módulos sin perder el estado en desarrollo.
Iteración	Aplicación de la regla de actualización de un método.
Jacobi	Método iterativo para sistemas lineales que usa solo los valores de la iteración anterior.
Jacobiano	Matriz de derivadas parciales $J_{ij} = \partial f_i / \partial x_j$.
Lista blanca	Conjunto cerrado de elementos permitidos; todo lo demás se rechaza.
Método de Newton	Método para sistemas no lineales que resuelve $J\,\Delta\mathbf{x} = -F$ en cada iteración.
Norma infinito	Máximo valor absoluto de las componentes de un vector.
Payload	Cuerpo JSON de una petición.
Proxy inverso	Servidor que recibe peticiones y las reenvía a otro (nginx hacia gunicorn).
Punto Fijo	Método que itera $x = g(x)$; aquí, con actualización secuencial para sistemas.
Refinamiento	Reducción del paso o aumento de segmentos entre filas consecutivas.
Registro	Estructura que enumera los métodos disponibles (`registry.py`, `registry.js`).
Regla de Cramer	Solución de $A\mathbf{x} = \mathbf{b}$ con $x_i = \det A_i / \det A$.
Regla de Simpson	Integración por parábolas (1/3) o cúbicas (3/8) sobre grupos de segmentos.
Regla del trapecio	Integración que une los puntos con rectas.
Residuo	Diferencia entre el lado izquierdo evaluado y el lado derecho de una ecuación.
Romberg	Integración que mejora trapecios sucesivos con extrapolación de Richardson.
Serializer	Clase de DRF que valida la entrada.
Slug	Identificador de un método en las URL.
Solver	Función que implementa un método numérico.
Store	Objeto con el estado de un formulario y sus operaciones (`createStore`).

Término	Definición
Tolerancia	Umbral de error para declarar convergencia.
WSGI	Interfaz entre gunicorn y Django.

Anexo Comandos de referencia rápida

Modo desarrollo: backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate.bat              # o Activate.ps1, o source
.venv/Scripts/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver 8000
```

Modo desarrollo: frontend

```
cd frontend
npm install
npm run dev
npm run build
npm run preview
```

Pruebas

```
cd backend
python manage.py test numeric_methods
python manage.py test numeric_methods -v 2
python manage.py test numeric_methods.tests.test_trapezoid
```

Docker

```
docker compose up -d --build            # primera vez o tras cambiar
código
docker compose up -d --build --wait     # igual, esperando a que los
servicios estén sanos
docker compose up -d                    # iniciar
docker compose ps                       # estado
docker compose logs -f backend          # logs del backend
docker compose logs -f frontend         # logs de nginx
docker compose down                     # detener
```

Scripts de Windows (doble clic en la raíz): `iniciar.bat`, `detener.bat`, `reconstruir.bat`.

API (Git Bash; en PowerShell usar `curl.exe`)

```
curl -s http://localhost:8080/api/methods/
curl -s "http://localhost:8080/api/examples/?method=trapecio"
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"A":[[4,1,1],[2,5,2],[1,2,4]],"b":[6,9,7]}' \
  http://localhost:8080/api/solve/jacobi/
curl -s -X POST -H "Content-Type: application/json" \
  -d '{"equation":"f1(x, y) = x^2 + x*y - 10","variables":["x","y"]}' \
```

```
  http://localhost:8080/api/expressions/preview
curl -sI http://localhost:8080/
```

Configuración

```
cp .env.example .env                                                    #
personalizar Docker
python -c "import secrets; print(secrets.token_urlsafe(50))" # generar
DJANGO_SECRET_KEY
ipconfig                                                                # IP del
equipo (Windows)
```

Anexo. Referencias

Documentación de las tecnologías

- Axios. (s. f.). *Axios documentation*. https://axios-http.com/docs/intro
- Chart.js. (s. f.). *Chart.js documentation* (versión 4). https://www.chartjs.org/docs/latest/
- Django Software Foundation. (s. f.). *Django documentation* (versión 5.2). https://docs.djangoproject.com/en/5.2/
- Docker Inc. (s. f.). *Docker Compose*. https://docs.docker.com/compose/
- Docker Inc. (s. f.). *Dockerfile reference*. https://docs.docker.com/reference/dockerfile/
- Encode OSS. (s. f.). *Django REST framework*. https://www.django-rest-framework.org/
- Gunicorn. (s. f.). *Gunicorn documentation*. https://docs.gunicorn.org/
- Johnson, A. (s. f.). *django-cors-headers*. https://github.com/adamchainz/django-cors-headers
- KaTeX. (s. f.). *KaTeX documentation*. https://katex.org/docs/
- Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., … Scopatz, A. (2017). SymPy: Symbolic computing in Python. *PeerJ Computer Science, 3*, e103. https://doi.org/10.7717/peerj-cs.103
- mpmath development team. (2023). *mpmath: A Python library for arbitrary-precision floating-point arithmetic* (versión 1.3.0). https://mpmath.org/
- nginx. (s. f.). *nginx documentation*. https://nginx.org/en/docs/
- Parallax. (s. f.). *jsPDF*. https://github.com/parallax/jsPDF
- Bengtsson, S. (s. f.). *jsPDF-AutoTable*. https://github.com/simonbengtsson/jsPDF-AutoTable
- Vite. (s. f.). *Vite documentation*. https://vite.dev/guide/
- Vue.js. (s. f.). *Vue.js documentation*. https://vuejs.org/guide/introduction.html
- Vue.js. (s. f.). *Vue Router documentation*. https://router.vuejs.org/

Bibliografía de métodos numéricos

- Bairstow, L. (1920). *Investigations relating to the stability of the aeroplane* (Reports and Memoranda No. 154). Advisory Committee for Aeronautics.
- Burden, R. L., Faires, J. D., & Burden, A. M. (2016). *Numerical analysis* (10.ª ed.). Cengage Learning.
- Chapra, S. C., & Canale, R. P. (2015). *Métodos numéricos para ingenieros* (7.ª ed.). McGraw-Hill.
- Hildebrand, F. B. (1974). *Introduction to numerical analysis* (2.ª ed.). McGraw-Hill.
- Mathews, J. H., & Fink, K. D. (2000). *Métodos numéricos con MATLAB* (3.ª ed.). Prentice Hall.
- Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). *Numerical recipes: The art of scientific computing* (3.ª ed.). Cambridge University Press.
- Romberg, W. (1955). Vereinfachte numerische Integration. *Det Kongelige Norske Videnskabers Selskabs Forhandlinger, 28*(7), 30–36.